

PID Control

1.0

Generated by Doxygen 1.17.0

1 PID Control Library	1
1.1 Introduction	1
1.2 Features	1
1.3 Library Organization	2
1.4 Controllers	2
1.4.1 Proportional Controller	2
1.4.2 Integral Controller	2
1.4.3 Derivative Controller	2
1.4.4 PI Controller	2
1.4.5 PD Controller	2
1.4.6 PID Controller	3
1.5 Filtering	3
1.6 Feed-Forward Control	3
1.7 Documentation	3
1.8 License	3
2 PID	5
3 Directory Hierarchy	7
3.1 Directories	7
4 Namespace Index	9
4.1 Namespace List	9
5 Class Index	11
5.1 Class List	11
6 File Index	13
6.1 File List	13
7 Directory Documentation	15
7.1 include/derivative Directory Reference	15
7.2 src/derivative Directory Reference	16
7.3 doc Directory Reference	16
7.4 include Directory Reference	16
7.5 include/integral Directory Reference	17
7.6 src/integral Directory Reference	18
7.7 include/pd Directory Reference	18
7.8 src/pd Directory Reference	19
7.9 include/pi Directory Reference	19
7.10 src/pi Directory Reference	20
7.11 include/pid Directory Reference	20
7.12 src/pid Directory Reference	21
7.13 include/proportional Directory Reference	22

7.14 src/proportional Directory Reference	22
7.15 src Directory Reference	23
7.16 include/utility Directory Reference	23
8 Namespace Documentation	25
8.1 control_system Namespace Reference	25
8.1.1 Detailed Description	25
8.2 control_system::derivative Namespace Reference	25
8.3 control_system::integral Namespace Reference	25
8.4 control_system::pd Namespace Reference	26
8.5 control_system::pi Namespace Reference	26
8.6 control_system::pid Namespace Reference	26
8.7 control_system::proportional Namespace Reference	26
8.8 derivative Namespace Reference	26
8.8.1 Detailed Description	27
8.9 integral Namespace Reference	27
8.9.1 Detailed Description	27
8.10 pd Namespace Reference	27
8.10.1 Detailed Description	27
8.11 pi Namespace Reference	27
8.11.1 Detailed Description	27
8.12 pid Namespace Reference	27
8.12.1 Detailed Description	27
8.13 proportional Namespace Reference	27
8.13.1 Detailed Description	27
9 Class Documentation	29
9.1 control_system::derivative::D< T > Class Template Reference	29
9.1.1 Detailed Description	30
9.1.2 Constructor & Destructor Documentation	30
9.1.2.1 D()	30
9.1.3 Member Function Documentation	31
9.1.3.1 get_dt()	31
9.1.3.2 get_e_k_1()	31
9.1.3.3 get_Kd()	31
9.1.3.4 get_u_max()	31
9.1.3.5 init()	32
9.1.3.6 merge()	32
9.1.3.7 reset()	32
9.1.3.8 set_dt()	33
9.1.3.9 set_Kd()	33
9.1.3.10 set_param()	33
9.1.3.11 set_u_max()	34

9.1.3.12 update()	34
9.1.4 Member Data Documentation	34
9.1.4.1 dt	34
9.1.4.2 e_k_1	35
9.1.4.3 Kd	35
9.1.4.4 start	35
9.1.4.5 u_max	35
9.2 control_system::derivative::D_LPF_1< T > Class Template Reference	36
9.2.1 Detailed Description	37
9.2.2 Constructor & Destructor Documentation	38
9.2.2.1 D_LPF_1()	38
9.2.3 Member Function Documentation	38
9.2.3.1 get_dt()	38
9.2.3.2 get_e_k_1()	38
9.2.3.3 get_fc()	39
9.2.3.4 get_Kd()	39
9.2.3.5 get_u_k_1()	39
9.2.3.6 get_u_max()	39
9.2.3.7 init()	40
9.2.3.8 merge()	40
9.2.3.9 reset()	40
9.2.3.10 set_dt()	41
9.2.3.11 set_fc()	41
9.2.3.12 set_Kd()	42
9.2.3.13 set_param()	42
9.2.3.14 set_u_max()	42
9.2.3.15 update()	43
9.2.4 Member Data Documentation	43
9.2.4.1 dt	43
9.2.4.2 e_k_1	43
9.2.4.3 Kd	44
9.2.4.4 start	44
9.2.4.5 tau	44
9.2.4.6 u_k_1	44
9.2.4.7 u_max	44
9.3 control_system::integral::I< T > Class Template Reference	45
9.3.1 Detailed Description	46
9.3.2 Constructor & Destructor Documentation	46
9.3.2.1 I()	46
9.3.3 Member Function Documentation	46
9.3.3.1 get_dt()	46
9.3.3.2 get_Ki()	47

9.3.3.3 <code>get_u_k_1()</code>	47
9.3.3.4 <code>get_u_max()</code>	47
9.3.3.5 <code>init()</code>	48
9.3.3.6 <code>merge()</code>	48
9.3.3.7 <code>reset()</code>	48
9.3.3.8 <code>set_dt()</code>	49
9.3.3.9 <code>set_Ki()</code>	49
9.3.3.10 <code>set_param()</code>	49
9.3.3.11 <code>set_u_0()</code>	50
9.3.3.12 <code>set_u_max()</code>	50
9.3.3.13 <code>update()</code>	51
9.3.4 Member Data Documentation	51
9.3.4.1 <code>dt</code>	51
9.3.4.2 <code>Ki</code>	51
9.3.4.3 <code>u_k_1</code>	52
9.3.4.4 <code>u_max</code>	52
9.4 <code>control_system::pd::PD< T ></code> Class Template Reference	52
9.4.1 Detailed Description	53
9.4.2 Constructor & Destructor Documentation	54
9.4.2.1 <code>PD()</code>	54
9.4.3 Member Function Documentation	54
9.4.3.1 <code>get_dt()</code>	54
9.4.3.2 <code>get_e_k_1()</code>	55
9.4.3.3 <code>get_Kd()</code>	55
9.4.3.4 <code>get_Kp()</code>	55
9.4.3.5 <code>get_u_max()</code>	55
9.4.3.6 <code>init()</code>	56
9.4.3.7 <code>merge()</code>	56
9.4.3.8 <code>reset()</code>	56
9.4.3.9 <code>set_dt()</code>	57
9.4.3.10 <code>set_Kd()</code>	57
9.4.3.11 <code>set_Kp()</code>	58
9.4.3.12 <code>set_param()</code>	58
9.4.3.13 <code>set_u_max()</code>	58
9.4.3.14 <code>update()</code>	59
9.4.4 Member Data Documentation	59
9.4.4.1 <code>dt</code>	59
9.4.4.2 <code>e_k_1</code>	59
9.4.4.3 <code>Kd</code>	60
9.4.4.4 <code>Kp</code>	60
9.4.4.5 <code>start</code>	60
9.4.4.6 <code>u_max</code>	60

9.5 control_system::pd::PD_LPF_1< T > Class Template Reference	60
9.5.1 Detailed Description	62
9.5.2 Constructor & Destructor Documentation	62
9.5.2.1 PD_LPF_1()	62
9.5.3 Member Function Documentation	63
9.5.3.1 get_dt()	63
9.5.3.2 get_e_k_1()	63
9.5.3.3 get_fc()	63
9.5.3.4 get_Kd()	63
9.5.3.5 get_Kp()	64
9.5.3.6 get_u_k_1()	64
9.5.3.7 get_u_max()	64
9.5.3.8 init()	65
9.5.3.9 merge()	65
9.5.3.10 reset()	66
9.5.3.11 set_dt()	66
9.5.3.12 set_fc()	66
9.5.3.13 set_Kd()	67
9.5.3.14 set_Kp()	67
9.5.3.15 set_param()	67
9.5.3.16 set_u_max()	68
9.5.3.17 update()	68
9.5.4 Member Data Documentation	69
9.5.4.1 dt	69
9.5.4.2 e_k_1	69
9.5.4.3 Kd	69
9.5.4.4 Kp	69
9.5.4.5 start	69
9.5.4.6 tau	70
9.5.4.7 u_k_1	70
9.5.4.8 u_max	70
9.6 control_system::pi::PI< T > Class Template Reference	70
9.6.1 Detailed Description	72
9.6.2 Constructor & Destructor Documentation	72
9.6.2.1 PI()	72
9.6.3 Member Function Documentation	72
9.6.3.1 get_dt()	72
9.6.3.2 get_Ki()	73
9.6.3.3 get_Kp()	73
9.6.3.4 get_u_k_1()	73
9.6.3.5 get_u_max()	73
9.6.3.6 init()	74

9.6.3.7 merge()	74
9.6.3.8 reset()	75
9.6.3.9 set_dt()	75
9.6.3.10 set_Ki()	75
9.6.3.11 set_Kp()	76
9.6.3.12 set_param()	76
9.6.3.13 set_u_0()	77
9.6.3.14 set_u_max()	77
9.6.3.15 update()	77
9.6.4 Member Data Documentation	78
9.6.4.1 dt	78
9.6.4.2 e_k_1	78
9.6.4.3 Ki	78
9.6.4.4 Kp	78
9.6.4.5 start	78
9.6.4.6 u_k_1	79
9.6.4.7 u_max	79
9.7 control_system::pid::PID< T > Class Template Reference	79
9.7.1 Detailed Description	81
9.7.2 Constructor & Destructor Documentation	81
9.7.2.1 PID()	81
9.7.3 Member Function Documentation	81
9.7.3.1 get_dt()	81
9.7.3.2 get_e_k_1()	82
9.7.3.3 get_e_k_2()	82
9.7.3.4 get_Kd()	82
9.7.3.5 get_Ki()	82
9.7.3.6 get_Kp()	83
9.7.3.7 get_u_k_1()	83
9.7.3.8 get_u_max()	83
9.7.3.9 init()	84
9.7.3.10 merge()	84
9.7.3.11 reset()	85
9.7.3.12 set_dt()	85
9.7.3.13 set_Kd()	85
9.7.3.14 set_Ki()	86
9.7.3.15 set_Kp()	86
9.7.3.16 set_param()	86
9.7.3.17 set_u_0()	87
9.7.3.18 set_u_max()	87
9.7.3.19 update()	88
9.7.4 Member Data Documentation	88

9.7.4.1 dt	88
9.7.4.2 e_k_1	88
9.7.4.3 e_k_2	89
9.7.4.4 Kd	89
9.7.4.5 Ki	89
9.7.4.6 Kp	89
9.7.4.7 start	89
9.7.4.8 u_k_1	90
9.7.4.9 u_max	90
9.8 control_system::pid::PID_GS< T > Class Template Reference	90
9.8.1 Detailed Description	93
9.8.2 Constructor & Destructor Documentation	93
9.8.2.1 PID_GS()	93
9.8.3 Member Function Documentation	94
9.8.3.1 get_D()	94
9.8.3.2 get_d_filter()	94
9.8.3.3 get_dt()	94
9.8.3.4 get_e_k_1()	94
9.8.3.5 get_fc()	95
9.8.3.6 get_ff()	95
9.8.3.7 get_I()	95
9.8.3.8 get_I_max()	95
9.8.3.9 get_Kd()	96
9.8.3.10 get_Ki()	96
9.8.3.11 get_Kp()	96
9.8.3.12 get_P()	96
9.8.3.13 get_u()	97
9.8.3.14 get_u_max()	97
9.8.3.15 init()	97
9.8.3.16 interpolate()	98
9.8.3.17 merge()	99
9.8.3.18 reset()	99
9.8.3.19 set_d_filter()	99
9.8.3.20 set_dt()	100
9.8.3.21 set_fc()	100
9.8.3.22 set_ff()	100
9.8.3.23 set_gain_lookup_table()	101
9.8.3.24 set_I_max()	101
9.8.3.25 set_Kd()	102
9.8.3.26 set_Ki()	102
9.8.3.27 set_Kp()	102
9.8.3.28 set_param()	103

9.8.3.29 set_u_max()	103
9.8.3.30 update()	104
9.8.3.31 update_gain()	104
9.8.4 Member Data Documentation	105
9.8.4.1 D	105
9.8.4.2 d_filter	105
9.8.4.3 dt	105
9.8.4.4 e_k_1	105
9.8.4.5 fc	106
9.8.4.6 I	106
9.8.4.7 I_max	106
9.8.4.8 Kd	106
9.8.4.9 Kd_arr	106
9.8.4.10 Ki	107
9.8.4.11 Ki_arr	107
9.8.4.12 Kp	107
9.8.4.13 Kp_arr	107
9.8.4.14 len_arr	107
9.8.4.15 lpf	107
9.8.4.16 op_cond_arr	108
9.8.4.17 P	108
9.8.4.18 start	108
9.8.4.19 u	108
9.8.4.20 u_ff	108
9.8.4.21 u_max	109
9.9 control_system::pid::PID_LPF_1< T > Class Template Reference	109
9.9.1 Detailed Description	111
9.9.2 Constructor & Destructor Documentation	111
9.9.2.1 PID_LPF_1()	111
9.9.3 Member Function Documentation	112
9.9.3.1 get_dt()	112
9.9.3.2 get_e_k_1()	112
9.9.3.3 get_e_k_2()	112
9.9.3.4 get_fc()	112
9.9.3.5 get_Kd()	113
9.9.3.6 get_Ki()	113
9.9.3.7 get_Kp()	113
9.9.3.8 get_u_k_1()	113
9.9.3.9 get_u_max()	114
9.9.3.10 init()	114
9.9.3.11 merge()	115
9.9.3.12 reset()	115

9.9.3.13 set_dt()	115
9.9.3.14 set_fc()	116
9.9.3.15 set_Kd()	116
9.9.3.16 set_Ki()	116
9.9.3.17 set_Kp()	117
9.9.3.18 set_param()	117
9.9.3.19 set_u_0()	118
9.9.3.20 set_u_max()	118
9.9.3.21 update()	118
9.9.4 Member Data Documentation	119
9.9.4.1 dt	119
9.9.4.2 e_k_1	119
9.9.4.3 e_k_2	119
9.9.4.4 Kd	119
9.9.4.5 Ki	119
9.9.4.6 Kp	120
9.9.4.7 start	120
9.9.4.8 tau	120
9.9.4.9 u_k_1	120
9.9.4.10 u_k_2	120
9.9.4.11 u_max	121
9.10 control_system::pid::PID_LPF_1_FF< T > Class Template Reference	121
9.10.1 Detailed Description	123
9.10.2 Constructor & Destructor Documentation	123
9.10.2.1 PID_LPF_1_FF()	123
9.10.3 Member Function Documentation	124
9.10.3.1 get_dt()	124
9.10.3.2 get_e_k_1()	124
9.10.3.3 get_e_k_2()	124
9.10.3.4 get_fc()	124
9.10.3.5 get_ff()	125
9.10.3.6 get_Kd()	125
9.10.3.7 get_Ki()	125
9.10.3.8 get_Kp()	125
9.10.3.9 get_u_k_1()	126
9.10.3.10 get_u_max()	126
9.10.3.11 init()	126
9.10.3.12 merge()	127
9.10.3.13 reset()	127
9.10.3.14 set_dt()	128
9.10.3.15 set_fc()	128
9.10.3.16 set_ff()	128

9.10.3.17 set_Kd()	129
9.10.3.18 set_Ki()	129
9.10.3.19 set_Kp()	129
9.10.3.20 set_param()	130
9.10.3.21 set_u_0()	130
9.10.3.22 set_u_max()	131
9.10.3.23 update()	131
9.10.4 Member Data Documentation	132
9.10.4.1 dt	132
9.10.4.2 e_k_1	132
9.10.4.3 e_k_2	132
9.10.4.4 Kd	132
9.10.4.5 Ki	132
9.10.4.6 Kp	133
9.10.4.7 start	133
9.10.4.8 tau	133
9.10.4.9 u_ff	133
9.10.4.10 u_k_1	133
9.10.4.11 u_k_2	134
9.10.4.12 u_max	134
9.11 control_system::pid::PID_P< T > Class Template Reference	134
9.11.1 Detailed Description	136
9.11.2 Constructor & Destructor Documentation	137
9.11.2.1 PID_P()	137
9.11.3 Member Function Documentation	137
9.11.3.1 get_D()	137
9.11.3.2 get_d_filter()	137
9.11.3.3 get_dt()	138
9.11.3.4 get_e_k_1()	138
9.11.3.5 get_fc()	138
9.11.3.6 get_ff()	138
9.11.3.7 get_I()	139
9.11.3.8 get_I_max()	139
9.11.3.9 get_Kd()	139
9.11.3.10 get_Ki()	139
9.11.3.11 get_Kp()	140
9.11.3.12 get_P()	140
9.11.3.13 get_u()	140
9.11.3.14 get_u_max()	140
9.11.3.15 init()	141
9.11.3.16 merge()	141
9.11.3.17 reset()	142

9.11.3.18 set_d_filter()	142
9.11.3.19 set_dt()	142
9.11.3.20 set_fc()	143
9.11.3.21 set_ff()	143
9.11.3.22 set_I_max()	143
9.11.3.23 set_Kd()	144
9.11.3.24 set_Ki()	144
9.11.3.25 set_Kp()	144
9.11.3.26 set_param()	145
9.11.3.27 set_u_max()	145
9.11.3.28 update()	146
9.11.4 Member Data Documentation	146
9.11.4.1 D	146
9.11.4.2 d_filter	147
9.11.4.3 dt	147
9.11.4.4 e_k_1	147
9.11.4.5 fc	147
9.11.4.6 I	147
9.11.4.7 I_max	148
9.11.4.8 Kd	148
9.11.4.9 Ki	148
9.11.4.10 Kp	148
9.11.4.11 lpf	148
9.11.4.12 P	148
9.11.4.13 start	149
9.11.4.14 u	149
9.11.4.15 u_ff	149
9.11.4.16 u_max	149
9.12 control_system::pid::PID_S< T > Class Template Reference	149
9.12.1 Detailed Description	152
9.12.2 Constructor & Destructor Documentation	152
9.12.2.1 PID_S()	152
9.12.3 Member Function Documentation	153
9.12.3.1 get_D()	153
9.12.3.2 get_d_filter()	153
9.12.3.3 get_dt()	153
9.12.3.4 get_e_k_1()	153
9.12.3.5 get_fc()	154
9.12.3.6 get_ff()	154
9.12.3.7 get_I()	154
9.12.3.8 get_I_max()	154
9.12.3.9 get_Kp()	155

9.12.3.10	get_P()	155
9.12.3.11	get_T_d()	155
9.12.3.12	get_T_i()	155
9.12.3.13	get_u()	156
9.12.3.14	get_u_max()	156
9.12.3.15	init()	156
9.12.3.16	merge()	157
9.12.3.17	reset()	157
9.12.3.18	set_d_filter()	158
9.12.3.19	set_dt()	158
9.12.3.20	set_fc()	159
9.12.3.21	set_ff()	159
9.12.3.22	set_l_max()	159
9.12.3.23	set_Kp()	160
9.12.3.24	set_param()	160
9.12.3.25	set_Td()	161
9.12.3.26	set_Ti()	161
9.12.3.27	set_u_max()	161
9.12.3.28	update()	162
9.12.4	Member Data Documentation	162
9.12.4.1	D	162
9.12.4.2	d_filter	162
9.12.4.3	dt	163
9.12.4.4	e_k_1	163
9.12.4.5	fc	163
9.12.4.6	l	163
9.12.4.7	l_max	163
9.12.4.8	Kp	164
9.12.4.9	lpf	164
9.12.4.10	P	164
9.12.4.11	start	164
9.12.4.12	T_d	164
9.12.4.13	T_i	165
9.12.4.14	u	165
9.12.4.15	u_ff	165
9.12.4.16	u_max	165
9.13	control_system::proportional::P< T > Class Template Reference	165
9.13.1	Detailed Description	166
9.13.2	Constructor & Destructor Documentation	167
9.13.2.1	P()	167
9.13.3	Member Function Documentation	167
9.13.3.1	get_Kp()	167

9.13.3.2 <code>get_u_max()</code>	167
9.13.3.3 <code>init()</code>	168
9.13.3.4 <code>reset()</code>	168
9.13.3.5 <code>set_Kp()</code>	168
9.13.3.6 <code>set_param()</code>	169
9.13.3.7 <code>set_u_max()</code>	169
9.13.3.8 <code>update()</code>	169
9.13.4 Member Data Documentation	170
9.13.4.1 <code>Kp</code>	170
9.13.4.2 <code>u_max</code>	170
10 File Documentation	171
10.1 <code>doc/mainpage.dox</code> File Reference	171
10.2 <code>include/control_system.h</code> File Reference	171
10.2.1 Detailed Description	171
10.3 <code>control_system.h</code>	172
10.4 <code>include/derivative/d.h</code> File Reference	173
10.4.1 Detailed Description	174
10.5 <code>d.h</code>	175
10.6 <code>include/derivative/d_lpf_1.h</code> File Reference	175
10.6.1 Detailed Description	177
10.7 <code>d_lpf_1.h</code>	177
10.8 <code>include/integral/i.h</code> File Reference	178
10.8.1 Detailed Description	179
10.9 <code>i.h</code>	180
10.10 <code>include/pd/pd.h</code> File Reference	181
10.10.1 Detailed Description	182
10.11 <code>pd.h</code>	182
10.12 <code>include/pd/pd_lpf_1.h</code> File Reference	183
10.12.1 Detailed Description	184
10.13 <code>pd_lpf_1.h</code>	185
10.14 <code>include/pi/pi.h</code> File Reference	186
10.14.1 Detailed Description	187
10.15 <code>pi.h</code>	188
10.16 <code>include/pid/pid.h</code> File Reference	188
10.16.1 Detailed Description	190
10.17 <code>pid.h</code>	190
10.18 <code>include/pid/pid_gs.h</code> File Reference	191
10.18.1 Detailed Description	192
10.19 <code>pid_gs.h</code>	193
10.20 <code>include/pid/pid_lpf_1.h</code> File Reference	195
10.20.1 Detailed Description	196

10.21	pid_lpf_1.h	197
10.22	include/pid/pid_lpf_1_ff.h File Reference	198
10.22.1	Detailed Description	199
10.23	pid_lpf_1_ff.h	200
10.24	include/pid/pid_p.h File Reference	201
10.24.1	Detailed Description	202
10.25	pid_p.h	203
10.26	include/pid/pid_s.h File Reference	204
10.26.1	Detailed Description	205
10.27	pid_s.h	206
10.28	include/proportional/p.h File Reference	207
10.28.1	Detailed Description	209
10.29	p.h	209
10.30	include/utility/utility.h File Reference	210
10.30.1	Detailed Description	210
10.30.2	Function Documentation	211
10.30.2.1	saturate()	211
10.31	utility.h	212
10.32	README.md File Reference	212
10.33	src/derivative/d.tpp File Reference	212
10.34	d.tpp	213
10.35	src/derivative/d_lpf_1.tpp File Reference	215
10.36	d_lpf_1.tpp	215
10.37	src/integral/i.tpp File Reference	217
10.38	i.tpp	218
10.39	src/pd/pd.tpp File Reference	219
10.40	pd.tpp	220
10.41	src/pd/pd_lpf_1.tpp File Reference	222
10.42	pd_lpf_1.tpp	222
10.43	src/pi/pi.tpp File Reference	224
10.44	pi.tpp	225
10.45	src/pid/pid.tpp File Reference	227
10.46	pid.tpp	227
10.47	src/pid/pid_gs.tpp File Reference	230
10.48	pid_gs.tpp	230
10.49	src/pid/pid_lpf_1.tpp File Reference	234
10.50	pid_lpf_1.tpp	235
10.51	src/pid/pid_lpf_1_ff.tpp File Reference	237
10.52	pid_lpf_1_ff.tpp	238
10.53	src/pid/pid_p.tpp File Reference	241
10.54	pid_p.tpp	241
10.55	src/pid/pid_s.tpp File Reference	245

10.56 pid_s.hpp	245
10.57 src/proportional/p.hpp File Reference	249
10.58 p.hpp	249
Index	251

Chapter 1

PID Control Library

1.1 Introduction

This library provides a collection of reusable C++ components for implementing Proportional-Integral-Derivative (PID) based control systems.

The library is designed to provide modular implementations of individual control actions as well as commonly used combinations such as P, PI, PD, and PID controllers.

1.2 Features

The library provides implementations for:

- Proportional (P) control
- Integral (I) control
- Derivative (D) control
- Proportional-Integral (PI) control
- Proportional-Derivative (PD) control
- Proportional-Integral-Derivative (PID) control
- PID controllers with different structural forms
- Derivative low-pass filtering
- PID low-pass filtering
- Feed-forward PID control
- Utility functions for control-system implementations

1.3 Library Organization

The library is organized into separate modules according to the type of controller:

```
include/  
+-- control_system.h  
+-- derivative/  
    +-- d.h  
    +-- d_lpf_1.h  
+-- integral/  
    +-- i.h  
+-- pd/  
    +-- pd.h  
    +-- pd_lpf_1.h  
+-- pi/  
    +-- pi.h  
+-- pid/  
    +-- pid.h  
    +-- pid_gs.h  
    +-- pid_lpf_1.h  
    +-- pid_lpf_1_ff.h  
    +-- pid_p.h  
    +-- pid_s.h  
+-- proportional/  
    +-- p.h  
+-- utility/  
    +-- utility.h
```

The implementation of the templates is located in the corresponding `.tpp` files under the `src` directory.

1.4 Controllers

1.4.1 Proportional Controller

The proportional controller generates a control action proportional to the current error.

1.4.2 Integral Controller

The integral controller accumulates the error over time and is commonly used to eliminate steady-state error.

1.4.3 Derivative Controller

The derivative controller generates a control action based on the rate of change of the error.

1.4.4 PI Controller

Combines proportional and integral actions to provide improved tracking while eliminating steady-state error.

1.4.5 PD Controller

Combines proportional and derivative actions to improve transient response and damping.

1.4.6 PID Controller

Combines proportional, integral, and derivative actions:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

where:

- $e(t)$ is the control error.
- K_p is the proportional gain.
- K_i is the integral gain.
- K_d is the derivative gain.
- $u(t)$ is the controller output.

1.5 Filtering

The library provides low-pass filtered derivative and PID implementations to reduce the influence of high-frequency measurement noise on the derivative action.

1.6 Feed-Forward Control

The library also provides a PID implementation with feed-forward support. Feed-forward control can be used when a known component of the required control action can be estimated from the desired trajectory or system dynamics.

1.7 Documentation

This documentation is generated using Doxygen.

The public API is provided through the header files under the `include` directory, while template implementations are contained in the corresponding `.hpp` files under the `src` directory.

1.8 License

Author

Version

Chapter 2

PID

PID

Chapter 3

Directory Hierarchy

3.1 Directories

derivative	15
d.h	173
d_lpf_1.h	175
derivative	16
d.tpp	212
d_lpf_1.tpp	215
doc	16
include	16
derivative	15
d.h	173
d_lpf_1.h	175
integral	17
i.h	178
pd	18
pd.h	181
pd_lpf_1.h	183
pi	19
pi.h	186
pid	20
pid.h	188
pid_gs.h	191
pid_lpf_1.h	195
pid_lpf_1_ff.h	198
pid_p.h	201
pid_s.h	204
proportional	22
p.h	207
utility	23
utility.h	210
control_system.h	171
integral	17
i.h	178
integral	18
i.tpp	217
pd	18

pd.h	181
pd_lpf_1.h	183
pd	19
pd.tpp	219
pd_lpf_1.tpp	222
pi	19
pi.h	186
pi	20
pi.tpp	224
pid	20
pid.h	188
pid_gs.h	191
pid_lpf_1.h	195
pid_lpf_1_ff.h	198
pid_p.h	201
pid_s.h	204
pid	21
pid.tpp	227
pid_gs.tpp	230
pid_lpf_1.tpp	234
pid_lpf_1_ff.tpp	237
pid_p.tpp	241
pid_s.tpp	245
proportional	22
p.h	207
proportional	22
p.tpp	249
src	23
derivative	16
d.tpp	212
d_lpf_1.tpp	215
integral	18
i.tpp	217
pd	19
pd.tpp	219
pd_lpf_1.tpp	222
pi	20
pi.tpp	224
pid	21
pid.tpp	227
pid_gs.tpp	230
pid_lpf_1.tpp	234
pid_lpf_1_ff.tpp	237
pid_p.tpp	241
pid_s.tpp	245
proportional	22
p.tpp	249
utility	23
utility.h	210

Chapter 4

Namespace Index

4.1 Namespace List

Here is a list of all namespaces with brief descriptions:

control_system		
	Contains control-system algorithms and controllers	25
control_system::derivative		25
control_system::integral		25
control_system::pd		26
control_system::pi		26
control_system::pid		26
control_system::proportional		26
derivative		
	Contains derivative controller implementations	26
integral		
	Contains integral controller implementations	27
pd		
	Contains proportional-derivative controller implementations	27
pi		
	Contains proportional-integral controller implementations	27
pid		
	Contains proportional-integral-derivative controller implementations	27
proportional		
	Contains proportional control implementations	27

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

control_system::derivative::D< T >	
Discrete-time derivative controller	29
control_system::derivative::D_LPF_1< T >	
Derivative controller with a first-order low-pass filter	36
control_system::integral::I< T >	
Discrete-time integral controller	45
control_system::pd::PD< T >	
Discrete-time proportional-derivative controller	52
control_system::pd::PD_LPF_1< T >	
PD controller with first-order low-pass filtered derivative	60
control_system::pi::PI< T >	
Discrete-time proportional-integral controller	70
control_system::pid::PID< T >	
Discrete-time proportional-integral-derivative controller	79
control_system::pid::PID_GS< T >	
PID controller with gain scheduling	90
control_system::pid::PID_LPF_1< T >	
PID controller with a first-order low-pass filtered derivative	109
control_system::pid::PID_LPF_1_FF< T >	
PID controller with first-order low-pass filtered derivative and feed-forward	121
control_system::pid::PID_P< T >	
PID controller with configurable derivative filtering and feed-forward	134
control_system::pid::PID_S< T >	
PID controller with configurable derivative filtering	149
control_system::proportional::P< T >	
Proportional controller	165

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

include/control_system.h	
Main public header for the Control System library	171
include/derivative/d.h	
Derivative (D) controller	173
include/derivative/d_lpf_1.h	
Derivative controller with first-order low-pass filtering	175
include/integral/i.h	
Integral (I) controller	178
include/pd/pd.h	
Proportional-Derivative (PD) controller	181
include/pd/pd_lpf_1.h	
Proportional-Derivative (PD) controller with first-order low-pass filtering	183
include/pi/pi.h	
Proportional-Integral (PI) controller	186
include/pid/pid.h	
Proportional-Integral-Derivative (PID) controller	188
include/pid/pid_gs.h	
PID controller with gain scheduling	191
include/pid/pid_lpf_1.h	
PID controller with first-order low-pass filtered derivative	195
include/pid/pid_lpf_1_ff.h	
PID controller with first-order low-pass filtered derivative and feed-forward	198
include/pid/pid_p.h	
PID controller with configurable derivative filtering and feed-forward	201
include/pid/pid_s.h	
PID controller with setpoint weighting	204
include/proportional/p.h	
Proportional (P) controller	207
include/utility/utility.h	
Utility functions for the Control System library	210
src/derivative/d.tpp	212
src/derivative/d_lpf_1.tpp	215
src/integral/i.tpp	217
src/pd/pd.tpp	219
src/pd/pd_lpf_1.tpp	222

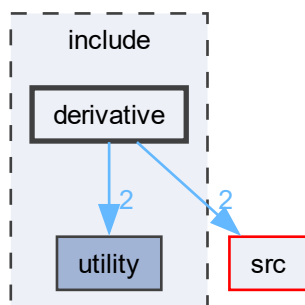
src/pi/ pi.tpp	224
src/pid/ pid.tpp	227
src/pid/ pid_gs.tpp	230
src/pid/ pid_lpf_1.tpp	234
src/pid/ pid_lpf_1_ff.tpp	237
src/pid/ pid_p.tpp	241
src/pid/ pid_s.tpp	245
src/proportional/ p.tpp	249

Chapter 7

Directory Documentation

7.1 include/derivative Directory Reference

Directory dependency graph for derivative:

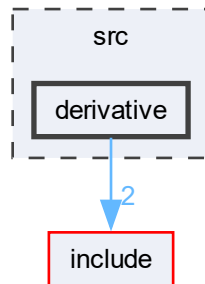


Files

- file [d.h](#)
Derivative (D) controller.
- file [d_lpf_1.h](#)
Derivative controller with first-order low-pass filtering.

7.2 src/derivative Directory Reference

Directory dependency graph for derivative:



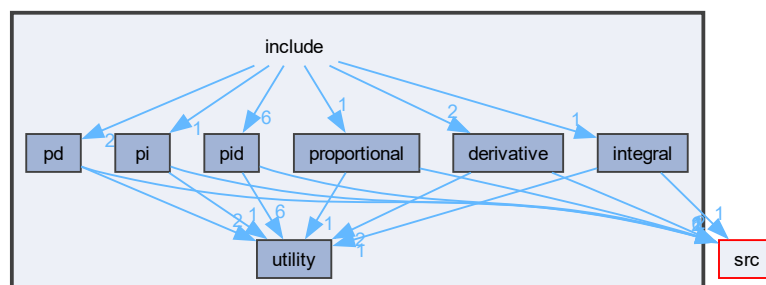
Files

- file [d.hpp](#)
- file [d_lpf_1.hpp](#)

7.3 doc Directory Reference

7.4 include Directory Reference

Directory dependency graph for include:



Directories

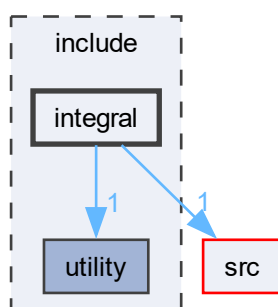
- directory [derivative](#)
- directory [integral](#)
- directory [pd](#)
- directory [pi](#)
- directory [pid](#)
- directory [proportional](#)
- directory [utility](#)

Files

- file [control_system.h](#)
Main public header for the Control System library.

7.5 include/integral Directory Reference

Directory dependency graph for integral:

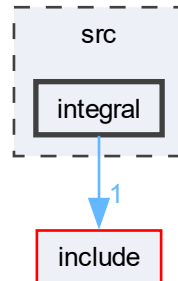


Files

- file [i.h](#)
Integral (I) controller.

7.6 src/integral Directory Reference

Directory dependency graph for integral:

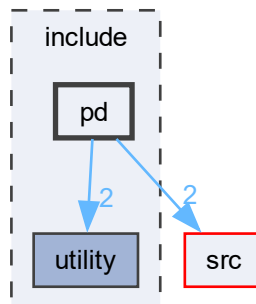


Files

- file [i.tpp](#)

7.7 include/pd Directory Reference

Directory dependency graph for pd:

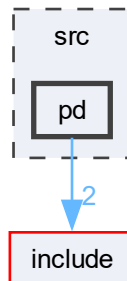


Files

- file [pd.h](#)
Proportional-Derivative (PD) controller.
- file [pd_lpf_1.h](#)
Proportional-Derivative (PD) controller with first-order low-pass filtering.

7.8 src/pd Directory Reference

Directory dependency graph for pd:

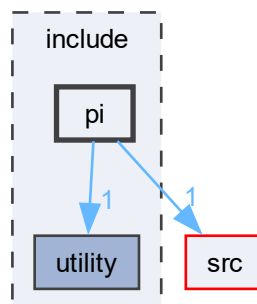


Files

- file [pd.hpp](#)
- file [pd_lpf_1.hpp](#)

7.9 include/pi Directory Reference

Directory dependency graph for pi:

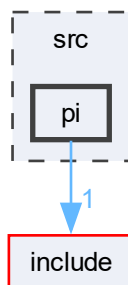


Files

- file [pi.h](#)
Proportional-Integral (PI) controller.

7.10 src/pi Directory Reference

Directory dependency graph for pi:

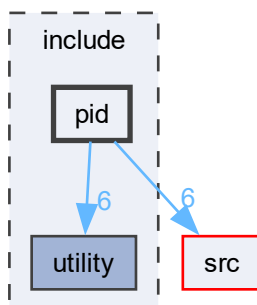


Files

- file [pi.hpp](#)

7.11 include/pid Directory Reference

Directory dependency graph for pid:

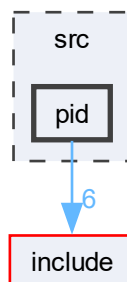


Files

- file [pid.h](#)
Proportional-Integral-Derivative (PID) controller.
- file [pid_gs.h](#)
PID controller with gain scheduling.
- file [pid_lpf_1.h](#)
PID controller with first-order low-pass filtered derivative.
- file [pid_lpf_1_ff.h](#)
PID controller with first-order low-pass filtered derivative and feed-forward.
- file [pid_p.h](#)
PID controller with configurable derivative filtering and feed-forward.
- file [pid_s.h](#)
PID controller with setpoint weighting.

7.12 src/pid Directory Reference

Directory dependency graph for pid:

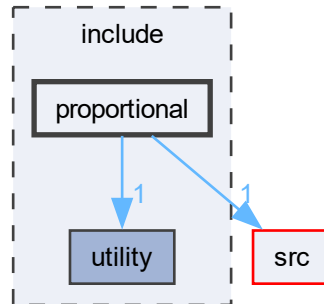


Files

- file [pid.tpp](#)
- file [pid_gs.tpp](#)
- file [pid_lpf_1.tpp](#)
- file [pid_lpf_1_ff.tpp](#)
- file [pid_p.tpp](#)
- file [pid_s.tpp](#)

7.13 include/proportional Directory Reference

Directory dependency graph for proportional:

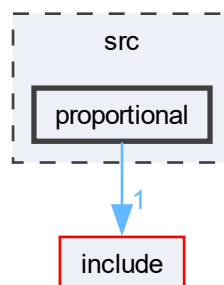


Files

- file [p.h](#)
Proportional (P) controller.

7.14 src/proportional Directory Reference

Directory dependency graph for proportional:

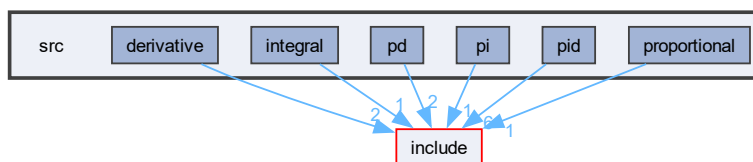


Files

- file [p.hpp](#)

7.15 src Directory Reference

Directory dependency graph for src:

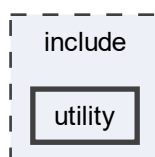


Directories

- directory [derivative](#)
- directory [integral](#)
- directory [pd](#)
- directory [pi](#)
- directory [pid](#)
- directory [proportional](#)

7.16 include/utility Directory Reference

Directory dependency graph for utility:



Files

- file [utility.h](#)
Utility functions for the Control System library.

Chapter 8

Namespace Documentation

8.1 `control_system` Namespace Reference

Contains control-system algorithms and controllers.

Namespaces

- namespace [derivative](#)
- namespace [integral](#)
- namespace [pd](#)
- namespace [pi](#)
- namespace [pid](#)
- namespace [proportional](#)

8.1.1 Detailed Description

Contains control-system algorithms and controllers.

8.2 `control_system::derivative` Namespace Reference

Classes

- class [D](#)
Discrete-time derivative controller.
- class [D_LPF_1](#)
Derivative controller with a first-order low-pass filter.

8.3 `control_system::integral` Namespace Reference

Classes

- class [I](#)
Discrete-time integral controller.

8.4 control_system::pd Namespace Reference

Classes

- class [PD](#)
Discrete-time proportional-derivative controller.
- class [PD_LPF_1](#)
PD controller with first-order low-pass filtered derivative.

8.5 control_system::pi Namespace Reference

Classes

- class [PI](#)
Discrete-time proportional-integral controller.

8.6 control_system::pid Namespace Reference

Classes

- class [PID](#)
Discrete-time proportional-integral-derivative controller.
- class [PID_GS](#)
PID controller with gain scheduling.
- class [PID_LPF_1](#)
PID controller with a first-order low-pass filtered derivative.
- class [PID_LPF_1_FF](#)
PID controller with first-order low-pass filtered derivative and feed-forward.
- class [PID_P](#)
PID controller with configurable derivative filtering and feed-forward.
- class [PID_S](#)
PID controller with configurable derivative filtering.

8.7 control_system::proportional Namespace Reference

Classes

- class [P](#)
Proportional controller.

8.8 derivative Namespace Reference

Contains derivative controller implementations.

8.8.1 Detailed Description

Contains derivative controller implementations.

8.9 integral Namespace Reference

Contains integral controller implementations.

8.9.1 Detailed Description

Contains integral controller implementations.

8.10 pd Namespace Reference

Contains proportional-derivative controller implementations.

8.10.1 Detailed Description

Contains proportional-derivative controller implementations.

8.11 pi Namespace Reference

Contains proportional-integral controller implementations.

8.11.1 Detailed Description

Contains proportional-integral controller implementations.

8.12 pid Namespace Reference

Contains proportional-integral-derivative controller implementations.

8.12.1 Detailed Description

Contains proportional-integral-derivative controller implementations.

8.13 proportional Namespace Reference

Contains proportional control implementations.

8.13.1 Detailed Description

Contains proportional control implementations.

Chapter 9

Class Documentation

9.1 control_system::derivative::D< T > Class Template Reference

Discrete-time derivative controller.

```
#include <d.h>
```

Public Member Functions

- [D](#) ()
Constructs a derivative controller.
- [T get_dt](#) ()
Gets the controller sampling time.
- [T get_e_k_1](#) ()
Gets the previous control error.
- [T get_Kd](#) ()
Gets the derivative gain.
- [T get_u_max](#) ()
Gets the maximum controller output.
- [void init](#) (T dt_, T Kd_, T u_max_)
Initializes the derivative controller.
- [void merge](#) (T u_k_1_)
Merges an external controller output into the derivative state.
- [void reset](#) ()
Resets the derivative controller state.
- [void set_dt](#) (T dt_)
Sets the controller sampling time.
- [void set_Kd](#) (T Kd_)
Sets the derivative gain.
- [void set_param](#) (T dt_, T Kd_, T u_max_)
Sets the derivative controller parameters.
- [void set_u_max](#) (T u_max_)
Sets the maximum controller output.
- [T update](#) (T x_0, T x)
Computes the derivative control output.

Private Attributes

- T `dt` = 0.0
Controller sampling time.
- T `e_k_1` = 0.0
Previous control error.
- T `Kd` = 0.0
Derivative gain.
- bool `start` = true
Controller startup state.
- T `u_max` = 9999999999
Maximum controller output.

9.1.1 Detailed Description

template<typename T>
class control_system::derivative::D< T >

Discrete-time derivative controller.

The `D` class implements a discrete-time derivative controller using a configurable derivative gain and sampling time.

The derivative contribution is calculated from the change in control error between successive controller iterations.

The previous control error is maintained as internal state and the controller output is limited by the configured maximum output value.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::derivative::D<double> controller;

controller.init(
    0.001,
    Kd,
    100.0
);

double output = controller.update(reference, measurement);
```

Definition at line 76 of file [d.h](#).

9.1.2 Constructor & Destructor Documentation

9.1.2.1 D()

template<typename T>
control_system::derivative::D< T >::D ()

Constructs a derivative controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file [d.h](#).

9.1.3 Member Function Documentation

9.1.3.1 get_dt()

```
template<typename T>
T control_system::derivative::D< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 78 of file [d.h](#).

9.1.3.2 get_e_k_1()

```
template<typename T>
T control_system::derivative::D< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 90 of file [d.h](#).

9.1.3.3 get_Kd()

```
template<typename T>
T control_system::derivative::D< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 84 of file [d.h](#).

9.1.3.4 get_u_max()

```
template<typename T>
T control_system::derivative::D< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

9.1.3.5 init()

```
template<typename T>
void control_system::derivative::D< T >::init (
    T dt_,
    T Kd_,
    T u_max_)
```

Initializes the derivative controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kd_</i>	Derivative gain.
in	<i>u_↔ max_</i>	Maximum allowed controller output.

Initializes the controller sampling time, derivative gain, and output limit.

Definition at line 14 of file [d.h](#).

9.1.3.6 merge()

```
template<typename T>
void control_system::derivative::D< T >::merge (
    T u_k_l_)
```

Merges an external controller output into the derivative state.

Parameters

in	<i>u_k_↔ l_</i>	Previous or externally provided controller output.
----	---------------------	--

Updates the controller state using the specified previous controller output.

Definition at line 55 of file [d.h](#).

9.1.3.7 reset()

```
template<typename T>
void control_system::derivative::D< T >::reset ()
```

Resets the derivative controller state.

Resets the stored previous control error and startup state.

Definition at line 48 of file [d.h](#).

9.1.3.8 set_dt()

```
template<typename T>
void control_system::derivative::D< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt_{\leftarrow}$ —	Sampling time in seconds.
----	------------------------	---------------------------

Definition at line 60 of file [d.h](#).

9.1.3.9 set_Kd()

```
template<typename T>
void control_system::derivative::D< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd_{\leftarrow}$ —	Derivative gain.
----	------------------------	------------------

Definition at line 66 of file [d.h](#).

9.1.3.10 set_param()

```
template<typename T>
void control_system::derivative::D< T >::set_param (
    T dt_,
    T Kd_,
    T u_max_)
```

Sets the derivative controller parameters.

Parameters

in	$dt_{\leftarrow}$	Controller sampling time in seconds.
in	$Kd_{\leftarrow}$	Derivative gain.
in	$u_{\leftarrow max}$	Maximum allowed controller output.

Updates the controller sampling time, derivative gain, and maximum output limit.

Definition at line 21 of file [d.h](#).

9.1.3.11 set_u_max()

```
template<typename T>
void control_system::derivative::D< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_{\leftarrow max_}$	Maximum allowed controller output.
----	------------------------	------------------------------------

Definition at line 72 of file [d.h](#).

9.1.3.12 update()

```
template<typename T>
T control_system::derivative::D< T >::update (
    T x_0,
    T x)
```

Computes the derivative control output.

Parameters

in	$x_{\leftarrow 0}$	Reference or desired value.
in	x	Current or measured value.

Returns

Derivative controller output limited by the configured maximum output.

Calculates the derivative contribution from the change in control error between the current and previous controller iterations.

Definition at line 29 of file [d.h](#).

9.1.4 Member Data Documentation

9.1.4.1 dt

```
template<typename T>
T control_system::derivative::D< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time derivative controller in seconds.

Definition at line 207 of file [d.h](#).

9.1.4.2 e_k_1

```
template<typename T>
T control_system::derivative::D< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line 215 of file [d.h](#).

9.1.4.3 Kd

```
template<typename T>
T control_system::derivative::D< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 220 of file [d.h](#).

9.1.4.4 start

```
template<typename T>
bool control_system::derivative::D< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line 228 of file [d.h](#).

9.1.4.5 u_max

```
template<typename T>
T control_system::derivative::D< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 237 of file [d.h](#).

The documentation for this class was generated from the following files:

- [include/derivative/d.h](#)
- [src/derivative/d.tpp](#)

9.2 control_system::derivative::D_LPF_1< T > Class Template Reference

Derivative controller with a first-order low-pass filter.

```
#include <d_lpf_1.h>
```

Public Member Functions

- [D_LPF_1](#) ()
Constructs a derivative controller with filtered output.
- [T get_dt](#) ()
Gets the controller sampling time.
- [T get_e_k_1](#) ()
Gets the previous control error.
- [T get_fc](#) ()
Gets the low-pass filter cutoff frequency.
- [T get_Kd](#) ()
Gets the derivative gain.
- [T get_u_k_1](#) ()
Gets the previous controller output.
- [T get_u_max](#) ()
Gets the maximum controller output.
- [void init](#) (T dt_, T Kd_, T fc_, T u_max_)
Initializes the derivative controller.
- [void merge](#) (T u_k_1_)
Merges an external controller output into the derivative state.
- [void reset](#) ()
Resets the derivative controller state.
- [void set_dt](#) (T dt_)
Sets the controller sampling time.
- [void set_fc](#) (T fc_)
Sets the low-pass filter cutoff frequency.
- [void set_Kd](#) (T Kd_)
Sets the derivative gain.
- [void set_param](#) (T dt_, T Kd_, T fc_, T u_max_)
Sets the derivative controller parameters.
- [void set_u_max](#) (T u_max_)
Sets the maximum controller output.
- [T update](#) (T x_0, T x)
Computes the filtered derivative control output.

Private Attributes

- T `dt` = 0.0
Controller sampling time.
- T `e_k_1` = 0.0
Previous control error.
- T `Kd` = 0.0
Derivative gain.
- bool `start` = true
Controller startup state.
- T `tau` = 0.0
Time constant of the first-order low-pass filter.
- T `u_k_1` = 0.0
Previous controller output.
- T `u_max` = 9999999999
Maximum controller output.

9.2.1 Detailed Description

template<typename T>
class control_system::derivative::D_LPF_1< T >

Derivative controller with a first-order low-pass filter.

The `D_LPF_1` class implements a discrete-time derivative controller with a first-order low-pass filter applied to the derivative action.

The controller uses the sampling time, derivative gain, and filter cutoff frequency to calculate the derivative contribution.

The previous control error and controller output are maintained as internal state, and the controller output is limited by the configured maximum output.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::derivative::D_LPF_1<double> controller;

controller.init(
    0.001,
    Kd,
    10.0,
    100.0
);

double output = controller.update(reference, measurement);
```

Definition at line 79 of file `d_lpf_1.h`.

9.2.2 Constructor & Destructor Documentation

9.2.2.1 D_LPF_1()

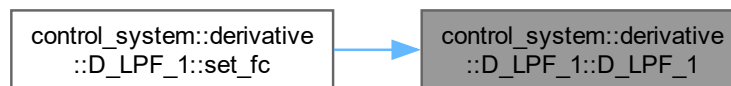
```
template<typename T>
control_system::derivative::D_LPF_1< T >::D_LPF_1 ()
```

Constructs a derivative controller with filtered output.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file [d_lpf_1.h](#).

Here is the caller graph for this function:



9.2.3 Member Function Documentation

9.2.3.1 get_dt()

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 91 of file [d_lpf_1.h](#).

9.2.3.2 get_e_k_1()

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 109 of file [d_lpf_1.h](#).

9.2.3.3 get_fc()

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::get_fc ()
```

Gets the low-pass filter cutoff frequency.

Returns

Cutoff frequency in Hz.

Definition at line 103 of file [d_lpf_1.h](#).

9.2.3.4 get_Kd()

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 97 of file [d_lpf_1.h](#).

9.2.3.5 get_u_k_1()

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::get_u_k_1 ()
```

Gets the previous controller output.

Returns

Previous controller output.

Definition at line 115 of file [d_lpf_1.h](#).

9.2.3.6 get_u_max()

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 121 of file [d_lpf_1.h](#).

9.2.3.7 init()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::init (
    T dt_,
    T Kd_,
    T fc_,
    T u_max_)
```

Initializes the derivative controller.

Parameters

in	$dt_$	Controller sampling time in seconds.
in	$Kd_$	Derivative gain.
in	$fc_$	Low-pass filter cutoff frequency in Hz.
in	$u_{\leftarrow max_}$	Maximum allowed controller output.

Initializes the controller sampling time, derivative gain, filter cutoff frequency, and output limit.

Definition at line 17 of file [d_lpf_1.h](#).

9.2.3.8 merge()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::merge (
    T u_k_1_)
```

Merges an external controller output into the derivative state.

Parameters

in	$u_{k_{\leftarrow} 1_}$	Previous or externally provided controller output.
----	--------------------------	--

Updates the internal previous-output state using the specified controller output.

Definition at line 61 of file [d_lpf_1.h](#).

9.2.3.9 reset()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::reset ()
```

Resets the derivative controller state.

Resets the stored control error, previous controller output, and startup state.

Definition at line 53 of file [d_lpf_1.h](#).

9.2.3.10 set_dt()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt \leftrightarrow$ _ $\leftrightarrow$	Sampling time in seconds.
----	---	---------------------------

Definition at line 67 of file [d_lpf_1.h](#).

9.2.3.11 set_fc()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::set_fc (
    T fc_)
```

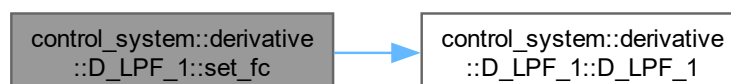
Sets the low-pass filter cutoff frequency.

Parameters

in	$fc \leftrightarrow$ _ $\leftrightarrow$	Cutoff frequency in Hz.
----	---	-------------------------

Definition at line 79 of file [d_lpf_1.h](#).

Here is the call graph for this function:



9.2.3.12 set_Kd()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd_{\leftrightarrow}$	Derivative gain.
	—	

Definition at line 73 of file [d_lpf_1.h](#).

9.2.3.13 set_param()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::set_param (
    T dt_,
    T Kd_,
    T fc_,
    T u_max_)
```

Sets the derivative controller parameters.

Parameters

in	$dt_{\rightarrow}$	Controller sampling time in seconds.
in	$Kd_{\rightarrow}$	Derivative gain.
in	$fc_{\rightarrow}$	Low-pass filter cutoff frequency in Hz.
in	$u_{\leftrightarrow max_{\rightarrow}}$	Maximum allowed controller output.

Updates the controller sampling time, derivative gain, filter cutoff frequency, and maximum output limit.

Definition at line 24 of file [d_lpf_1.h](#).

9.2.3.14 set_u_max()

```
template<typename T>
void control_system::derivative::D_LPF_1< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_{\leftarrow \max}$	Maximum allowed controller output.
----	-----------------------	------------------------------------

Definition at line 85 of file [d_lpf_1.h](#).

9.2.3.15 update()

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::update (
    T x_0,
    T x)
```

Computes the filtered derivative control output.

Parameters

in	$x_{\leftarrow 0}$	Reference or desired value.
in	x	Current or measured value.

Returns

Filtered derivative controller output limited by the configured maximum output.

Calculates the derivative contribution from the change in control error and applies the configured first-order low-pass filtering.

Definition at line 33 of file [d_lpf_1.h](#).

9.2.4 Member Data Documentation**9.2.4.1 dt**

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time derivative controller in seconds.

Definition at line 234 of file [d_lpf_1.h](#).

9.2.4.2 e_k_1

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line 242 of file [d_lpf_1.h](#).

9.2.4.3 Kd

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 247 of file [d_lpf_1.h](#).

9.2.4.4 start

```
template<typename T>
bool control_system::derivative::D_LPF_1< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line 255 of file [d_lpf_1.h](#).

9.2.4.5 tau

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::tau = 0.0 [private]
```

Time constant of the first-order low-pass filter.

Definition at line 268 of file [d_lpf_1.h](#).

9.2.4.6 u_k_1

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::u_k_1 = 0.0 [private]
```

Previous controller output.

Stores the controller output from the previous iteration.

Definition at line 263 of file [d_lpf_1.h](#).

9.2.4.7 u_max

```
template<typename T>
T control_system::derivative::D_LPF_1< T >::u_max = 999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 277 of file [d_lpf_1.h](#).

The documentation for this class was generated from the following files:

- [include/derivative/d_lpf_1.h](#)
- [src/derivative/d_lpf_1.tpp](#)

9.3 control_system::integral::l< T > Class Template Reference

Discrete-time integral controller.

```
#include <i.h>
```

Public Member Functions

- `l()`
Constructs an integral controller.
- `T get_dt()`
Gets the controller sampling time.
- `T get_Ki()`
Gets the integral gain.
- `T get_u_k_1()`
Gets the previous controller output.
- `T get_u_max()`
Gets the maximum controller output.
- `void init(T dt_, T Ki_, T u_k_1_, T u_max_)`
Initializes the integral controller.
- `void merge(T u_k_1_)`
Merges an external controller output into the integral state.
- `void reset()`
Resets the integral controller state.
- `void set_dt(T dt_)`
Sets the controller sampling time.
- `void set_Ki(T Ki_)`
Sets the integral gain.
- `void set_param(T dt_, T Ki_, T u_k_1_, T u_max_)`
Sets the integral controller parameters.
- `void set_u_0(T u_k_1_)`
Sets the previous controller output.
- `void set_u_max(T u_max_)`
Sets the maximum controller output.
- `T update(T x_0, T x)`
Computes the integral control output.

Private Attributes

- `T dt = 0.0`
Controller sampling time.
- `T Ki = 0.0`
Integral gain.
- `T u_k_1 = 0.0`
Previous controller output.
- `T u_max = 9999999999`
Maximum controller output.

9.3.1 Detailed Description

template<typename T>
class control_system::integral::I< T >

Discrete-time integral controller.

The `I` class implements a discrete-time integral controller using a configurable integral gain and sampling time.

The controller maintains its previous output as internal state and applies a configurable maximum output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::integral::I<double> controller;

controller.init(
    0.001,
    Ki,
    0.0,
    100.0
);

double output = controller.update(reference, measurement);
```

Definition at line 73 of file [i.h](#).

9.3.2 Constructor & Destructor Documentation

9.3.2.1 I()

template<typename T>
control_system::integral::I< T >::I ()

Constructs an integral controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file [i.h](#).

9.3.3 Member Function Documentation

9.3.3.1 get_dt()

template<typename T>
T control_system::integral::I< T >::get_dt ()

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 74 of file [i.h](#).

9.3.3.2 get_Ki()

```
template<typename T>
T control_system::integral::I< T >::get_Ki ()
```

Gets the integral gain.

Returns

Integral gain.

Definition at line 80 of file [i.h](#).

9.3.3.3 get_u_k_1()

```
template<typename T>
T control_system::integral::I< T >::get_u_k_1 ()
```

Gets the previous controller output.

Returns

Previous controller output.

Definition at line 86 of file [i.h](#).

9.3.3.4 get_u_max()

```
template<typename T>
T control_system::integral::I< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 92 of file [i.h](#).

9.3.3.5 init()

```
template<typename T>
void control_system::integral::I< T >::init (
    T dt_,
    T Ki_,
    T u_k_1_,
    T u_max_)
```

Initializes the integral controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Ki_</i>	Integral gain.
in	<i>u_k_↵ 1_</i>	Previous controller output.
in	<i>u_↵ max_</i>	Maximum allowed controller output.

Initializes the controller sampling time, integral gain, previous controller output, and output limit.

Definition at line 13 of file [i.h](#).

9.3.3.6 merge()

```
template<typename T>
void control_system::integral::I< T >::merge (
    T u_k_1_)
```

Merges an external controller output into the integral state.

Parameters

in	<i>u_k_↵ 1_</i>	Previous or externally provided controller output.
----	---------------------	--

Updates the internal previous-output state using the specified controller output.

Definition at line 44 of file [i.h](#).

9.3.3.7 reset()

```
template<typename T>
void control_system::integral::I< T >::reset ()
```

Resets the integral controller state.

Resets the stored controller output to its initial state.

Definition at line 38 of file [i.h](#).

9.3.3.8 set_dt()

```
template<typename T>
void control_system::integral::I< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt_{\leftarrow}$ — $\leftarrow$	Sampling time in seconds.
----	-------------------------------------	---------------------------

Definition at line 50 of file [i.h](#).

9.3.3.9 set_Ki()

```
template<typename T>
void control_system::integral::I< T >::set_Ki (
    T Ki_)
```

Sets the integral gain.

Parameters

in	$Ki_{\leftarrow}$ — $\leftarrow$	Integral gain.
----	-------------------------------------	----------------

Definition at line 56 of file [i.h](#).

9.3.3.10 set_param()

```
template<typename T>
void control_system::integral::I< T >::set_param (
    T dt_,
    T Ki_,
    T u_k_l_,
    T u_max_)
```

Sets the integral controller parameters.

Parameters

in	$dt_{\leftarrow}$	Controller sampling time in seconds.
----	-------------------	--------------------------------------

in	K_i	Integral gain.
in	u_{k-1}	Previous controller output.
in	u_{max}	Maximum allowed controller output.

Updates the controller sampling time, integral gain, previous controller output, and maximum output limit.

Definition at line 19 of file [i.h](#).

9.3.3.11 set_u_0()

```
template<typename T>
void control_system::integral::I< T >::set_u_0 (
    T u_k_1_)
```

Sets the previous controller output.

Parameters

in	u_{k-1}	Previous controller output.
----	-----------	-----------------------------

Definition at line 62 of file [i.h](#).

9.3.3.12 set_u_max()

```
template<typename T>
void control_system::integral::I< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	u_{max}	Maximum allowed controller output.
----	-----------	------------------------------------

Definition at line 68 of file [i.h](#).

9.3.3.13 update()

```
template<typename T>
T control_system::integral::I< T >::update (
    T x_0,
    T x)
```

Computes the integral control output.

Parameters

in	$x \leftarrow$ _0	Reference or desired value.
in	x	Current or measured value.

Returns

Integral controller output limited by the configured maximum output.

Calculates the integral contribution using the current control error, sampling time, integral gain, and stored controller state.

Definition at line 28 of file [i.h](#).

9.3.4 Member Data Documentation

9.3.4.1 dt

```
template<typename T>
T control_system::integral::I< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time integral controller in seconds.

Definition at line 213 of file [i.h](#).

9.3.4.2 Ki

```
template<typename T>
T control_system::integral::I< T >::Ki = 0.0 [private]
```

Integral gain.

Definition at line 226 of file [i.h](#).

9.3.4.3 u_k_1

```
template<typename T>
T control_system::integral::I< T >::u_k_1 = 0.0 [private]
```

Previous controller output.

Stores the controller output from the previous iteration.

Definition at line 221 of file [i.h](#).

9.3.4.4 u_max

```
template<typename T>
T control_system::integral::I< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 235 of file [i.h](#).

The documentation for this class was generated from the following files:

- [include/integral/i.h](#)
- [src/integral/i.tpp](#)

9.4 control_system::pd::PD< T > Class Template Reference

Discrete-time proportional-derivative controller.

```
#include <pd.h>
```

Public Member Functions

- [PD](#) ()
Constructs a [PD](#) controller.
- T [get_dt](#) ()
Gets the controller sampling time.
- T [get_e_k_1](#) ()
Gets the previous control error.
- T [get_Kd](#) ()
Gets the derivative gain.
- T [get_Kp](#) ()
Gets the proportional gain.
- T [get_u_max](#) ()
Gets the maximum controller output.
- void [init](#) (T dt_, T Kp_, T Kd_, T u_max_)
Initializes the [PD](#) controller.
- void [merge](#) (T u_k_1_)

- *Merges an external controller output into the **PD** state.*
- void **reset** ()
*Resets the **PD** controller state.*
- void **set_dt** (T dt_)
Sets the controller sampling time.
- void **set_Kd** (T Kd_)
Sets the derivative gain.
- void **set_Kp** (T Kp_)
Sets the proportional gain.
- void **set_param** (T dt_, T Kp_, T Kd_, T u_max_)
*Sets the **PD** controller parameters.*
- void **set_u_max** (T u_max_)
Sets the maximum controller output.
- T **update** (T x_0, T x)
*Computes the **PD** control output.*

Private Attributes

- T **dt** = 0.0
Controller sampling time.
- T **e_k_1** = 0.0
Previous control error.
- T **Kd** = 0.0
Derivative gain.
- T **Kp** = 0.0
Proportional gain.
- bool **start** = true
Controller startup state.
- T **u_max** = 9999999999
Maximum controller output.

9.4.1 Detailed Description

template<typename T>
class control_system::pd::PD< T >

Discrete-time proportional-derivative controller.

The **PD** class implements a discrete-time proportional-derivative controller using configurable proportional and derivative gains.

The controller uses the sampling time for its discrete-time derivative action and maintains the previous control error as internal state.

Template Parameters

T	Numeric data type used for the controller.
----------	--

Example

```

control_system::pd::PD<double> controller;

controller.init(
    0.001,
    Kp,
    Kd,
    100.0
);

double output = controller.update(reference, measurement);

```

Definition at line 74 of file [pd.h](#).

9.4.2 Constructor & Destructor Documentation**9.4.2.1 PD()**

```

template<typename T>
control_system::pd::PD< T >::PD ()

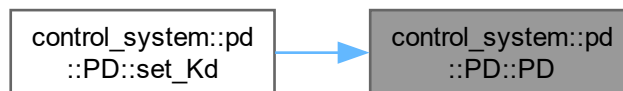
```

Constructs a **PD** controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file [pd.h](#).

Here is the caller graph for this function:

**9.4.3 Member Function Documentation****9.4.3.1 get_dt()**

```

template<typename T>
T control_system::pd::PD< T >::get_dt ()

```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 86 of file [pd.h](#).

9.4.3.2 get_e_k_1()

```
template<typename T>
T control_system::pd::PD< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 104 of file [pd.h](#).

9.4.3.3 get_Kd()

```
template<typename T>
T control_system::pd::PD< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 98 of file [pd.h](#).

9.4.3.4 get_Kp()

```
template<typename T>
T control_system::pd::PD< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 92 of file [pd.h](#).

9.4.3.5 get_u_max()

```
template<typename T>
T control_system::pd::PD< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 110 of file [pd.h](#).

9.4.3.6 init()

```
template<typename T>
void control_system::pd::PD< T >::init (
    T dt_,
    T Kp_,
    T Kd_,
    T u_max_)
```

Initializes the PD controller.

Parameters

in	$dt_$	Controller sampling time in seconds.
in	$Kp_$	Proportional gain.
in	$Kd_$	Derivative gain.
in	$u_{\leftarrow max_}$	Maximum allowed controller output.

Initializes the controller sampling time, proportional and derivative gains, and output limit.

Definition at line 15 of file [pd.h](#).

9.4.3.7 merge()

```
template<typename T>
void control_system::pd::PD< T >::merge (
    T u_k_l_)
```

Merges an external controller output into the PD state.

Parameters

in	$u_{k_{\leftarrow} l_}$	Previous or externally provided controller output.
----	--------------------------	--

Updates the controller state using the specified previous controller output.

Definition at line 57 of file [pd.h](#).

9.4.3.8 reset()

```
template<typename T>
void control_system::pd::PD< T >::reset ()
```

Resets the PD controller state.

Resets the stored previous control error and startup state.

Definition at line 50 of file [pd.h](#).

9.4.3.9 set_dt()

```
template<typename T>
void control_system::pd::PD< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt \leftarrow$ —	Sampling time in seconds.
----	----------------------	---------------------------

Definition at line 62 of file [pd.h](#).

9.4.3.10 set_Kd()

```
template<typename T>
void control_system::pd::PD< T >::set_Kd (
    T Kd_)
```

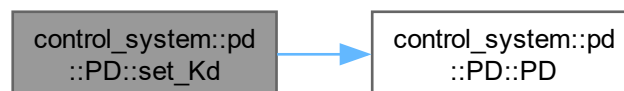
Sets the derivative gain.

Parameters

in	$Kd \leftarrow$ —	Derivative gain.
----	----------------------	------------------

Definition at line 74 of file [pd.h](#).

Here is the call graph for this function:



9.4.3.11 set_Kp()

```
template<typename T>
void control_system::pd::PD< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 68 of file [pd.h](#).

9.4.3.12 set_param()

```
template<typename T>
void control_system::pd::PD< T >::set_param (
    T dt_,
    T Kp_,
    T Kd_,
    T u_max_)
```

Sets the [PD](#) controller parameters.

Parameters

in	$dt_{\leftarrow}$	Controller sampling time in seconds.
in	$Kp_{\leftarrow}$	Proportional gain.
in	$Kd_{\leftarrow}$	Derivative gain.
in	$u_{\leftarrow max_{\leftarrow}}$	Maximum allowed controller output.

Updates the controller sampling time, proportional and derivative gains, and maximum output limit.

Definition at line 22 of file [pd.h](#).

9.4.3.13 set_u_max()

```
template<typename T>
void control_system::pd::PD< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_{\leftarrow \max}$	Maximum allowed controller output.
----	-----------------------	------------------------------------

Definition at line 80 of file [pd.h](#).

9.4.3.14 update()

```
template<typename T>
T control_system::pd::PD< T >::update (
    T x_0,
    T x)
```

Computes the [PD](#) control output.

Parameters

in	$x_{\leftarrow 0}$	Reference or desired value.
in	x	Current or measured value.

Returns

[PD](#) controller output limited by the configured maximum output.

Calculates the proportional and derivative contributions using the current control error and the stored previous control error.

Definition at line 31 of file [pd.h](#).

9.4.4 Member Data Documentation**9.4.4.1 dt**

```
template<typename T>
T control_system::pd::PD< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time [PD](#) controller in seconds.

Definition at line 221 of file [pd.h](#).

9.4.4.2 e_k_1

```
template<typename T>
T control_system::pd::PD< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line 229 of file [pd.h](#).

9.4.4.3 Kd

```
template<typename T>
T control_system::pd::PD< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 239 of file [pd.h](#).

9.4.4.4 Kp

```
template<typename T>
T control_system::pd::PD< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 234 of file [pd.h](#).

9.4.4.5 start

```
template<typename T>
bool control_system::pd::PD< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line 247 of file [pd.h](#).

9.4.4.6 u_max

```
template<typename T>
T control_system::pd::PD< T >::u_max = 999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 256 of file [pd.h](#).

The documentation for this class was generated from the following files:

- [include/pd/pd.h](#)
- [src/pd/pd.cpp](#)

9.5 control_system::pd::PD_LPF_1< T > Class Template Reference

[PD](#) controller with first-order low-pass filtered derivative.

```
#include <pd_lpf_1.h>
```

Public Member Functions

- [PD_LPF_1](#) ()
Constructs a [PD](#) controller with filtered derivative.
- [T get_dt](#) ()
Gets the controller sampling time.
- [T get_e_k_1](#) ()
Gets the previous control error.
- [T get_fc](#) ()
Gets the derivative filter cutoff frequency.
- [T get_Kd](#) ()
Gets the derivative gain.
- [T get_Kp](#) ()
Gets the proportional gain.
- [T get_u_k_1](#) ()
Gets the previous controller output.
- [T get_u_max](#) ()
Gets the maximum controller output.
- void [init](#) (T dt_, T Kp_, T Kd_, T fc_, T u_max_)
Initializes the [PD](#) controller.
- void [merge](#) (T u_k_1_)
Merges an external controller output into the [PD](#) state.
- void [reset](#) ()
Resets the [PD](#) controller state.
- void [set_dt](#) (T dt_)
Sets the controller sampling time.
- void [set_fc](#) (T fc_)
Sets the derivative filter cutoff frequency.
- void [set_Kd](#) (T Kd_)
Sets the derivative gain.
- void [set_Kp](#) (T Kp_)
Sets the proportional gain.
- void [set_param](#) (T dt_, T Kp_, T Kd_, T fc_, T u_max_)
Sets the [PD](#) controller parameters.
- void [set_u_max](#) (T u_max_)
Sets the maximum controller output.
- [T update](#) (T x_0, T x)
Computes the [PD](#) control output.

Private Attributes

- [T dt](#) = 0.0
Controller sampling time.
- [T e_k_1](#) = 0.0
Previous control error.
- [T Kd](#) = 0.0
Derivative gain.
- [T Kp](#) = 0.0
Proportional gain.
- bool [start](#) = true

- `T tau` = 0.0
Controller startup state.
- `T u_k_1` = 0.0
Time constant of the derivative low-pass filter.
- `T u_max` = 9999999999
Previous controller output.
- `T u_max` = 9999999999
Maximum controller output.

9.5.1 Detailed Description

template<typename T>
class control_system::pd::PD_LPF_1< T >

PD controller with first-order low-pass filtered derivative.

The `PD_LPF_1` class implements a discrete-time proportional- derivative controller with a first-order low-pass filter associated with the derivative action.

The controller uses the sampling time, proportional gain, derivative gain, and derivative filter cutoff frequency to calculate the control output.

The controller maintains the previous control error and controller output as internal state and applies a configurable maximum output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::pd::PD_LPF_1<double> controller;

controller.init(
    0.001,
    Kp,
    Kd,
    10.0,
    100.0
);

double output = controller.update(reference, measurement);
```

Definition at line 82 of file `pd_lpf_1.h`.

9.5.2 Constructor & Destructor Documentation

9.5.2.1 PD_LPF_1()

template<typename T>
control_system::pd::PD_LPF_1< T >::PD_LPF_1 ()

Constructs a PD controller with filtered derivative.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file `pd_lpf_1.h`.

9.5.3 Member Function Documentation

9.5.3.1 get_dt()

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 97 of file [pd_lpf_1.h](#).

9.5.3.2 get_e_k_1()

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 121 of file [pd_lpf_1.h](#).

9.5.3.3 get_fc()

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::get_fc ()
```

Gets the derivative filter cutoff frequency.

Returns

Cutoff frequency in Hz.

Definition at line 115 of file [pd_lpf_1.h](#).

9.5.3.4 get_Kd()

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 109 of file [pd_lpf_1.h](#).

9.5.3.5 `get_Kp()`

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 103 of file [pd_lpf_1.h](#).

9.5.3.6 `get_u_k_1()`

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::get_u_k_1 ()
```

Gets the previous controller output.

Returns

Previous controller output.

Definition at line 127 of file [pd_lpf_1.h](#).

9.5.3.7 `get_u_max()`

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 133 of file [pd_lpf_1.h](#).

9.5.3.8 init()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::init (
    T dt_,
    T Kp_,
    T Kd_,
    T fc_,
    T u_max_)
```

Initializes the [PD](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Kd_</i>	Derivative gain.
in	<i>fc_</i>	Derivative filter cutoff frequency in Hz.
in	<i>u_↔ max_</i>	Maximum allowed controller output.

Initializes the controller sampling time, proportional and derivative gains, derivative filter cutoff frequency, and output limit.

Definition at line 17 of file [pd_lpf_1.h](#).

9.5.3.9 merge()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::merge (
    T u_k_l_)
```

Merges an external controller output into the [PD](#) state.

Parameters

in	<i>u_k_↔ l_</i>	Previous or externally provided controller output.
----	---------------------	--

Updates the internal previous-output state using the specified controller output.

Definition at line 62 of file [pd_lpf_1.h](#).

9.5.3.10 reset()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::reset ()
```

Resets the [PD](#) controller state.

Resets the stored control error, previous controller output, and startup state.

Definition at line [54](#) of file [pd_lpf_1.h](#).

9.5.3.11 set_dt()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt_{\leftarrow}$ $_ \leftarrow$	Sampling time in seconds.
----	--------------------------------------	---------------------------

Definition at line [67](#) of file [pd_lpf_1.h](#).

9.5.3.12 set_fc()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::set_fc (
    T fc_)
```

Sets the derivative filter cutoff frequency.

Parameters

in	$fc_{\leftarrow}$ $_ \leftarrow$	Cutoff frequency in Hz.
----	--------------------------------------	-------------------------

Definition at line [85](#) of file [pd_lpf_1.h](#).

9.5.3.13 set_Kd()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd_{\leftarrow}$	Derivative gain.
	—	

Definition at line 79 of file [pd_lpf_1.h](#).

9.5.3.14 set_Kp()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 73 of file [pd_lpf_1.h](#).

9.5.3.15 set_param()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::set_param (
    T dt_,
    T Kp_,
    T Kd_,
    T fc_,
    T u_max_)
```

Sets the [PD](#) controller parameters.

Parameters

in	$dt_{\leftarrow}$	Controller sampling time in seconds.
in	$Kp_{\leftarrow}$	Proportional gain.

in	$Kd_{_}$	Derivative gain.
in	$fc_{_}$	Derivative filter cutoff frequency in Hz.
in	$u_{\leftrightarrow max_}$	Maximum allowed controller output.

Updates the controller sampling time, proportional and derivative gains, derivative filter cutoff frequency, and output limit.

Definition at line 24 of file [pd_lpf_1.h](#).

9.5.3.16 set_u_max()

```
template<typename T>
void control_system::pd::PD_LPF_1< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_{\leftrightarrow max_}$	Maximum allowed controller output.
----	-----------------------------	------------------------------------

Definition at line 91 of file [pd_lpf_1.h](#).

9.5.3.17 update()

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::update (
    T x_0,
    T x)
```

Computes the [PD](#) control output.

Parameters

in	$x_{\leftrightarrow 0}$	Reference or desired value.
in	x	Current or measured value.

Returns

[PD](#) controller output limited by the configured maximum output.

Calculates the proportional and derivative contributions using the current and previous control errors. The derivative action is associated with a first-order low-pass filter.

Definition at line 34 of file [pd_lpf_1.h](#).

9.5.4 Member Data Documentation

9.5.4.1 dt

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time PD controller in seconds.

Definition at line 256 of file [pd_lpf_1.h](#).

9.5.4.2 e_k_1

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line 264 of file [pd_lpf_1.h](#).

9.5.4.3 Kd

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 274 of file [pd_lpf_1.h](#).

9.5.4.4 Kp

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 269 of file [pd_lpf_1.h](#).

9.5.4.5 start

```
template<typename T>
bool control_system::pd::PD_LPF_1< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line 282 of file [pd_lpf_1.h](#).

9.5.4.6 tau

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::tau = 0.0 [private]
```

Time constant of the derivative low-pass filter.

Time constant associated with the first-order derivative filter.

Definition at line 299 of file [pd_lpf_1.h](#).

9.5.4.7 u_k_1

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::u_k_1 = 0.0 [private]
```

Previous controller output.

Stores the controller output from the previous iteration.

Definition at line 290 of file [pd_lpf_1.h](#).

9.5.4.8 u_max

```
template<typename T>
T control_system::pd::PD_LPF_1< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 308 of file [pd_lpf_1.h](#).

The documentation for this class was generated from the following files:

- [include/pd/pd_lpf_1.h](#)
- [src/pd/pd_lpf_1.hpp](#)

9.6 control_system::pi::PI< T > Class Template Reference

Discrete-time proportional-integral controller.

```
#include <pi.h>
```


Public Member Functions

- [PI](#) ()
Constructs a [PI](#) controller.
- [T get_dt](#) ()
Gets the controller sampling time.
- [T get_Ki](#) ()
Gets the integral gain.
- [T get_Kp](#) ()
Gets the proportional gain.
- [T get_u_k_1](#) ()
Gets the previous controller output.
- [T get_u_max](#) ()
Gets the maximum controller output.
- [void init](#) (T dt_, T Kp_, T Ki_, T u_k_1_, T u_max_)
Initializes the [PI](#) controller.
- [void merge](#) (T u_k_1_)
Merges an external controller output into the [PI](#) state.
- [void reset](#) ()
Resets the [PI](#) controller state.
- [void set_dt](#) (T dt_)
Sets the controller sampling time.
- [void set_Ki](#) (T Ki_)
Sets the integral gain.
- [void set_Kp](#) (T Kp_)
Sets the proportional gain.
- [void set_param](#) (T dt_, T Kp_, T Ki_, T u_k_1_, T u_max_)
Sets the [PI](#) controller parameters.
- [void set_u_0](#) (T u_k_1_)
Sets the previous controller output.
- [void set_u_max](#) (T u_max_)
Sets the maximum controller output.
- [T update](#) (T x_0, T x)
Computes the [PI](#) control output.

Private Attributes

- [T dt](#) = 0.0
Controller sampling time.
- [T e_k_1](#) = 0.0
Previous control error.
- [T Ki](#) = 0.0
Integral gain.
- [T Kp](#) = 0.0
Proportional gain.
- [bool start](#) = true
Controller startup state.
- [T u_k_1](#) = 0.0
Previous controller output.
- [T u_max](#) = 9999999999
Maximum controller output.

9.6.1 Detailed Description

`template<typename T>`
`class control_system::pi::PI< T >`

Discrete-time proportional-integral controller.

The [PI](#) class implements a discrete-time proportional-integral controller using configurable proportional and integral gains.

The controller uses the sampling time for its discrete-time integral action and maintains the previous control error and controller output as internal state.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::pi::PI<double> controller;

controller.init(
    0.001,
    Kp,
    Ki,
    0.0,
    100.0
);

double output = controller.update(reference, measurement);
```

Definition at line [76](#) of file [pi.h](#).

9.6.2 Constructor & Destructor Documentation

9.6.2.1 PI()

`template<typename T>`
`control_system::pi::PI< T >::PI ()`

Constructs a [PI](#) controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line [5](#) of file [pi.h](#).

9.6.3 Member Function Documentation

9.6.3.1 get_dt()

`template<typename T>`
`T control_system::pi::PI< T >::get_dt ()`

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line [97](#) of file [pi.h](#).

9.6.3.2 get_Ki()

```
template<typename T>
T control_system::pi::PI< T >::get_Ki ()
```

Gets the integral gain.

Returns

Integral gain.

Definition at line 109 of file [pi.h](#).

9.6.3.3 get_Kp()

```
template<typename T>
T control_system::pi::PI< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 103 of file [pi.h](#).

9.6.3.4 get_u_k_1()

```
template<typename T>
T control_system::pi::PI< T >::get_u_k_1 ()
```

Gets the previous controller output.

Returns

Previous controller output.

Definition at line 115 of file [pi.h](#).

9.6.3.5 get_u_max()

```
template<typename T>
T control_system::pi::PI< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 121 of file [pi.h](#).

9.6.3.6 init()

```
template<typename T>
void control_system::pi::PI< T >::init (
    T dt_,
    T Kp_,
    T Ki_,
    T u_k_1_,
    T u_max_)
```

Initializes the [PI](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Ki_</i>	Integral gain.
in	<i>u_k_↔ 1_</i>	Previous controller output.
in	<i>u_↔ max_</i>	Maximum allowed controller output.

Initializes the controller sampling time, proportional and integral gains, previous controller output, and output limit.

Definition at line [16](#) of file [pi.h](#).

9.6.3.7 merge()

```
template<typename T>
void control_system::pi::PI< T >::merge (
    T u_k_1_)
```

Merges an external controller output into the [PI](#) state.

Parameters

in	<i>u_k_↔ 1_</i>	Previous or externally provided controller output.
----	---------------------	--

Updates the internal previous-output state using the specified controller output.

Definition at line [61](#) of file [pi.h](#).

9.6.3.8 reset()

```
template<typename T>
void control_system::pi::PI< T >::reset ()
```

Resets the **PI** controller state.

Resets the stored previous control error, previous controller output, and startup state.

Definition at line 53 of file [pi.h](#).

9.6.3.9 set_dt()

```
template<typename T>
void control_system::pi::PI< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt_{\leftarrow}$ _←	Sampling time in seconds.
----	-------------------------	---------------------------

Definition at line 67 of file [pi.h](#).

9.6.3.10 set_Ki()

```
template<typename T>
void control_system::pi::PI< T >::set_Ki (
    T Ki_)
```

Sets the integral gain.

Parameters

in	$Ki_{\leftarrow}$ _←	Integral gain.
----	-------------------------	----------------

Definition at line 79 of file [pi.h](#).

9.6.3.11 set_Kp()

```
template<typename T>
void control_system::pi::PI< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 73 of file [pi.h](#).

9.6.3.12 set_param()

```
template<typename T>
void control_system::pi::PI< T >::set_param (
    T dt_,
    T Kp_,
    T Ki_,
    T u_k_1_,
    T u_max_)
```

Sets the [PI](#) controller parameters.

Parameters

in	$dt_{\leftarrow}$	Controller sampling time in seconds.
in	$Kp_{\leftarrow}$	Proportional gain.
in	$Ki_{\leftarrow}$	Integral gain.
in	$u_{k_{\leftarrow}1_{\leftarrow}}$	Previous controller output.
in	$u_{\leftarrow max_{\leftarrow}}$	Maximum allowed controller output.

Updates the controller sampling time, gains, previous controller output, and maximum output limit.

Definition at line 23 of file [pi.h](#).

9.6.3.13 set_u_0()

```
template<typename T>
void control_system::pi::PI< T >::set_u_0 (
    T u_k_1_)
```

Sets the previous controller output.

Parameters

in	u_{k-1}	Previous controller output.
----	-----------	-----------------------------

Definition at line 85 of file [pi.h](#).

9.6.3.14 set_u_max()

```
template<typename T>
void control_system::pi::PI< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	u_{max}	Maximum allowed controller output.
----	-----------	------------------------------------

Definition at line 91 of file [pi.h](#).

9.6.3.15 update()

```
template<typename T>
T control_system::pi::PI< T >::update (
    T x_0,
    T x)
```

Computes the [PI](#) control output.

Parameters

in	x_0	Reference or desired value.
in	x	Current or measured value.

Returns

[PI](#) controller output limited by the configured maximum output.

Calculates the proportional and integral contributions using the current control error and the stored controller state.

Definition at line 33 of file [pi.h](#).

9.6.4 Member Data Documentation

9.6.4.1 dt

```
template<typename T>
T control_system::pi::PI< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time [PI](#) controller in seconds.

Definition at line [233](#) of file [pi.h](#).

9.6.4.2 e_k_1

```
template<typename T>
T control_system::pi::PI< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line [249](#) of file [pi.h](#).

9.6.4.3 Ki

```
template<typename T>
T control_system::pi::PI< T >::Ki = 0.0 [private]
```

Integral gain.

Definition at line [259](#) of file [pi.h](#).

9.6.4.4 Kp

```
template<typename T>
T control_system::pi::PI< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line [254](#) of file [pi.h](#).

9.6.4.5 start

```
template<typename T>
bool control_system::pi::PI< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line [276](#) of file [pi.h](#).

9.6.4.6 u_k_1

```
template<typename T>
T control_system::pi::PI< T >::u_k_1 = 0.0 [private]
```

Previous controller output.

Stores the controller output from the previous iteration.

Definition at line 241 of file [pi.h](#).

9.6.4.7 u_max

```
template<typename T>
T control_system::pi::PI< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 268 of file [pi.h](#).

The documentation for this class was generated from the following files:

- [include/pi/pi.h](#)
- [src/pi/pi.hpp](#)

9.7 control_system::pid::PID< T > Class Template Reference

Discrete-time proportional-integral-derivative controller.

```
#include <pid.h>
```

Public Member Functions

- [PID \(\)](#)
Constructs a [PID](#) controller.
- [T get_dt \(\)](#)
Gets the controller sampling time.
- [T get_e_k_1 \(\)](#)
Gets the previous control error.
- [T get_e_k_2 \(\)](#)
Gets the control error from two iterations ago.
- [T get_Kd \(\)](#)
Gets the derivative gain.
- [T get_Ki \(\)](#)
Gets the integral gain.
- [T get_Kp \(\)](#)
Gets the proportional gain.
- [T get_u_k_1 \(\)](#)

- Gets the previous controller output.*

 - T `get_u_max` ()
- Gets the maximum controller output.*

 - void `init` (T dt_, T Kp_, T Ki_, T Kd_, T u_k_1_, T u_max_)
- Initializes the `PID` controller.*

 - void `merge` (T u_k_1_)
- Merges an external controller output into the `PID` state.*

 - void `reset` ()
- Resets the `PID` controller state.*

 - void `set_dt` (T dt_)
- Sets the controller sampling time.*

 - void `set_Kd` (T Kd_)
- Sets the derivative gain.*

 - void `set_Ki` (T Ki_)
- Sets the integral gain.*

 - void `set_Kp` (T Kp_)
- Sets the proportional gain.*

 - void `set_param` (T dt_, T Kp_, T Ki_, T Kd_, T u_k_1_, T u_max_)
- Sets the `PID` controller parameters.*

 - void `set_u_0` (T u_k_1_)
- Sets the previous controller output.*

 - void `set_u_max` (T u_max_)
- Sets the maximum controller output.*

 - T `update` (T x_0, T x)
- Computes the `PID` control output.*

Private Attributes

- T `dt` = 0.0

Controller sampling time.
- T `e_k_1` = 0.0

Previous control error.
- T `e_k_2` = 0.0

Control error from two iterations ago.
- T `Kd` = 0.0

Derivative gain.
- T `Ki` = 0.0

Integral gain.
- T `Kp` = 0.0

Proportional gain.
- uint8_t `start` = 0

Controller startup state.
- T `u_k_1` = 0.0

Previous controller output.
- T `u_max` = 999999999

Maximum controller output.

9.7.1 Detailed Description

template<typename T>
class control_system::pid::PID< T >

Discrete-time proportional-integral-derivative controller.

The [PID](#) class implements a discrete-time [PID](#) controller using configurable proportional, integral, and derivative gains.

The controller uses the sampling time to calculate the discrete-time integral and derivative contributions.

The previous control output and previous error values are maintained internally as controller state.

Template Parameters

<i>T</i>	Numeric data type used for the controller.
----------	--

Example

```
control_system::pid::PID<double> controller;

controller.init(
    0.001,
    Kp,
    Ki,
    Kd,
    0.0,
    100.0
);

double output = controller.update(reference, measurement);
```

Definition at line [82](#) of file [pid.h](#).

9.7.2 Constructor & Destructor Documentation

9.7.2.1 PID()

template<typename T>
control_system::pid::PID< T >::PID ()

Constructs a [PID](#) controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line [5](#) of file [pid.h](#).

9.7.3 Member Function Documentation

9.7.3.1 get_dt()

template<typename T>
T control_system::pid::PID< T >::get_dt ()

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line [108](#) of file [pid.h](#).

9.7.3.2 `get_e_k_1()`

```
template<typename T>
T control_system::pid::PID< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 138 of file [pid.h](#).

9.7.3.3 `get_e_k_2()`

```
template<typename T>
T control_system::pid::PID< T >::get_e_k_2 ()
```

Gets the control error from two iterations ago.

Returns

Control error from two controller iterations ago.

Definition at line 144 of file [pid.h](#).

9.7.3.4 `get_Kd()`

```
template<typename T>
T control_system::pid::PID< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 126 of file [pid.h](#).

9.7.3.5 `get_Ki()`

```
template<typename T>
T control_system::pid::PID< T >::get_Ki ()
```

Gets the integral gain.

Returns

Integral gain.

Definition at line 120 of file [pid.h](#).

9.7.3.6 get_Kp()

```
template<typename T>
T control_system::pid::PID< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 114 of file [pid.h](#).

9.7.3.7 get_u_k_1()

```
template<typename T>
T control_system::pid::PID< T >::get_u_k_1 ()
```

Gets the previous controller output.

Returns

Previous controller output.

Definition at line 132 of file [pid.h](#).

9.7.3.8 get_u_max()

```
template<typename T>
T control_system::pid::PID< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 150 of file [pid.h](#).

9.7.3.9 init()

```
template<typename T>
void control_system::pid::PID< T >::init (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T u_k_1_,
    T u_max_)
```

Initializes the [PID](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Ki_</i>	Integral gain.
in	<i>Kd_</i>	Derivative gain.
in	<i>u_k_1</i>	Previous controller output.
in	<i>u_↔ max_</i>	Maximum allowed controller output.

Initializes the controller parameters and previous output state.

Definition at line 18 of file [pid.h](#).

9.7.3.10 merge()

```
template<typename T>
void control_system::pid::PID< T >::merge (
    T u_k_1_)
```

Merges an external controller output into the [PID](#) state.

Parameters

in	<i>u_k_↔ 1_</i>	Previous or externally provided controller output.
----	---------------------	--

Updates the internal previous-output state with the specified value.

Definition at line 66 of file [pid.h](#).

9.7.3.11 reset()

```
template<typename T>
void control_system::pid::PID< T >::reset ()
```

Resets the [PID](#) controller state.

Resets the stored previous errors, previous output, and startup state of the controller.

Definition at line [57](#) of file [pid.h](#).

9.7.3.12 set_dt()

```
template<typename T>
void control_system::pid::PID< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt \leftrightarrow$ _	Sampling time in seconds.
----	---------------------------	---------------------------

Definition at line [72](#) of file [pid.h](#).

9.7.3.13 set_Kd()

```
template<typename T>
void control_system::pid::PID< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd \leftrightarrow$ _	Derivative gain.
----	---------------------------	------------------

Definition at line [90](#) of file [pid.h](#).

9.7.3.14 set_Ki()

```
template<typename T>
void control_system::pid::PID< T >::set_Ki (
    T Ki_)
```

Sets the integral gain.

Parameters

in	$K_{i_}$	Integral gain.
----	-----------	----------------

Definition at line 84 of file [pid.h](#).

9.7.3.15 set_Kp()

```
template<typename T>
void control_system::pid::PID< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$K_{p_}$	Proportional gain.
----	-----------	--------------------

Definition at line 78 of file [pid.h](#).

9.7.3.16 set_param()

```
template<typename T>
void control_system::pid::PID< T >::set_param (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T u_k_1_,
    T u_max_)
```

Sets the [PID](#) controller parameters.

Parameters

in	$dt_$	Controller sampling time in seconds.
----	--------	--------------------------------------

in	$Kp_$	Proportional gain.
in	$Ki_$	Integral gain.
in	$Kd_$	Derivative gain.
in	u_k_1	Previous controller output.
in	$u_ \leftrightarrow max_$	Maximum allowed controller output.

Updates the controller gains, sampling time, previous output, and output limit.

Definition at line 25 of file [pid.h](#).

9.7.3.17 set_u_0()

```
template<typename T>
void control_system::pid::PID< T >::set_u_0 (
    T u_k_1_)
```

Sets the previous controller output.

Parameters

in	$u_k_ \leftrightarrow 1_$	Previous controller output.
----	------------------------------	-----------------------------

Definition at line 96 of file [pid.h](#).

9.7.3.18 set_u_max()

```
template<typename T>
void control_system::pid::PID< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_ \leftrightarrow max_$	Maximum allowed controller output.
----	-----------------------------	------------------------------------

Definition at line 102 of file [pid.h](#).

9.7.3.19 update()

```
template<typename T>
T control_system::pid::PID< T >::update (
    T x_0,
    T x)
```

Computes the [PID](#) control output.

Parameters

in	$x \leftarrow$ _0	Reference or desired value.
in	x	Current or measured value.

Returns

[PID](#) controller output limited by the configured maximum output.

Calculates the control output from the proportional, integral, and derivative contributions using the current control error and the stored controller state.

Definition at line [36](#) of file [pid.h](#).

9.7.4 Member Data Documentation

9.7.4.1 dt

```
template<typename T>
T control_system::pid::PID< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time [PID](#) controller in seconds.

Definition at line [269](#) of file [pid.h](#).

9.7.4.2 e_k_1

```
template<typename T>
T control_system::pid::PID< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line [285](#) of file [pid.h](#).

9.7.4.3 e_k_2

```
template<typename T>
T control_system::pid::PID< T >::e_k_2 = 0.0 [private]
```

Control error from two iterations ago.

Definition at line 290 of file [pid.h](#).

9.7.4.4 Kd

```
template<typename T>
T control_system::pid::PID< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 305 of file [pid.h](#).

9.7.4.5 Ki

```
template<typename T>
T control_system::pid::PID< T >::Ki = 0.0 [private]
```

Integral gain.

Definition at line 300 of file [pid.h](#).

9.7.4.6 Kp

```
template<typename T>
T control_system::pid::PID< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 295 of file [pid.h](#).

9.7.4.7 start

```
template<typename T>
uint8_t control_system::pid::PID< T >::start = 0 [private]
```

Controller startup state.

Indicates the startup state of the discrete-time [PID](#) controller.

Definition at line 323 of file [pid.h](#).

9.7.4.8 u_k_1

```
template<typename T>
T control_system::pid::PID< T >::u_k_1 = 0.0 [private]
```

Previous controller output.

Stores the controller output from the previous iteration.

Definition at line 277 of file [pid.h](#).

9.7.4.9 u_max

```
template<typename T>
T control_system::pid::PID< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 314 of file [pid.h](#).

The documentation for this class was generated from the following files:

- include/pid/[pid.h](#)
- src/pid/[pid.tpp](#)

9.8 control_system::pid::PID_GS< T > Class Template Reference

[PID](#) controller with gain scheduling.

```
#include <pid_gs.h>
```

Public Member Functions

- [PID_GS](#) ()
Constructs a gain-scheduled [PID](#) controller.
- T [get_D](#) ()
Gets the derivative contribution.
- bool [get_d_filter](#) ()
Gets the derivative filter enable state.
- T [get_dt](#) ()
Gets the controller sampling time.
- T [get_e_k_1](#) ()
Gets the previous control error.
- T [get_fc](#) ()
Gets the derivative filter cutoff frequency.
- T [get_ff](#) ()
Gets the feed-forward control input.
- T [get_I](#) ()

- Gets the integral contribution.*

 - T [get_I_max](#) ()

Gets the maximum integral contribution.
- T [get_Kd](#) ()

Gets the derivative gain.
- T [get_Ki](#) ()

Gets the integral gain.
- T [get_Kp](#) ()

Gets the proportional gain.
- T [get_P](#) ()

Gets the proportional contribution.
- T [get_u](#) ()

Gets the controller output.
- T [get_u_max](#) ()

Gets the maximum controller output.
- void [init](#) (T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_=false, T fc_=10.0)

Initializes the [PID](#) controller.
- void [merge](#) (T u_k_1_)

Merges an external controller output into the [PID](#) state.
- void [reset](#) ()

Resets the [PID](#) controller state.
- void [set_d_filter](#) (bool d_filter_)

Enables or disables derivative filtering.
- void [set_dt](#) (T dt_)

Sets the controller sampling time.
- void [set_fc](#) (T fc_)

Sets the derivative filter cutoff frequency.
- void [set_ff](#) (T u_ff_)

Sets the feed-forward control input.
- void [set_gain_lookup_table](#) (T *op_cond_arr_, T *Kp_arr_, T *Ki_arr_, T *Kd_arr_, uint16_t len_arr_)

Configures the gain-scheduling lookup tables.
- void [set_I_max](#) (T I_max_)

Sets the maximum integral contribution.
- void [set_Kd](#) (T Kd_)

Sets the derivative gain.
- void [set_Ki](#) (T Ki_)

Sets the integral gain.
- void [set_Kp](#) (T Kp_)

Sets the proportional gain.
- void [set_param](#) (T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_=false, T fc_=10.0)

Sets the [PID](#) controller parameters.
- void [set_u_max](#) (T u_max_)

Sets the maximum controller output.
- T [update](#) (T x_0, T x, T u_ff_=0.0, T op_cond_=0.0)

Computes the gain-scheduled [PID](#) control output.

Private Member Functions

- T [interpolate](#) (T x, T x1, T y1, T x2, T y2)

Linearly interpolates a value between two lookup-table points.
- void [update_gain](#) (T op_cond_)

Updates the [PID](#) gains according to the operating condition.

Private Attributes

- T **D** = 0.0
Derivative contribution.
- bool **d_filter** = false
Derivative filter enable state.
- T **dt** = 0.0
Controller sampling time.
- T **e_k_1** = 0.0
Previous control error.
- T **fc** = 0.0
Cutoff frequency of the derivative filter.
- T **I** = 0.0
Integral contribution.
- T **I_max** = 0.0
Maximum allowed integral contribution.
- T **Kd** = 0.0
Derivative gain.
- T * **Kd_arr**
Derivative-gain lookup table.
- T **Ki** = 0.0
Integral gain.
- T * **Ki_arr**
Integral-gain lookup table.
- T **Kp** = 0.0
Proportional gain.
- T * **Kp_arr**
Proportional-gain lookup table.
- uint16_t **len_arr**
Number of entries in the gain lookup tables.
- LPF_1 < T > **lpf**
First-order low-pass filter used for derivative filtering.
- T * **op_cond_arr**
Operating-condition lookup table.
- T **P** = 0.0
Proportional contribution.
- bool **start** = true
Controller startup state.
- T **u** = 0.0
PID controller output.
- T **u_ff** = 0.0
Feed-forward control input.
- T **u_max** = 0.0
Maximum allowed controller output.

9.8.1 Detailed Description

`template<typename T>`
`class control_system::pid::PID_GS< T >`

PID controller with gain scheduling.

The `PID_GS` class implements a discrete-time PID controller whose proportional, integral, and derivative gains can be adjusted according to a configurable operating condition.

Gain scheduling is implemented using lookup tables containing operating-condition values and corresponding proportional, integral, and derivative gains. Gain values between lookup-table points can be obtained through interpolation.

The derivative contribution can optionally be filtered using a first-order low-pass filter. A feed-forward contribution can also be added to the controller output.

The integral contribution and final controller output are subject to configurable limits.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::pid::PID_GS<double> controller;

controller.init(
    0.001,
    Kp,
    Ki,
    Kd,
    I_max,
    u_max
);

controller.set_gain_lookup_table(
    op_cond_arr,
    Kp_arr,
    Ki_arr,
    Kd_arr,
    len
);

double output = controller.update(
    reference,
    measurement,
    feed_forward,
    operating_condition
);
```

Definition at line 102 of file `pid_gs.h`.

9.8.2 Constructor & Destructor Documentation

9.8.2.1 PID_GS()

`template<typename T>`
`control_system::pid::PID_GS< T >::PID_GS ()`

Constructs a gain-scheduled PID controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file `pid_gs.h`.

9.8.3 Member Function Documentation

9.8.3.1 `get_D()`

```
template<typename T>  
T control\_system::pid::PID\_GS< T >::get_D ()
```

Gets the derivative contribution.

Returns

Current derivative contribution.

Definition at line [267](#) of file [pid_gs.h](#).

9.8.3.2 `get_d_filter()`

```
template<typename T>  
bool control\_system::pid::PID\_GS< T >::get_d_filter ()
```

Gets the derivative filter enable state.

Returns

`true` if derivative filtering is enabled, otherwise `false`.

Definition at line [237](#) of file [pid_gs.h](#).

9.8.3.3 `get_dt()`

```
template<typename T>  
T control\_system::pid::PID\_GS< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line [201](#) of file [pid_gs.h](#).

9.8.3.4 `get_e_k_1()`

```
template<typename T>  
T control\_system::pid::PID\_GS< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line [279](#) of file [pid_gs.h](#).

9.8.3.5 get_fc()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_fc ()
```

Gets the derivative filter cutoff frequency.

Returns

Cutoff frequency in Hz.

Definition at line 243 of file [pid_gs.h](#).

9.8.3.6 get_ff()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_ff ()
```

Gets the feed-forward control input.

Returns

Feed-forward control input.

Definition at line 249 of file [pid_gs.h](#).

9.8.3.7 get_I()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_I ()
```

Gets the integral contribution.

Returns

Current integral contribution.

Definition at line 261 of file [pid_gs.h](#).

9.8.3.8 get_I_max()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_I_max ()
```

Gets the maximum integral contribution.

Returns

Maximum allowed integral contribution.

Definition at line 225 of file [pid_gs.h](#).

9.8.3.9 get_Kd()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Current derivative gain.

Definition at line 219 of file [pid_gs.h](#).

9.8.3.10 get_Ki()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_Ki ()
```

Gets the integral gain.

Returns

Current integral gain.

Definition at line 213 of file [pid_gs.h](#).

9.8.3.11 get_Kp()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Current proportional gain.

Definition at line 207 of file [pid_gs.h](#).

9.8.3.12 get_P()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_P ()
```

Gets the proportional contribution.

Returns

Current proportional contribution.

Definition at line 255 of file [pid_gs.h](#).

9.8.3.13 get_u()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_u ()
```

Gets the controller output.

Returns

Current [PID](#) controller output.

Definition at line [273](#) of file [pid_gs.h](#).

9.8.3.14 get_u_max()

```
template<typename T>
T control_system::pid::PID_GS< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line [231](#) of file [pid_gs.h](#).

9.8.3.15 init()

```
template<typename T>
void control_system::pid::PID_GS< T >::init (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T I_max_,
    T u_max_,
    bool d_filter_ = false,
    T fc_ = 10.0)
```

Initializes the [PID](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Initial proportional gain.

in	<i>Ki_</i>	Initial integral gain.
in	<i>Kd_</i>	Initial derivative gain.
in	<i>I_↔ max_</i>	Maximum allowed integral contribution.
in	<i>u_↔ max_</i>	Maximum allowed controller output.
in	<i>d_↔ filter_</i>	Enables or disables derivative filtering.
in	<i>fc_</i>	Derivative filter cutoff frequency in Hz.

Initializes the controller parameters and optionally configures the derivative low-pass filter.

Definition at line 25 of file [pid_gs.h](#).

9.8.3.16 interpolate()

```
template<typename T>
T control_system::pid::PID_GS< T >::interpolate (
    T x,
    T x1,
    T y1,
    T x2,
    T y2) [private]
```

Linearly interpolates a value between two lookup-table points.

Parameters

in	<i>x</i>	Input value to interpolate.
in	<i>x1</i>	Lower or first independent-variable value.
in	<i>y1</i>	Value corresponding to <i>x1</i> .
in	<i>x2</i>	Upper or second independent-variable value.
in	<i>y2</i>	Value corresponding to <i>x2</i> .

Returns

Interpolated value corresponding to *x*.

Definition at line 285 of file [pid_gs.h](#).

9.8.3.17 merge()

```
template<typename T>
void control_system::pid::PID_GS< T >::merge (
    T u_k_1_)
```

Merges an external controller output into the [PID](#) state.

Parameters

in	u_{k-1}	Previous or externally provided controller output.
----	-----------	--

Updates the internal controller state using the specified previous controller output.

Definition at line 93 of file [pid_gs.h](#).

9.8.3.18 reset()

```
template<typename T>
void control_system::pid::PID_GS< T >::reset ()
```

Resets the [PID](#) controller state.

Resets the stored control error, proportional, integral, derivative, controller output, and startup state.

Definition at line 80 of file [pid_gs.h](#).

9.8.3.19 set_d_filter()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_d_filter (
    bool d_filter_)
```

Enables or disables derivative filtering.

Parameters

in	d_{filter}	Derivative filter enable state.
----	--------------	---------------------------------

Definition at line 182 of file [pid_gs.h](#).

9.8.3.20 set_dt()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt_{\leftarrow}$ _←	Sampling time in seconds.
----	-------------------------	---------------------------

Definition at line 145 of file [pid_gs.h](#).

9.8.3.21 set_fc()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_fc (
    T fc_)
```

Sets the derivative filter cutoff frequency.

Parameters

in	$fc_{\leftarrow}$ _←	Cutoff frequency in Hz.
----	-------------------------	-------------------------

Definition at line 188 of file [pid_gs.h](#).

9.8.3.22 set_ff()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_ff (
    T u_ff_)
```

Sets the feed-forward control input.

Parameters

in	$u_{\leftarrow}$ $ff_{\leftarrow}$	Feed-forward control input.
----	---------------------------------------	-----------------------------

Definition at line 195 of file [pid_gs.h](#).

9.8.3.23 set_gain_lookup_table()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_gain_lookup_table (
    T * op_cond_arr_,
    T * Kp_arr_,
    T * Ki_arr_,
    T * Kd_arr_,
    uint16_t len_arr_)
```

Configures the gain-scheduling lookup tables.

Parameters

in	<i>op_cond_arr_</i>	Array containing operating-condition values.
in	<i>Kp_arr_</i>	Array containing proportional gain values.
in	<i>Ki_arr_</i>	Array containing integral gain values.
in	<i>Kd_arr_</i>	Array containing derivative gain values.
in	<i>len_arr_</i>	Number of elements in each lookup table.

Configures the lookup tables used to determine the proportional, integral, and derivative gains as a function of the operating condition.

Definition at line 98 of file [pid_gs.h](#).

9.8.3.24 set_I_max()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_I_max (
    T I_max_)
```

Sets the maximum integral contribution.

Parameters

in	<i>I_max_</i>	Maximum allowed integral contribution.
----	---------------	--

Definition at line 170 of file [pid_gs.h](#).

9.8.3.25 set_Kd()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd_{\leftarrow}$	Derivative gain.
	—	

Definition at line 164 of file [pid_gs.h](#).

9.8.3.26 set_Ki()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_Ki (
    T Ki_)
```

Sets the integral gain.

Parameters

in	$Ki_{\leftarrow}$	Integral gain.
	—	

Definition at line 158 of file [pid_gs.h](#).

9.8.3.27 set_Kp()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 152 of file [pid_gs.h](#).

9.8.3.28 set_param()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_param (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T I_max_,
    T u_max_,
    bool d_filter_ = false,
    T fc_ = 10.0)
```

Sets the [PID](#) controller parameters.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Ki_</i>	Integral gain.
in	<i>Kd_</i>	Derivative gain.
in	<i>I_↔ max_</i>	Maximum allowed integral contribution.
in	<i>u_↔ max_</i>	Maximum allowed controller output.
in	<i>d_↔ filter_</i>	Enables or disables derivative filtering.
in	<i>fc_</i>	Derivative filter cutoff frequency in Hz.

Updates the controller parameters and derivative filter configuration.

Definition at line 32 of file [pid_gs.h](#).

9.8.3.29 set_u_max()

```
template<typename T>
void control_system::pid::PID_GS< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_{\leftarrow max_}$	Maximum allowed controller output.
----	------------------------	------------------------------------

Definition at line 176 of file [pid_gs.h](#).

9.8.3.30 update()

```
template<typename T>
T control\_system::pid::PID\_GS< T >::update (
    T  $x_0$ ,
    T  $x$ ,
    T  $u_{ff\_}$  = 0.0,
    T  $op_{cond\_}$  = 0.0)
```

Computes the gain-scheduled [PID](#) control output.

Parameters

in	x_0	Reference or desired value.
in	x	Current or measured value.
in	$u_{ff_}$	Feed-forward control input.
in	$op_{\leftarrow cond_}$	Current operating condition used for gain scheduling.

Returns

[PID](#) controller output.

Updates the [PID](#) gains according to the supplied operating condition, calculates the proportional, integral, derivative, and feed-forward contributions, and produces the controller output.

The integral contribution and final controller output are subject to their configured limits.

Definition at line 46 of file [pid_gs.h](#).

9.8.3.31 update_gain()

```
template<typename T>
void control\_system::pid::PID\_GS< T >::update_gain (
    T  $op_{cond\_}$ ) [private]
```

Updates the [PID](#) gains according to the operating condition.

Parameters

in	<i>op_↔ cond_</i>	Current operating condition.
----	-----------------------	------------------------------

Determines the proportional, integral, and derivative gains corresponding to the supplied operating condition using the configured gain lookup tables.

Definition at line 108 of file [pid_gs.h](#).

9.8.4 Member Data Documentation**9.8.4.1 D**

```
template<typename T>
T control_system::pid::PID_GS< T >::D = 0.0 [private]
```

Derivative contribution.

Definition at line 468 of file [pid_gs.h](#).

9.8.4.2 d_filter

```
template<typename T>
bool control_system::pid::PID_GS< T >::d_filter = false [private]
```

Derivative filter enable state.

Definition at line 440 of file [pid_gs.h](#).

9.8.4.3 dt

```
template<typename T>
T control_system::pid::PID_GS< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time [PID](#) controller in seconds.

Definition at line 410 of file [pid_gs.h](#).

9.8.4.4 e_k_1

```
template<typename T>
T control_system::pid::PID_GS< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Definition at line 453 of file [pid_gs.h](#).

9.8.4.5 fc

```
template<typename T>
T control_system::pid::PID_GS< T >::fc = 0.0 [private]
```

Cutoff frequency of the derivative filter.

Cutoff frequency in Hz.

Definition at line 448 of file [pid_gs.h](#).

9.8.4.6 I

```
template<typename T>
T control_system::pid::PID_GS< T >::I = 0.0 [private]
```

Integral contribution.

Definition at line 463 of file [pid_gs.h](#).

9.8.4.7 I_max

```
template<typename T>
T control_system::pid::PID_GS< T >::I_max = 0.0 [private]
```

Maximum allowed integral contribution.

Definition at line 430 of file [pid_gs.h](#).

9.8.4.8 Kd

```
template<typename T>
T control_system::pid::PID_GS< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 425 of file [pid_gs.h](#).

9.8.4.9 Kd_arr

```
template<typename T>
T* control_system::pid::PID_GS< T >::Kd_arr [private]
```

Derivative-gain lookup table.

Definition at line 502 of file [pid_gs.h](#).

9.8.4.10 Ki

```
template<typename T>
T control_system::pid::PID_GS< T >::Ki = 0.0 [private]
```

Integral gain.

Definition at line 420 of file [pid_gs.h](#).

9.8.4.11 Ki_arr

```
template<typename T>
T* control_system::pid::PID_GS< T >::Ki_arr [private]
```

Integral-gain lookup table.

Definition at line 497 of file [pid_gs.h](#).

9.8.4.12 Kp

```
template<typename T>
T control_system::pid::PID_GS< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 415 of file [pid_gs.h](#).

9.8.4.13 Kp_arr

```
template<typename T>
T* control_system::pid::PID_GS< T >::Kp_arr [private]
```

Proportional-gain lookup table.

Definition at line 492 of file [pid_gs.h](#).

9.8.4.14 len_arr

```
template<typename T>
uint16_t control_system::pid::PID_GS< T >::len_arr [private]
```

Number of entries in the gain lookup tables.

Definition at line 507 of file [pid_gs.h](#).

9.8.4.15 lpf

```
template<typename T>
LPF_1<T> control_system::pid::PID_GS< T >::lpf [private]
```

First-order low-pass filter used for derivative filtering.

Definition at line 376 of file [pid_gs.h](#).

9.8.4.16 op_cond_arr

```
template<typename T>
T* control_system::pid::PID_GS< T >::op_cond_arr [private]
```

Operating-condition lookup table.

Stores the operating-condition values corresponding to the scheduled [PID](#) gains.

Definition at line [487](#) of file [pid_gs.h](#).

9.8.4.17 P

```
template<typename T>
T control_system::pid::PID_GS< T >::P = 0.0 [private]
```

Proportional contribution.

Definition at line [458](#) of file [pid_gs.h](#).

9.8.4.18 start

```
template<typename T>
bool control_system::pid::PID_GS< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line [515](#) of file [pid_gs.h](#).

9.8.4.19 u

```
template<typename T>
T control_system::pid::PID_GS< T >::u = 0.0 [private]
```

[PID](#) controller output.

Definition at line [473](#) of file [pid_gs.h](#).

9.8.4.20 u_ff

```
template<typename T>
T control_system::pid::PID_GS< T >::u_ff = 0.0 [private]
```

Feed-forward control input.

Definition at line [478](#) of file [pid_gs.h](#).

9.8.4.21 u_max

```
template<typename T>
T control_system::pid::PID_GS< T >::u_max = 0.0 [private]
```

Maximum allowed controller output.

Definition at line 435 of file [pid_gs.h](#).

The documentation for this class was generated from the following files:

- [include/pid/pid_gs.h](#)
- [src/pid/pid_gs.cpp](#)

9.9 control_system::pid::PID_LPF_1< T > Class Template Reference

[PID](#) controller with a first-order low-pass filtered derivative.

```
#include <pid_lpf_1.h>
```

Public Member Functions

- [PID_LPF_1](#) ()
Constructs a [PID](#) controller with filtered derivative.
- T [get_dt](#) ()
Gets the controller sampling time.
- T [get_e_k_1](#) ()
Gets the previous control error.
- T [get_e_k_2](#) ()
Gets the control error from two iterations ago.
- T [get_fc](#) ()
Gets the derivative filter cutoff frequency.
- T [get_Kd](#) ()
Gets the derivative gain.
- T [get_Ki](#) ()
Gets the integral gain.
- T [get_Kp](#) ()
Gets the proportional gain.
- T [get_u_k_1](#) ()
Gets the previous controller output.
- T [get_u_max](#) ()
Gets the maximum controller output.
- void [init](#) (T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_)
Initializes the [PID](#) controller.
- void [merge](#) (T u_k_1_)
Merges an external controller output into the [PID](#) state.
- void [reset](#) ()
Resets the [PID](#) controller state.
- void [set_dt](#) (T dt_)

- Sets the controller sampling time.*
 - void `set_fc` (T fc_)
- Sets the derivative filter cutoff frequency.*
 - void `set_Kd` (T Kd_)
- Sets the derivative gain.*
 - void `set_Ki` (T Ki_)
- Sets the integral gain.*
 - void `set_Kp` (T Kp_)
- Sets the proportional gain.*
 - void `set_param` (T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_)
- Sets the [PID](#) controller parameters.*
 - void `set_u_0` (T u_k_1_)
- Sets the previous controller output.*
 - void `set_u_max` (T u_max_)
- Sets the maximum controller output.*
 - T `update` (T x_0, T x)
- Computes the [PID](#) control output.*

Private Attributes

- T `dt` = 0.0
Controller sampling time.
- T `e_k_1` = 0.0
Previous control error.
- T `e_k_2` = 0.0
Control error from two iterations ago.
- T `Kd` = 0.0
Derivative gain.
- T `Ki` = 0.0
Integral gain.
- T `Kp` = 0.0
Proportional gain.
- uint8_t `start` = 0
Controller startup state.
- T `tau` = 0.0
Time constant of the derivative low-pass filter.
- T `u_k_1` = 0.0
Previous controller output.
- T `u_k_2` = 0.0
Controller output from two iterations ago.
- T `u_max` = 9999999999
Maximum controller output.

9.9.1 Detailed Description

template<typename T>
class control_system::pid::PID_LPF_1< T >

PID controller with a first-order low-pass filtered derivative.

The `PID_LPF_1` class implements a discrete-time PID controller in which the derivative contribution is filtered using a first-order low-pass filter.

The controller is configured using the sampling time, proportional gain, integral gain, derivative gain, and derivative filter cutoff frequency.

Previous controller outputs and control errors are retained internally to implement the discrete-time controller.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::pid::PID_LPF_1<double> controller;

controller.init(
    0.001,
    Kp,
    Ki,
    Kd,
    10.0,
    0.0,
    0.0,
    100.0
);

double output = controller.update(reference, measurement);
```

Definition at line 86 of file `pid_lpf_1.h`.

9.9.2 Constructor & Destructor Documentation

9.9.2.1 PID_LPF_1()

template<typename T>
control_system::pid::PID_LPF_1< T >::PID_LPF_1 ()

Constructs a PID controller with filtered derivative.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file `pid_lpf_1.h`.

9.9.3 Member Function Documentation

9.9.3.1 get_dt()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 119 of file [pid_lpf_1.h](#).

9.9.3.2 get_e_k_1()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 155 of file [pid_lpf_1.h](#).

9.9.3.3 get_e_k_2()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_e_k_2 ()
```

Gets the control error from two iterations ago.

Returns

Control error from two controller iterations ago.

Definition at line 161 of file [pid_lpf_1.h](#).

9.9.3.4 get_fc()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_fc ()
```

Gets the derivative filter cutoff frequency.

Returns

Cutoff frequency in Hz.

Definition at line 143 of file [pid_lpf_1.h](#).

9.9.3.5 get_Kd()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 137 of file [pid_lpf_1.h](#).

9.9.3.6 get_Ki()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_Ki ()
```

Gets the integral gain.

Returns

Integral gain.

Definition at line 131 of file [pid_lpf_1.h](#).

9.9.3.7 get_Kp()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 125 of file [pid_lpf_1.h](#).

9.9.3.8 get_u_k_1()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_u_k_1 ()
```

Gets the previous controller output.

Returns

Previous controller output.

Definition at line 149 of file [pid_lpf_1.h](#).

9.9.3.9 get_u_max()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 167 of file [pid_lpf_1.h](#).

9.9.3.10 init()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::init (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T fc_,
    T u_k_1_,
    T u_k_2_,
    T u_max_)
```

Initializes the [PID](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Ki_</i>	Integral gain.
in	<i>Kd_</i>	Derivative gain.
in	<i>fc_</i>	Derivative filter cutoff frequency in Hz.
in	<i>u_k_↔ 1_</i>	Previous controller output.
in	<i>u_k_↔ 2_</i>	Controller output from two iterations ago.
in	<i>u_↔ max_</i>	Maximum allowed controller output.

Initializes the [PID](#) controller parameters, derivative filter parameters, previous controller outputs, and controller output limit.

Definition at line 19 of file [pid_lpf_1.h](#).

9.9.3.11 merge()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::merge (
    T u_k_1_)
```

Merges an external controller output into the [PID](#) state.

Parameters

in	u_{k-1}	Previous or externally provided controller output.
----	-----------	--

Updates the internal previous-output state using the specified controller output.

Definition at line 71 of file [pid_lpf_1.h](#).

9.9.3.12 reset()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::reset ()
```

Resets the [PID](#) controller state.

Resets the stored controller outputs, control errors, and startup state.

Definition at line 61 of file [pid_lpf_1.h](#).

9.9.3.13 set_dt()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt_{\leftarrow}$	Sampling time in seconds.
----	-------------------	---------------------------

Definition at line 77 of file [pid_lpf_1.h](#).

9.9.3.14 set_fc()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_fc (
    T fc_)
```

Sets the derivative filter cutoff frequency.

Parameters

in	$fc_{\leftarrow}$ _←	Cutoff frequency in Hz.
----	-------------------------	-------------------------

Definition at line 101 of file [pid_lpf_1.h](#).

9.9.3.15 set_Kd()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd_{\leftarrow}$ —	Derivative gain.
----	------------------------	------------------

Definition at line 95 of file [pid_lpf_1.h](#).

9.9.3.16 set_Ki()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_Ki (
    T Ki_)
```

Sets the integral gain.

Parameters

in	$Ki_{\leftarrow}$ _←	Integral gain.
----	-------------------------	----------------

Definition at line 89 of file [pid_lpf_1.h](#).

9.9.3.17 set_Kp()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 83 of file [pid_lpf_1.h](#).

9.9.3.18 set_param()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_param (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T fc_,
    T u_k_1_,
    T u_k_2_,
    T u_max_)
```

Sets the [PID](#) controller parameters.

Parameters

in	$dt_{\leftarrow}$	Controller sampling time in seconds.
in	$Kp_{\leftarrow}$	Proportional gain.
in	$Ki_{\leftarrow}$	Integral gain.
in	$Kd_{\leftarrow}$	Derivative gain.
in	$fc_{\leftarrow}$	Derivative filter cutoff frequency in Hz.
in	$u_{k_{\leftarrow}1_{\leftarrow}}$	Previous controller output.
in	$u_{k_{\leftarrow}2_{\leftarrow}}$	Controller output from two iterations ago.
in	$u_{\leftarrow max_{\leftarrow}}$	Maximum allowed controller output.

Updates the [PID](#) gains, sampling time, derivative filter cutoff frequency, previous controller outputs, and output limit.

Definition at line 26 of file [pid_lpf_1.h](#).

9.9.3.19 set_u_0()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_u_0 (
    T u_k_1_)
```

Sets the previous controller output.

Parameters

in	u_{k-1}	Previous controller output.
----	-----------	-----------------------------

Definition at line 107 of file [pid_lpf_1.h](#).

9.9.3.20 set_u_max()

```
template<typename T>
void control_system::pid::PID_LPF_1< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	u_{max}	Maximum allowed controller output.
----	-----------	------------------------------------

Definition at line 113 of file [pid_lpf_1.h](#).

9.9.3.21 update()

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::update (
    T x_0,
    T x)
```

Computes the [PID](#) control output.

Parameters

in	x_0	Reference or desired value.
in	x	Current or measured value.

Returns

[PID](#) controller output limited by the configured maximum output.

Calculates the proportional, integral, and derivative contributions using the current and previous control errors. The derivative contribution is filtered using the configured first-order low-pass filter.

Definition at line 39 of file [pid_lpf_1.h](#).

9.9.4 Member Data Documentation

9.9.4.1 dt

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time PID controller in seconds.

Definition at line 294 of file [pid_lpf_1.h](#).

9.9.4.2 e_k_1

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line 315 of file [pid_lpf_1.h](#).

9.9.4.3 e_k_2

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::e_k_2 = 0.0 [private]
```

Control error from two iterations ago.

Definition at line 320 of file [pid_lpf_1.h](#).

9.9.4.4 Kd

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 335 of file [pid_lpf_1.h](#).

9.9.4.5 Ki

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::Ki = 0.0 [private]
```

Integral gain.

Definition at line 330 of file [pid_lpf_1.h](#).

9.9.4.6 Kp

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 325 of file [pid_lpf_1.h](#).

9.9.4.7 start

```
template<typename T>
uint8_t control_system::pid::PID_LPF_1< T >::start = 0 [private]
```

Controller startup state.

Indicates the startup state of the discrete-time PID controller.

Definition at line 362 of file [pid_lpf_1.h](#).

9.9.4.8 tau

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::tau = 0.0 [private]
```

Time constant of the derivative low-pass filter.

Time constant associated with the first-order derivative filter.

Definition at line 344 of file [pid_lpf_1.h](#).

9.9.4.9 u_k_1

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::u_k_1 = 0.0 [private]
```

Previous controller output.

Stores the controller output from the previous iteration.

Definition at line 302 of file [pid_lpf_1.h](#).

9.9.4.10 u_k_2

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::u_k_2 = 0.0 [private]
```

Controller output from two iterations ago.

Definition at line 307 of file [pid_lpf_1.h](#).

9.9.4.11 u_max

```
template<typename T>
T control_system::pid::PID_LPF_1< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 353 of file [pid_lpf_1.h](#).

The documentation for this class was generated from the following files:

- [include/pid/pid_lpf_1.h](#)
- [src/pid/pid_lpf_1.tpp](#)

9.10 control_system::pid::PID_LPF_1_FF< T > Class Template Reference

[PID](#) controller with first-order low-pass filtered derivative and feed-forward.

```
#include <pid_lpf_1_ff.h>
```

Public Member Functions

- [PID_LPF_1_FF \(\)](#)
Constructs a [PID](#) controller with filtered derivative and feed-forward support.
- [T get_dt \(\)](#)
Gets the controller sampling time.
- [T get_e_k_1 \(\)](#)
Gets the previous control error.
- [T get_e_k_2 \(\)](#)
Gets the control error from two iterations ago.
- [T get_fc \(\)](#)
Gets the derivative filter cutoff frequency.
- [T get_ff \(\)](#)
Gets the feed-forward control input.
- [T get_Kd \(\)](#)
Gets the derivative gain.
- [T get_Ki \(\)](#)
Gets the integral gain.
- [T get_Kp \(\)](#)
Gets the proportional gain.
- [T get_u_k_1 \(\)](#)
Gets the previous controller output.
- [T get_u_max \(\)](#)
Gets the maximum controller output.
- [void init \(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_\)](#)
Initializes the [PID](#) controller.

- void `merge` (T u_k_1_)
Merges an external controller output into the `PID` state.
- void `reset` ()
Resets the `PID` controller state.
- void `set_dt` (T dt_)
Sets the controller sampling time.
- void `set_fc` (T fc_)
Sets the derivative filter cutoff frequency.
- void `set_ff` (T u_ff_)
Sets the feed-forward control input.
- void `set_Kd` (T Kd_)
Sets the derivative gain.
- void `set_Ki` (T Ki_)
Sets the integral gain.
- void `set_Kp` (T Kp_)
Sets the proportional gain.
- void `set_param` (T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_)
Sets the `PID` controller parameters.
- void `set_u_0` (T u_k_1_)
Sets the previous controller output.
- void `set_u_max` (T u_max_)
Sets the maximum controller output.
- T `update` (T x_0, T x, T u_ff_=0.0)
Computes the `PID` control output with feed-forward.

Private Attributes

- T `dt` = 0.0
Controller sampling time.
- T `e_k_1` = 0.0
Previous control error.
- T `e_k_2` = 0.0
Control error from two iterations ago.
- T `Kd` = 0.0
Derivative gain.
- T `Ki` = 0.0
Integral gain.
- T `Kp` = 0.0
Proportional gain.
- uint8_t `start` = 0
Controller startup state.
- T `tau` = 0.0
Time constant of the derivative low-pass filter.
- T `u_ff` = 0.0
Feed-forward control input.
- T `u_k_1` = 0.0
Previous controller output.
- T `u_k_2` = 0.0
Controller output from two iterations ago.
- T `u_max` = 9999999999
Maximum controller output.

9.10.1 Detailed Description

```
template<typename T>
class control_system::pid::PID_LPF_1_FF< T >
```

PID controller with first-order low-pass filtered derivative and feed-forward.

The `PID_LPF_1_FF` class implements a discrete-time PID controller with a first-order low-pass filter applied to the derivative contribution.

The controller also supports a feed-forward contribution that can be supplied to the controller output.

The controller is configured using the sampling time, proportional gain, integral gain, derivative gain, derivative filter cutoff frequency, previous controller outputs, and maximum output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::pid::PID_LPF_1_FF<double> controller;

controller.init(
    0.001,
    Kp,
    Ki,
    Kd,
    10.0,
    0.0,
    0.0,
    100.0
);

double output = controller.update(reference, measurement, feed_forward);
```

Definition at line 89 of file `pid_lpf_1_ff.h`.

9.10.2 Constructor & Destructor Documentation

9.10.2.1 PID_LPF_1_FF()

```
template<typename T>
control_system::pid::PID_LPF_1_FF< T >::PID_LPF_1_FF ()
```

Constructs a PID controller with filtered derivative and feed-forward support.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file `pid_lpf_1_ff.h`.

9.10.3 Member Function Documentation

9.10.3.1 `get_dt()`

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 128 of file [pid_lpf_1_ff.h](#).

9.10.3.2 `get_e_k_1()`

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 170 of file [pid_lpf_1_ff.h](#).

9.10.3.3 `get_e_k_2()`

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_e_k_2 ()
```

Gets the control error from two iterations ago.

Returns

Control error from two controller iterations ago.

Definition at line 176 of file [pid_lpf_1_ff.h](#).

9.10.3.4 `get_fc()`

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_fc ()
```

Gets the derivative filter cutoff frequency.

Returns

Cutoff frequency in Hz.

Definition at line 152 of file [pid_lpf_1_ff.h](#).

9.10.3.5 get_ff()

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_ff ()
```

Gets the feed-forward control input.

Returns

Feed-forward control input.

Definition at line 158 of file [pid_lpf_1_ff.h](#).

9.10.3.6 get_Kd()

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 146 of file [pid_lpf_1_ff.h](#).

9.10.3.7 get_Ki()

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_Ki ()
```

Gets the integral gain.

Returns

Integral gain.

Definition at line 140 of file [pid_lpf_1_ff.h](#).

9.10.3.8 get_Kp()

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 134 of file [pid_lpf_1_ff.h](#).

9.10.3.9 `get_u_k_1()`

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_u_k_1 ()
```

Gets the previous controller output.

Returns

Previous controller output.

Definition at line 164 of file [pid_lpf_1_ff.h](#).

9.10.3.10 `get_u_max()`

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 182 of file [pid_lpf_1_ff.h](#).

9.10.3.11 `init()`

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::init (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T fc_,
    T u_k_1_,
    T u_k_2_,
    T u_max_)
```

Initializes the [PID](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.

in	K_i	Integral gain.
in	K_d	Derivative gain.
in	f_c	Derivative filter cutoff frequency in Hz.
in	u_{k-1}	Previous controller output.
in	u_{k-2}	Controller output from two iterations ago.
in	u_{max}	Maximum allowed controller output.

Initializes the [PID](#) controller parameters, derivative filter parameters, previous controller outputs, and output limit.

Definition at line 20 of file [pid_lpf_1_ff.h](#).

9.10.3.12 merge()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::merge (
    T u_k_1_)
```

Merges an external controller output into the [PID](#) state.

Parameters

in	u_{k-1}	Previous or externally provided controller output.
----	-----------	--

Updates the internal previous-output state using the specified controller output.

Definition at line 74 of file [pid_lpf_1_ff.h](#).

9.10.3.13 reset()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::reset ()
```

Resets the [PID](#) controller state.

Resets the stored controller outputs, control errors, feed-forward value, and startup state.

Definition at line 63 of file [pid_lpf_1_ff.h](#).

9.10.3.14 set_dt()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	$dt_{\leftarrow}$ $_{\leftarrow}$	Sampling time in seconds.
----	--------------------------------------	---------------------------

Definition at line 80 of file [pid_lpf_1_ff.h](#).

9.10.3.15 set_fc()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_fc (
    T fc_)
```

Sets the derivative filter cutoff frequency.

Parameters

in	$fc_{\leftarrow}$ $_{\leftarrow}$	Cutoff frequency in Hz.
----	--------------------------------------	-------------------------

Definition at line 104 of file [pid_lpf_1_ff.h](#).

9.10.3.16 set_ff()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_ff (
    T u_ff_)
```

Sets the feed-forward control input.

Parameters

in	$u_{\leftarrow}$ $ff_{\leftarrow}$	Feed-forward control input.
----	---------------------------------------	-----------------------------

Definition at line 110 of file [pid_lpf_1_ff.h](#).

9.10.3.17 set_Kd()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd_{\leftarrow}$	Derivative gain.
	—	

Definition at line 98 of file [pid_lpf_1_ff.h](#).

9.10.3.18 set_Ki()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_Ki (
    T Ki_)
```

Sets the integral gain.

Parameters

in	$Ki_{\leftarrow}$	Integral gain.
	—	

Definition at line 92 of file [pid_lpf_1_ff.h](#).

9.10.3.19 set_Kp()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 86 of file [pid_lpf_1_ff.h](#).

9.10.3.20 set_param()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_param (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T fc_,
    T u_k_1_,
    T u_k_2_,
    T u_max_)
```

Sets the [PID](#) controller parameters.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Ki_</i>	Integral gain.
in	<i>Kd_</i>	Derivative gain.
in	<i>fc_</i>	Derivative filter cutoff frequency in Hz.
in	<i>u_k_↔ 1_</i>	Previous controller output.
in	<i>u_k_↔ 2_</i>	Controller output from two iterations ago.
in	<i>u_↔ max_</i>	Maximum allowed controller output.

Updates the [PID](#) gains, sampling time, derivative filter cutoff frequency, previous controller outputs, and output limit.

Definition at line 27 of file [pid_lpf_1_ff.h](#).

9.10.3.21 set_u_0()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_u_0 (
    T u_k_1_)
```

Sets the previous controller output.

Parameters

in	u_{k-1}	Previous controller output.
----	-----------	-----------------------------

Definition at line 116 of file [pid_lpf_1_ff.h](#).

9.10.3.22 set_u_max()

```
template<typename T>
void control_system::pid::PID_LPF_1_FF< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	u_{max}	Maximum allowed controller output.
----	-----------	------------------------------------

Definition at line 122 of file [pid_lpf_1_ff.h](#).

9.10.3.23 update()

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::update (
    T x_0,
    T x,
    T u_ff_ = 0.0)
```

Computes the [PID](#) control output with feed-forward.

Parameters

in	x_0	Reference or desired value.
in	x	Current or measured value.
in	u_{ff}	Feed-forward control input.

Returns

[PID](#) controller output including the feed-forward contribution and limited by the configured maximum output.

Calculates the proportional, integral, and derivative contributions. The derivative contribution is filtered using the configured first-order low-pass filter, and the feed-forward contribution is incorporated into the final controller output.

Definition at line 40 of file [pid_lpf_1_ff.h](#).

9.10.4 Member Data Documentation

9.10.4.1 dt

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time PID controller in seconds.

Definition at line 313 of file [pid_lpf_1_ff.h](#).

9.10.4.2 e_k_1

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Stores the control error from the previous iteration.

Definition at line 334 of file [pid_lpf_1_ff.h](#).

9.10.4.3 e_k_2

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::e_k_2 = 0.0 [private]
```

Control error from two iterations ago.

Definition at line 339 of file [pid_lpf_1_ff.h](#).

9.10.4.4 Kd

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 354 of file [pid_lpf_1_ff.h](#).

9.10.4.5 Ki

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::Ki = 0.0 [private]
```

Integral gain.

Definition at line 349 of file [pid_lpf_1_ff.h](#).

9.10.4.6 Kp

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 344 of file [pid_lpf_1_ff.h](#).

9.10.4.7 start

```
template<typename T>
uint8_t control_system::pid::PID_LPF_1_FF< T >::start = 0 [private]
```

Controller startup state.

Indicates the startup state of the discrete-time PID controller.

Definition at line 386 of file [pid_lpf_1_ff.h](#).

9.10.4.8 tau

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::tau = 0.0 [private]
```

Time constant of the derivative low-pass filter.

Time constant associated with the first-order derivative filter.

Definition at line 363 of file [pid_lpf_1_ff.h](#).

9.10.4.9 u_ff

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::u_ff = 0.0 [private]
```

Feed-forward control input.

Definition at line 368 of file [pid_lpf_1_ff.h](#).

9.10.4.10 u_k_1

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::u_k_1 = 0.0 [private]
```

Previous controller output.

Stores the controller output from the previous iteration.

Definition at line 321 of file [pid_lpf_1_ff.h](#).

9.10.4.11 u_k_2

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::u_k_2 = 0.0 [private]
```

Controller output from two iterations ago.

Definition at line 326 of file [pid_lpf_1_ff.h](#).

9.10.4.12 u_max

```
template<typename T>
T control_system::pid::PID_LPF_1_FF< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the maximum allowed magnitude of the controller output.

Definition at line 377 of file [pid_lpf_1_ff.h](#).

The documentation for this class was generated from the following files:

- [include/pid/pid_lpf_1_ff.h](#)
- [src/pid/pid_lpf_1_ff.cpp](#)

9.11 control_system::pid::PID_P< T > Class Template Reference

[PID](#) controller with configurable derivative filtering and feed-forward.

```
#include <pid_p.h>
```

Public Member Functions

- [PID_P\(\)](#)
Constructs a [PID](#) controller.
- [T get_D\(\)](#)
Gets the derivative contribution.
- [bool get_d_filter\(\)](#)
Gets the derivative filter enable state.
- [T get_dt\(\)](#)
Gets the controller sampling time.
- [T get_e_k_1\(\)](#)
Gets the previous control error.
- [T get_fc\(\)](#)
Gets the derivative filter cutoff frequency.
- [T get_ff\(\)](#)
Gets the feed-forward control input.
- [T get_I\(\)](#)
Gets the integral contribution.

- T [get_I_max](#) ()
Gets the maximum integral contribution.
- T [get_Kd](#) ()
Gets the derivative gain.
- T [get_Ki](#) ()
Gets the integral gain.
- T [get_Kp](#) ()
Gets the proportional gain.
- T [get_P](#) ()
Gets the proportional contribution.
- T [get_u](#) ()
Gets the controller output.
- T [get_u_max](#) ()
Gets the maximum controller output.
- void [init](#) (T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_=false, T fc_=10.0)
Initializes the [PID](#) controller.
- void [merge](#) (T u_k_1_)
Merges an external controller output into the [PID](#) state.
- void [reset](#) ()
Resets the [PID](#) controller state.
- void [set_d_filter](#) (bool d_filter_)
Enables or disables derivative filtering.
- void [set_dt](#) (T dt_)
Sets the controller sampling time.
- void [set_fc](#) (T fc_)
Sets the derivative filter cutoff frequency.
- void [set_ff](#) (T u_ff_)
Sets the feed-forward control input.
- void [set_I_max](#) (T I_max_)
Sets the maximum integral contribution.
- void [set_Kd](#) (T Kd_)
Sets the derivative gain.
- void [set_Ki](#) (T Ki_)
Sets the integral gain.
- void [set_Kp](#) (T Kp_)
Sets the proportional gain.
- void [set_param](#) (T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_=false, T fc_=10.0)
Sets the [PID](#) controller parameters.
- void [set_u_max](#) (T u_max_)
Sets the maximum controller output.
- T [update](#) (T x_0, T x, T u_ff_=0.0)
Computes the [PID](#) control output.

Private Attributes

- T [D](#) = 0.0
Derivative contribution.
- bool [d_filter](#) = false
Derivative filter enable state.
- T [dt](#) = 0.0

- Controller sampling time.*
 - `T e_k_1` = 0.0
- Previous control error.*
 - `T fc` = 0.0
- Cutoff frequency of the derivative filter.*
 - `T l` = 0.0
- Integral contribution.*
 - `T l_max` = 0.0
- Maximum allowed integral contribution.*
 - `T Kd` = 0.0
- Derivative gain.*
 - `T Ki` = 0.0
- Integral gain.*
 - `T Kp` = 0.0
- Proportional gain.*
 - `LPF_1 < T > lpf`
- First-order low-pass filter used for derivative filtering.*
 - `T P` = 0.0
- Proportional contribution.*
 - `bool start` = true
- Controller startup state.*
 - `T u` = 0.0
- PID controller output.*
 - `T u_ff` = 0.0
- Feed-forward control input.*
 - `T u_max` = 0.0
- Maximum allowed controller output.*

9.11.1 Detailed Description

```
template<typename T>
class control_system::pid::PID_P< T >
```

PID controller with configurable derivative filtering and feed-forward.

The `PID_P` class implements a discrete-time PID controller with configurable proportional, integral, and derivative gains.

The derivative contribution can optionally be passed through a first-order low-pass filter. A feed-forward contribution can also be added to the controller output.

The integral contribution is constrained by the configured integral limit, while the final controller output is constrained by the configured output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::pid::PID_P<double> controller;

controller.init(
    0.001,
    Kp,
    Ki,
    Kd,
    I_max,
    u_max
);

double output = controller.update(reference, measurement);
```

Definition at line 86 of file [pid_p.h](#).

9.11.2 Constructor & Destructor Documentation

9.11.2.1 PID_P()

```
template<typename T>
control_system::pid::PID_P< T >::PID_P ()
```

Constructs a [PID](#) controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file [pid_p.h](#).

9.11.3 Member Function Documentation

9.11.3.1 get_D()

```
template<typename T>
T control_system::pid::PID_P< T >::get_D ()
```

Gets the derivative contribution.

Returns

Current derivative contribution.

Definition at line 218 of file [pid_p.h](#).

9.11.3.2 get_d_filter()

```
template<typename T>
bool control_system::pid::PID_P< T >::get_d_filter ()
```

Gets the derivative filter enable state.

Returns

true if derivative filtering is enabled, otherwise false.

Definition at line 188 of file [pid_p.h](#).

9.11.3.3 `get_dt()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 152 of file [pid_p.h](#).

9.11.3.4 `get_e_k_1()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 230 of file [pid_p.h](#).

9.11.3.5 `get_fc()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_fc ()
```

Gets the derivative filter cutoff frequency.

Returns

Cutoff frequency in Hz.

Definition at line 194 of file [pid_p.h](#).

9.11.3.6 `get_ff()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_ff ()
```

Gets the feed-forward control input.

Returns

Feed-forward control input.

Definition at line 200 of file [pid_p.h](#).

9.11.3.7 get_I()

```
template<typename T>
T control_system::pid::PID_P< T >::get_I ()
```

Gets the integral contribution.

Returns

Current integral contribution.

Definition at line 212 of file [pid_p.h](#).

9.11.3.8 get_I_max()

```
template<typename T>
T control_system::pid::PID_P< T >::get_I_max ()
```

Gets the maximum integral contribution.

Returns

Maximum allowed integral contribution.

Definition at line 176 of file [pid_p.h](#).

9.11.3.9 get_Kd()

```
template<typename T>
T control_system::pid::PID_P< T >::get_Kd ()
```

Gets the derivative gain.

Returns

Derivative gain.

Definition at line 170 of file [pid_p.h](#).

9.11.3.10 get_Ki()

```
template<typename T>
T control_system::pid::PID_P< T >::get_Ki ()
```

Gets the integral gain.

Returns

Integral gain.

Definition at line 164 of file [pid_p.h](#).

9.11.3.11 `get_Kp()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 158 of file [pid_p.h](#).

9.11.3.12 `get_P()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_P ()
```

Gets the proportional contribution.

Returns

Current proportional contribution.

Definition at line 206 of file [pid_p.h](#).

9.11.3.13 `get_u()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_u ()
```

Gets the controller output.

Returns

Current [PID](#) controller output.

Definition at line 224 of file [pid_p.h](#).

9.11.3.14 `get_u_max()`

```
template<typename T>
T control_system::pid::PID_P< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 182 of file [pid_p.h](#).

9.11.3.15 init()

```
template<typename T>
void control_system::pid::PID_P< T >::init (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T I_max_,
    T u_max_,
    bool d_filter_ = false,
    T fc_ = 10.0)
```

Initializes the [PID](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Ki_</i>	Integral gain.
in	<i>Kd_</i>	Derivative gain.
in	<i>I_↔ max_</i>	Maximum allowed integral contribution.
in	<i>u_↔ max_</i>	Maximum allowed controller output.
in	<i>d_↔ filter_</i>	Enables or disables derivative filtering.
in	<i>fc_</i>	Cutoff frequency of the derivative filter in Hz.

Initializes the controller parameters and optionally configures the derivative low-pass filter.

Definition at line 25 of file [pid_p.h](#).

9.11.3.16 merge()

```
template<typename T>
void control_system::pid::PID_P< T >::merge (
    T u_k_l_)
```

Merges an external controller output into the [PID](#) state.

Parameters

in	u_{k-1}	Previous or externally provided controller output.
----	-----------	--

Updates the internal controller state using the specified previous controller output.

Definition at line 91 of file [pid_p.h](#).

9.11.3.17 reset()

```
template<typename T>
void control_system::pid::PID_P< T >::reset ()
```

Resets the **PID** controller state.

Resets the stored control error, proportional, integral, derivative, controller output, feed-forward value, and startup state.

Definition at line 78 of file [pid_p.h](#).

9.11.3.18 set_d_filter()

```
template<typename T>
void control_system::pid::PID_P< T >::set_d_filter (
    bool d_filter_)
```

Enables or disables derivative filtering.

Parameters

in	d_{filter}	Derivative filter enable state.
----	--------------	---------------------------------

Definition at line 133 of file [pid_p.h](#).

9.11.3.19 set_dt()

```
template<typename T>
void control_system::pid::PID_P< T >::set_dt (
    T dt_)
```

Sets the controller sampling time.

Parameters

in	dt	Sampling time in seconds.
----	------	---------------------------

Definition at line 96 of file [pid_p.h](#).

9.11.3.20 set_fc()

```
template<typename T>
void control_system::pid::PID_P< T >::set_fc (
    T fc_)
```

Sets the derivative filter cutoff frequency.

Parameters

in	$fc_{\leftarrow}$ —	Cutoff frequency in Hz.
----	------------------------	-------------------------

Definition at line 139 of file [pid_p.h](#).

9.11.3.21 set_ff()

```
template<typename T>
void control_system::pid::PID_P< T >::set_ff (
    T u_ff_)
```

Sets the feed-forward control input.

Parameters

in	$u_{\leftarrow}$ $ff_{\leftarrow}$	Feed-forward control input.
----	---------------------------------------	-----------------------------

Definition at line 146 of file [pid_p.h](#).

9.11.3.22 set_I_max()

```
template<typename T>
void control_system::pid::PID_P< T >::set_I_max (
    T I_max_)
```

Sets the maximum integral contribution.

Parameters

in	$I_{\leftarrow}$ $max_{\leftarrow}$ —	Maximum allowed integral contribution.
----	---	--

Definition at line 121 of file [pid_p.h](#).

9.11.3.23 set_Kd()

```
template<typename T>
void control_system::pid::PID_P< T >::set_Kd (
    T Kd_)
```

Sets the derivative gain.

Parameters

in	$Kd_{\leftarrow}$	Derivative gain.
	—	

Definition at line 115 of file [pid_p.h](#).

9.11.3.24 set_Ki()

```
template<typename T>
void control_system::pid::PID_P< T >::set_Ki (
    T Ki_)
```

Sets the integral gain.

Parameters

in	$Ki_{\leftarrow}$	Integral gain.
	—	

Definition at line 109 of file [pid_p.h](#).

9.11.3.25 set_Kp()

```
template<typename T>
void control_system::pid::PID_P< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 103 of file [pid_p.h](#).

9.11.3.26 set_param()

```
template<typename T>
void control_system::pid::PID_P< T >::set_param (
    T dt_,
    T Kp_,
    T Ki_,
    T Kd_,
    T I_max_,
    T u_max_,
    bool d_filter_ = false,
    T fc_ = 10.0)
```

Sets the [PID](#) controller parameters.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.
in	<i>Ki_</i>	Integral gain.
in	<i>Kd_</i>	Derivative gain.
in	<i>I_↵ max_</i>	Maximum allowed integral contribution.
in	<i>u_↵ max_</i>	Maximum allowed controller output.
in	<i>d_↵ filter_</i>	Enables or disables derivative filtering.
in	<i>fc_</i>	Cutoff frequency of the derivative filter in Hz.

Updates the [PID](#) gains, sampling time, output limits, and derivative filter configuration.

Definition at line 32 of file [pid_p.h](#).

9.11.3.27 set_u_max()

```
template<typename T>
void control_system::pid::PID_P< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_{\leftarrow max_}$	Maximum allowed controller output.
----	------------------------	------------------------------------

Definition at line 127 of file [pid_p.h](#).

9.11.3.28 update()

```
template<typename T>
T control_system::pid::PID_P< T >::update (
    T x_0,
    T x,
    T u_ff_ = 0.0)
```

Computes the [PID](#) control output.

Parameters

in	x_0	Reference or desired value.
in	x	Current or measured value.
in	$u_{\leftarrow ff_}$	Feed-forward control input.

Returns

[PID](#) controller output.

Calculates the proportional, integral, derivative, and feed-forward contributions and combines them to produce the controller output.

The integral contribution and final controller output are subject to their configured limits.

Definition at line 46 of file [pid_p.h](#).

9.11.4 Member Data Documentation**9.11.4.1 D**

```
template<typename T>
T control_system::pid::PID_P< T >::D = 0.0 [private]
```

Derivative contribution.

Definition at line 407 of file [pid_p.h](#).

9.11.4.2 d_filter

```
template<typename T>
bool control_system::pid::PID_P< T >::d_filter = false [private]
```

Derivative filter enable state.

Definition at line 379 of file [pid_p.h](#).

9.11.4.3 dt

```
template<typename T>
T control_system::pid::PID_P< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time [PID](#) controller in seconds.

Definition at line 349 of file [pid_p.h](#).

9.11.4.4 e_k_1

```
template<typename T>
T control_system::pid::PID_P< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Definition at line 392 of file [pid_p.h](#).

9.11.4.5 fc

```
template<typename T>
T control_system::pid::PID_P< T >::fc = 0.0 [private]
```

Cutoff frequency of the derivative filter.

Cutoff frequency in Hz.

Definition at line 387 of file [pid_p.h](#).

9.11.4.6 I

```
template<typename T>
T control_system::pid::PID_P< T >::I = 0.0 [private]
```

Integral contribution.

Definition at line 402 of file [pid_p.h](#).

9.11.4.7 I_max

```
template<typename T>
T control_system::pid::PID_P< T >::I_max = 0.0 [private]
```

Maximum allowed integral contribution.

Definition at line 369 of file [pid_p.h](#).

9.11.4.8 Kd

```
template<typename T>
T control_system::pid::PID_P< T >::Kd = 0.0 [private]
```

Derivative gain.

Definition at line 364 of file [pid_p.h](#).

9.11.4.9 Ki

```
template<typename T>
T control_system::pid::PID_P< T >::Ki = 0.0 [private]
```

Integral gain.

Definition at line 359 of file [pid_p.h](#).

9.11.4.10 Kp

```
template<typename T>
T control_system::pid::PID_P< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 354 of file [pid_p.h](#).

9.11.4.11 lpf

```
template<typename T>
LPF_1<T> control_system::pid::PID_P< T >::lpf [private]
```

First-order low-pass filter used for derivative filtering.

Definition at line 341 of file [pid_p.h](#).

9.11.4.12 P

```
template<typename T>
T control_system::pid::PID_P< T >::P = 0.0 [private]
```

Proportional contribution.

Definition at line 397 of file [pid_p.h](#).

9.11.4.13 start

```
template<typename T>
bool control_system::pid::PID_P< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line 425 of file [pid_p.h](#).

9.11.4.14 u

```
template<typename T>
T control_system::pid::PID_P< T >::u = 0.0 [private]
```

PID controller output.

Definition at line 412 of file [pid_p.h](#).

9.11.4.15 u_ff

```
template<typename T>
T control_system::pid::PID_P< T >::u_ff = 0.0 [private]
```

Feed-forward control input.

Definition at line 417 of file [pid_p.h](#).

9.11.4.16 u_max

```
template<typename T>
T control_system::pid::PID_P< T >::u_max = 0.0 [private]
```

Maximum allowed controller output.

Definition at line 374 of file [pid_p.h](#).

The documentation for this class was generated from the following files:

- [include/pid/pid_p.h](#)
- [src/pid/pid_p.cpp](#)

9.12 control_system::pid::PID_S< T > Class Template Reference

PID controller with configurable derivative filtering.

```
#include <pid_s.h>
```

Public Member Functions

- [PID_S](#) ()
Constructs a [PID](#) controller.
- [T get_D](#) ()
Gets the derivative contribution.
- [bool get_d_filter](#) ()
Gets the derivative filter enable state.
- [T get_dt](#) ()
Gets the controller sampling time.
- [T get_e_k_1](#) ()
Gets the previous control error.
- [T get_fc](#) ()
Gets the derivative filter cutoff frequency.
- [T get_ff](#) ()
Gets the feed-forward control input.
- [T get_I](#) ()
Gets the integral contribution.
- [T get_I_max](#) ()
Gets the maximum integral contribution.
- [T get_Kp](#) ()
Gets the proportional gain.
- [T get_P](#) ()
Gets the proportional contribution.
- [T get_T_d](#) ()
Gets the derivative time constant.
- [T get_T_i](#) ()
Gets the integral time constant.
- [T get_u](#) ()
Gets the controller output.
- [T get_u_max](#) ()
Gets the maximum controller output.
- [void init](#) (T dt_, T Kp_, T T_i_, T T_d_, T I_max_, T u_max_, bool d_filter_=false, T fc_=10.0)
Initializes the [PID](#) controller.
- [void merge](#) (T u_k_1_)
Merges an external controller output into the [PID](#) state.
- [void reset](#) ()
Resets the [PID](#) controller state.
- [void set_d_filter](#) (bool d_filter_)
Enables or disables derivative filtering.
- [void set_dt](#) (T dt_)
Sets the controller sampling time.
- [void set_fc](#) (T fc_)
Sets the derivative filter cutoff frequency.
- [void set_ff](#) (T u_ff_)
Sets the feed-forward control input.
- [void set_I_max](#) (T I_max_)
Sets the maximum integral contribution.
- [void set_Kp](#) (T Kp_)
Sets the proportional gain.
- [void set_param](#) (T dt_, T Kp_, T T_i_, T T_d_, T I_max_, T u_max_, bool d_filter_=false, T fc_=10.0)

- Sets the *PID* controller parameters.
- void `set_Td` (T T_d_)
 - Sets the derivative time constant.
- void `set_Ti` (T T_i_)
 - Sets the integral time constant.
- void `set_u_max` (T u_max_)
 - Sets the maximum controller output.
- T `update` (T x_0, T x, T u_ff_=0.0)
 - Computes the *PID* control output.

Private Attributes

- T `D` = 0.0
 - Derivative contribution.
- bool `d_filter` = false
 - Derivative filter enable state.
- T `dt` = 0.0
 - Controller sampling time.
- T `e_k_1` = 0.0
 - Previous control error.
- T `fc` = 0.0
 - Cutoff frequency of the derivative filter.
- T `I` = 0.0
 - Integral contribution.
- T `I_max` = 0.0
 - Maximum allowed integral contribution.
- T `Kp` = 0.0
 - Proportional gain.
- LPF_1< T > `lpf`
 - First-order low-pass filter used for derivative filtering.
- T `P` = 0.0
 - Proportional contribution.
- bool `start` = true
 - Controller startup state.
- T `T_d` = 0.0
 - Derivative time constant.
- T `T_i` = 0.0
 - Integral time constant.
- T `u` = 0.0
 - *PID* controller output.
- T `u_ff` = 0.0
 - Feed-forward control input.
- T `u_max` = 0.0
 - Maximum allowed controller output.

9.12.1 Detailed Description

template<typename T>
class control_system::pid::PID_S< T >

PID controller with configurable derivative filtering.

The PID_S class implements a PID controller with configurable proportional, integral, and derivative time constants.

The derivative term can optionally be filtered using a first-order low-pass filter. A feed-forward term can also be included in the controller output.

The controller provides configurable integral and output limits and maintains its control state between successive updates.

Template Parameters

<i>T</i>	Numeric data type used for the controller.
----------	--

Example

```
control_system::pid::PID_S<double> controller;

controller.init(
    0.001,
    Kp,
    Ti,
    Td,
    I_max,
    u_max
);

double output = controller.update(reference, measurement);
```

Definition at line 78 of file [pid_s.h](#).

9.12.2 Constructor & Destructor Documentation

9.12.2.1 PID_S()

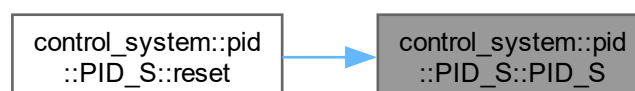
template<typename T>
control_system::pid::PID_S< T >::PID_S ()

Constructs a PID controller.

Constructs the controller with its parameters and internal state initialized to their default values.

Definition at line 5 of file [pid_s.h](#).

Here is the caller graph for this function:



9.12.3 Member Function Documentation

9.12.3.1 get_D()

```
template<typename T>
T control_system::pid::PID_S< T >::get_D ()
```

Gets the derivative contribution.

Returns

Current derivative contribution.

Definition at line 218 of file [pid_s.h](#).

9.12.3.2 get_d_filter()

```
template<typename T>
bool control_system::pid::PID_S< T >::get_d_filter ()
```

Gets the derivative filter enable state.

Returns

true if derivative filtering is enabled, otherwise false.

Definition at line 188 of file [pid_s.h](#).

9.12.3.3 get_dt()

```
template<typename T>
T control_system::pid::PID_S< T >::get_dt ()
```

Gets the controller sampling time.

Returns

Sampling time in seconds.

Definition at line 152 of file [pid_s.h](#).

9.12.3.4 get_e_k_1()

```
template<typename T>
T control_system::pid::PID_S< T >::get_e_k_1 ()
```

Gets the previous control error.

Returns

Control error from the previous controller iteration.

Definition at line 230 of file [pid_s.h](#).

9.12.3.5 `get_fc()`

```
template<typename T>
T control_system::pid::PID_S< T >::get_fc ()
```

Gets the derivative filter cutoff frequency.

Returns

Cutoff frequency in Hz.

Definition at line 194 of file [pid_s.h](#).

9.12.3.6 `get_ff()`

```
template<typename T>
T control_system::pid::PID_S< T >::get_ff ()
```

Gets the feed-forward control input.

Returns

Feed-forward control input.

Definition at line 200 of file [pid_s.h](#).

9.12.3.7 `get_I()`

```
template<typename T>
T control_system::pid::PID_S< T >::get_I ()
```

Gets the integral contribution.

Returns

Current integral contribution.

Definition at line 212 of file [pid_s.h](#).

9.12.3.8 `get_I_max()`

```
template<typename T>
T control_system::pid::PID_S< T >::get_I_max ()
```

Gets the maximum integral contribution.

Returns

Maximum allowed integral contribution.

Definition at line 176 of file [pid_s.h](#).

9.12.3.9 get_Kp()

```
template<typename T>
T control_system::pid::PID_S< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 158 of file [pid_s.h](#).

9.12.3.10 get_P()

```
template<typename T>
T control_system::pid::PID_S< T >::get_P ()
```

Gets the proportional contribution.

Returns

Current proportional contribution.

Definition at line 206 of file [pid_s.h](#).

9.12.3.11 get_T_d()

```
template<typename T>
T control_system::pid::PID_S< T >::get_T_d ()
```

Gets the derivative time constant.

Returns

Derivative time constant.

Definition at line 170 of file [pid_s.h](#).

9.12.3.12 get_T_i()

```
template<typename T>
T control_system::pid::PID_S< T >::get_T_i ()
```

Gets the integral time constant.

Returns

Integral time constant.

Definition at line 164 of file [pid_s.h](#).

9.12.3.13 get_u()

```
template<typename T>
T control_system::pid::PID_S< T >::get_u ()
```

Gets the controller output.

Returns

Current [PID](#) controller output.

Definition at line [224](#) of file [pid_s.h](#).

9.12.3.14 get_u_max()

```
template<typename T>
T control_system::pid::PID_S< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line [182](#) of file [pid_s.h](#).

9.12.3.15 init()

```
template<typename T>
void control_system::pid::PID_S< T >::init (
    T dt_,
    T Kp_,
    T T_i_,
    T T_d_,
    T I_max_,
    T u_max_,
    bool d_filter_ = false,
    T fc_ = 10.0)
```

Initializes the [PID](#) controller.

Parameters

in	<i>dt_</i>	Controller sampling time in seconds.
in	<i>Kp_</i>	Proportional gain.

in	T_i	Integral time constant.
in	T_d	Derivative time constant.
in	$I_{\leftarrow max}$	Maximum allowed integral contribution.
in	$u_{\leftarrow max}$	Maximum allowed controller output.
in	$d_{\leftarrow filter}$	Enables or disables derivative filtering.
in	fc	Cutoff frequency of the derivative filter in Hz.

Initializes the [PID](#) controller parameters and optionally configures the derivative low-pass filter.

Definition at line 25 of file [pid_s.h](#).

9.12.3.16 merge()

```
template<typename T>
void control_system::pid::PID_S< T >::merge (
    T u_k_l_)
```

Merges an external controller output into the [PID](#) state.

Parameters

in	$u_{k_{\leftarrow 1}}$	Previous or externally provided controller output.
----	------------------------	--

Updates the controller state using an externally provided previous controller output.

Definition at line 91 of file [pid_s.h](#).

9.12.3.17 reset()

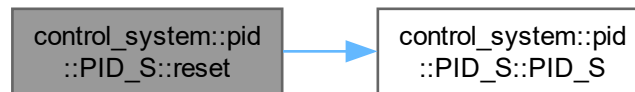
```
template<typename T>
void control_system::pid::PID_S< T >::reset ()
```

Resets the [PID](#) controller state.

Resets the stored control error, proportional, integral, derivative, and controller output states.

Definition at line 78 of file [pid_s.h](#).

Here is the call graph for this function:



9.12.3.18 set_d_filter()

```

template<typename T>
void control_system::pid::PID_S< T >::set_d_filter (
    bool d_filter_)
  
```

Enables or disables derivative filtering.

Parameters

in	$d_filter_$	Derivative filter enable state.
----	---------------	---------------------------------

Definition at line 133 of file [pid_s.h](#).

9.12.3.19 set_dt()

```

template<typename T>
void control_system::pid::PID_S< T >::set_dt (
    T dt_)
  
```

Sets the controller sampling time.

Parameters

in	$dt_$	Sampling time in seconds.
----	--------	---------------------------

Definition at line 96 of file [pid_s.h](#).

9.12.3.20 set_fc()

```
template<typename T>
void control_system::pid::PID_S< T >::set_fc (
    T fc_)
```

Sets the derivative filter cutoff frequency.

Parameters

in	$fc_{\leftarrow}$ —	Cutoff frequency in Hz.
----	------------------------	-------------------------

Definition at line 139 of file [pid_s.h](#).

9.12.3.21 set_ff()

```
template<typename T>
void control_system::pid::PID_S< T >::set_ff (
    T u_ff_)
```

Sets the feed-forward control input.

Parameters

in	$u_{\leftarrow}$ $ff_{\leftarrow}$	Feed-forward control input.
----	---------------------------------------	-----------------------------

Definition at line 146 of file [pid_s.h](#).

9.12.3.22 set_I_max()

```
template<typename T>
void control_system::pid::PID_S< T >::set_I_max (
    T I_max_)
```

Sets the maximum integral contribution.

Parameters

in	$I_{\leftarrow}$ $max_{\leftarrow}$ —	Maximum allowed integral contribution.
----	---	--

Definition at line 121 of file [pid_s.h](#).

9.12.3.23 set_Kp()

```
template<typename T>
void control_system::pid::PID_S< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 103 of file [pid_s.h](#).

9.12.3.24 set_param()

```
template<typename T>
void control_system::pid::PID_S< T >::set_param (
    T dt_,
    T Kp_,
    T T_i_,
    T T_d_,
    T I_max_,
    T u_max_,
    bool d_filter_ = false,
    T fc_ = 10.0)
```

Sets the [PID](#) controller parameters.

Parameters

in	$dt_{\leftarrow}$	Controller sampling time in seconds.
in	$Kp_{\leftarrow}$	Proportional gain.
in	$T_{i_{\leftarrow}}$	Integral time constant.
in	$T_{d_{\leftarrow}}$	Derivative time constant.
in	$I_{\leftarrow max_{\leftarrow}}$	Maximum allowed integral contribution.
in	$u_{\leftarrow max_{\leftarrow}}$	Maximum allowed controller output.
in	$d_{\leftarrow filter_{\leftarrow}}$	Enables or disables derivative filtering.
in	$fc_{\leftarrow}$	Cutoff frequency of the derivative filter in Hz.

Updates the [PID](#) controller parameters and derivative filter configuration.

Definition at line 32 of file [pid_s.h](#).

9.12.3.25 set_Td()

```
template<typename T>
void control_system::pid::PID_S< T >::set_Td (
    T T_d_)
```

Sets the derivative time constant.

Parameters

in	T_d	Derivative time constant.
----	-------	---------------------------

Definition at line 115 of file pid_s.h.

9.12.3.26 set_Ti()

```
template<typename T>
void control_system::pid::PID_S< T >::set_Ti (
    T T_i_)
```

Sets the integral time constant.

Parameters

in	T_i	Integral time constant.
----	-------	-------------------------

Definition at line 109 of file pid_s.h.

9.12.3.27 set_u_max()

```
template<typename T>
void control_system::pid::PID_S< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	u_{max}	Maximum allowed controller output.
----	-----------	------------------------------------

Definition at line 127 of file pid_s.h.

9.12.3.28 update()

```
template<typename T>
T control_system::pid::PID_S< T >::update (
    T x_0,
    T x,
    T u_ff_ = 0.0)
```

Computes the [PID](#) control output.

Parameters

in	x_0	Reference or desired value.
in	x	Current or measured value.
in	$u_{\leftrightarrow ff}$	Feed-forward control input.

Returns

[PID](#) controller output.

Calculates the proportional, integral, derivative, and feed-forward contributions and combines them to produce the controller output.

The integral contribution and controller output are subject to their configured limits.

Definition at line [46](#) of file [pid_s.h](#).

9.12.4 Member Data Documentation

9.12.4.1 D

```
template<typename T>
T control_system::pid::PID_S< T >::D = 0.0 [private]
```

Derivative contribution.

Definition at line [398](#) of file [pid_s.h](#).

9.12.4.2 d_filter

```
template<typename T>
bool control_system::pid::PID_S< T >::d_filter = false [private]
```

Derivative filter enable state.

Definition at line [370](#) of file [pid_s.h](#).

9.12.4.3 dt

```
template<typename T>
T control_system::pid::PID_S< T >::dt = 0.0 [private]
```

Controller sampling time.

Sampling period of the discrete-time PID controller in seconds.

Definition at line 340 of file [pid_s.h](#).

9.12.4.4 e_k_1

```
template<typename T>
T control_system::pid::PID_S< T >::e_k_1 = 0.0 [private]
```

Previous control error.

Definition at line 383 of file [pid_s.h](#).

9.12.4.5 fc

```
template<typename T>
T control_system::pid::PID_S< T >::fc = 0.0 [private]
```

Cutoff frequency of the derivative filter.

Cutoff frequency in Hz.

Definition at line 378 of file [pid_s.h](#).

9.12.4.6 I

```
template<typename T>
T control_system::pid::PID_S< T >::I = 0.0 [private]
```

Integral contribution.

Definition at line 393 of file [pid_s.h](#).

9.12.4.7 I_max

```
template<typename T>
T control_system::pid::PID_S< T >::I_max = 0.0 [private]
```

Maximum allowed integral contribution.

Definition at line 360 of file [pid_s.h](#).

9.12.4.8 Kp

```
template<typename T>
T control_system::pid::PID_S< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line 345 of file [pid_s.h](#).

9.12.4.9 lpf

```
template<typename T>
LPF_1<T> control_system::pid::PID_S< T >::lpf [private]
```

First-order low-pass filter used for derivative filtering.

Definition at line 332 of file [pid_s.h](#).

9.12.4.10 P

```
template<typename T>
T control_system::pid::PID_S< T >::P = 0.0 [private]
```

Proportional contribution.

Definition at line 388 of file [pid_s.h](#).

9.12.4.11 start

```
template<typename T>
bool control_system::pid::PID_S< T >::start = true [private]
```

Controller startup state.

Indicates whether the controller is in its startup state.

Definition at line 416 of file [pid_s.h](#).

9.12.4.12 T_d

```
template<typename T>
T control_system::pid::PID_S< T >::T_d = 0.0 [private]
```

Derivative time constant.

Definition at line 355 of file [pid_s.h](#).

9.12.4.13 T_i

```
template<typename T>
T control_system::pid::PID_S< T >::T_i = 0.0 [private]
```

Integral time constant.

Definition at line 350 of file [pid_s.h](#).

9.12.4.14 u

```
template<typename T>
T control_system::pid::PID_S< T >::u = 0.0 [private]
```

PID controller output.

Definition at line 403 of file [pid_s.h](#).

9.12.4.15 u_ff

```
template<typename T>
T control_system::pid::PID_S< T >::u_ff = 0.0 [private]
```

Feed-forward control input.

Definition at line 408 of file [pid_s.h](#).

9.12.4.16 u_max

```
template<typename T>
T control_system::pid::PID_S< T >::u_max = 0.0 [private]
```

Maximum allowed controller output.

Definition at line 365 of file [pid_s.h](#).

The documentation for this class was generated from the following files:

- [include/pid/pid_s.h](#)
- [src/pid/pid_s.hpp](#)

9.13 control_system::proportional::P< T > Class Template Reference

Proportional controller.

```
#include <p.h>
```

Public Member Functions

- `P ()`
Constructs a proportional controller.
- `T get_Kp ()`
Gets the proportional gain.
- `T get_u_max ()`
Gets the maximum controller output.
- `void init (T Kp_, T u_max_)`
Initializes the proportional controller.
- `void reset ()`
Resets the controller.
- `void set_Kp (T Kp_)`
Sets the proportional gain.
- `void set_param (T Kp_, T u_max_)`
Sets the controller parameters.
- `void set_u_max (T u_max_)`
Sets the maximum controller output.
- `T update (T x_0, T x)`
Computes the proportional control output.

Private Attributes

- `T Kp = 0.0`
Proportional gain.
- `T u_max = 9999999999`
Maximum controller output.

9.13.1 Detailed Description

template<typename T>
class control_system::proportional::P< T >

Proportional controller.

The `P` class implements a proportional controller in which the control output is proportional to the control error.

The controller uses a configurable proportional gain and maximum output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller.
----------------	--

Example

```
control_system::proportional::P<double> controller;

controller.init(2.0, 100.0);

double output = controller.update(reference, measurement);
```

Definition at line 70 of file `p.h`.

9.13.2 Constructor & Destructor Documentation

9.13.2.1 P()

```
template<typename T>  
control_system::proportional::P< T >::P ()
```

Constructs a proportional controller.

Constructs the controller with its parameters initialized to their default values.

Definition at line 5 of file [p.h](#).

9.13.3 Member Function Documentation

9.13.3.1 get_Kp()

```
template<typename T>  
T control_system::proportional::P< T >::get_Kp ()
```

Gets the proportional gain.

Returns

Proportional gain.

Definition at line 47 of file [p.h](#).

9.13.3.2 get_u_max()

```
template<typename T>  
T control_system::proportional::P< T >::get_u_max ()
```

Gets the maximum controller output.

Returns

Maximum allowed controller output.

Definition at line 53 of file [p.h](#).

9.13.3.3 init()

```
template<typename T>
void control_system::proportional::P< T >::init (
    T Kp_,
    T u_max_)
```

Initializes the proportional controller.

Parameters

in	Kp_{-}	Proportional gain.
in	$u_{\leftarrow max_{-}}$	Maximum allowed controller output.

Initializes the proportional gain and output limit.

Definition at line 11 of file [p.h](#).

9.13.3.4 reset()

```
template<typename T>
void control_system::proportional::P< T >::reset ()
```

Resets the controller.

Resets the controller state to its initial condition.

Definition at line 30 of file [p.h](#).

9.13.3.5 set_Kp()

```
template<typename T>
void control_system::proportional::P< T >::set_Kp (
    T Kp_)
```

Sets the proportional gain.

Parameters

in	$Kp_{\leftarrow}$	Proportional gain.
	—	

Definition at line 35 of file [p.h](#).

9.13.3.6 set_param()

```
template<typename T>
void control_system::proportional::P< T >::set_param (
    T Kp_,
    T u_max_)
```

Sets the controller parameters.

Parameters

in	$Kp_$	Proportional gain.
in	$u_{\leftarrow max_}$	Maximum allowed controller output.

Updates the proportional gain and maximum output limit.

Definition at line 17 of file [p.h](#).

9.13.3.7 set_u_max()

```
template<typename T>
void control_system::proportional::P< T >::set_u_max (
    T u_max_)
```

Sets the maximum controller output.

Parameters

in	$u_{\leftarrow max_}$	Maximum allowed controller output.
----	------------------------	------------------------------------

Definition at line 41 of file [p.h](#).

9.13.3.8 update()

```
template<typename T>
T control_system::proportional::P< T >::update (
    T x_0,
    T x)
```

Computes the proportional control output.

Parameters

in	$x_{\leftarrow 0}$	Reference or desired value.
in	x	Current or measured value.

Returns

Proportional control output limited to the configured maximum output.

Definition at line 24 of file [p.h](#).

9.13.4 Member Data Documentation

9.13.4.1 Kp

```
template<typename T>
T control\_system::proportional::P< T >::Kp = 0.0 [private]
```

Proportional gain.

Definition at line [155](#) of file [p.h](#).

9.13.4.2 u_max

```
template<typename T>
T control\_system::proportional::P< T >::u_max = 9999999999 [private]
```

Maximum controller output.

Defines the upper magnitude limit applied to the controller output.

Definition at line [164](#) of file [p.h](#).

The documentation for this class was generated from the following files:

- [include/proportional/p.h](#)
- [src/proportional/p.hpp](#)

Chapter 10

File Documentation

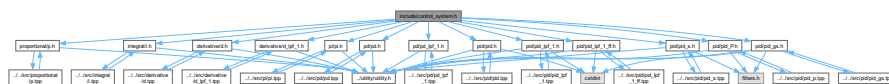
10.1 doc/mainpage.dox File Reference

10.2 include/control_system.h File Reference

Main public header for the Control System library.

```
#include "proportional/p.h"
#include "integral/i.h"
#include "derivative/d.h"
#include "derivative/d_lpf_1.h"
#include "pi/pi.h"
#include "pd/pd.h"
#include "pd/pd_lpf_1.h"
#include "pid/pid.h"
#include "pid/pid_lpf_1.h"
#include "pid/pid_lpf_1_ff.h"
#include "pid/pid_s.h"
#include "pid/pid_P.h"
#include "pid/pid_gs.h"
```

Include dependency graph for control_system.h:



10.2.1 Detailed Description

Main public header for the Control System library.

This header provides access to the control-system algorithms available in the Control System library.

The library provides implementations for:

- Proportional (P) control.

- Integral (I) control.
- Derivative (D) control.
- Derivative control with first-order low-pass filtering.
- Proportional-Integral (PI) control.
- Proportional-Derivative (PD) control.
- Proportional-Derivative control with first-order low-pass filtering.
- Proportional-Integral-Derivative (PID) control.
- PID control with first-order low-pass filtering.
- PID control with first-order low-pass filtering and feed-forward.
- PID control using setpoint weighting.
- PID control using gain scheduling.

Including this header provides access to the public control-system classes through a single include file.

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [control_system.h](#).

10.3 control_system.h

[Go to the documentation of this file.](#)

```
00001
00034
00035 #ifndef CONTROL_SYSTEM_H
00036 #define CONTROL_SYSTEM_H
00037
00041 #include "proportional/p.h"
00042
00046 #include "integral/i.h"
00047
00051 #include "derivative/d.h"
00052
00056 #include "derivative/d_lpf_1.h"
00057
00061 #include "pi/pi.h"
00062
00066 #include "pd/pd.h"
```

```

00067
00071 #include "pd/pd_lpf_1.h"
00072
00076 #include "pid/pid.h"
00077
00081 #include "pid/pid_lpf_1.h"
00082
00086 #include "pid/pid_lpf_1_ff.h"
00087
00091 #include "pid/pid_s.h"
00092
00096 #include "pid/pid_P.h"
00097
00101 #include "pid/pid_gs.h"
00102
00103 #endif

```

10.4 include/derivative/d.h File Reference

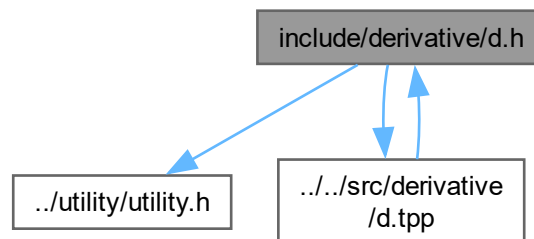
Derivative (D) controller.

```

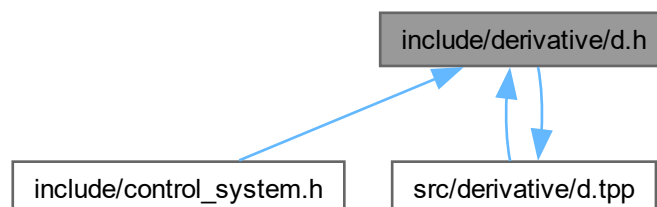
#include "../utility/utility.h"
#include "../../src/derivative/d.hpp"

```

Include dependency graph for d.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [control_system::derivative::D< T >](#)
Discrete-time derivative controller.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::derivative](#)
- namespace [derivative](#)
Contains derivative controller implementations.

10.4.1 Detailed Description

Derivative (D) controller.

This file defines the [control_system::derivative::D](#) class, which implements a discrete-time derivative controller.

The controller generates a control output based on the rate of change of the control error between a reference value and the measured or current value.

The controller maintains the previous control error as internal state and applies a configurable maximum output limit.

Template Parameters

<i>T</i>	Numeric data type used for the controller calculations.
----------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [d.h](#).

10.5 d.h

[Go to the documentation of this file.](#)

```

00001
00025
00026 #ifndef CONTROL_SYSTEM_DERIVATIVE_D_H
00027 #define CONTROL_SYSTEM_DERIVATIVE_D_H
00028
00029 #include "../utility/utility.h"
00030
00031 namespace control_system
00032 {
00037
00038     namespace derivative
00039     {
00044
00075         template <typename T>
00076         class D
00077         {
00078         public:
00086             D();
00087
00099             void init(T dt_, T Kd_, T u_max_);
00100
00112             void set_param(T dt_, T Kd_, T u_max_);
00113
00128             T update(T x_0, T x);
00129
00136             void reset();
00137
00148             void merge(T u_k_1_);
00149
00155             void set_dt(T dt_);
00156
00162             void set_Kd(T Kd_);
00163
00169             void set_u_max(T u_max_);
00170
00176             T get_dt();
00177
00183             T get_Kd();
00184
00190             T get_e_k_1();
00191
00197             T get_u_max();
00198
00199         private:
00207             T dt = 0.0;
00208
00215             T e_k_1 = 0.0;
00216
00220             T Kd = 0.0;
00221
00228             bool start = true;
00229
00237             T u_max = 9999999999;
00238         };
00239
00240 #include "../src/derivative/d.tpp"
00241     }
00242 }
00243
00244 #endif

```

10.6 include/derivative/d_lpf_1.h File Reference

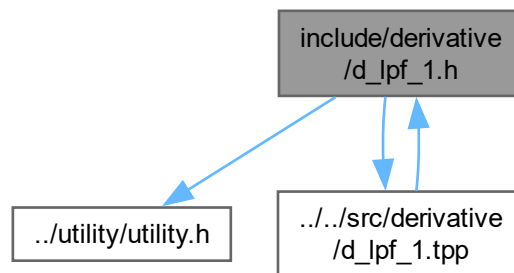
Derivative controller with first-order low-pass filtering.

```

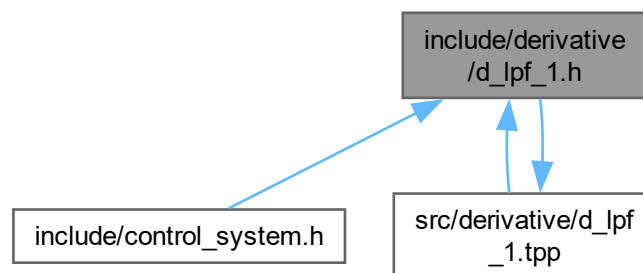
#include "../utility/utility.h"
#include "../src/derivative/d_lpf_1.tpp"

```

Include dependency graph for `d_lpf_1.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class [control_system::derivative::D_LPF_1< T >](#)
Derivative controller with a first-order low-pass filter.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::derivative](#)
- namespace [derivative](#)
Contains derivative controller implementations.

10.6.1 Detailed Description

Derivative controller with first-order low-pass filtering.

This file defines the `control_system::derivative::D_LPF_1` class, which implements a discrete-time derivative controller with a first-order low-pass filter.

The controller uses a configurable derivative gain, sampling time, and filter cutoff frequency. The previous control error and controller output are maintained as internal state variables.

The controller output is constrained by a configurable maximum output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller calculations.
----------------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [d_lpf_1.h](#).

10.7 d_lpf_1.h

[Go to the documentation of this file.](#)

```

00001
00026
00027 #ifndef CONTROL_SYSTEM_DERIVATIVE_D_LPF_1_H
00028 #define CONTROL_SYSTEM_DERIVATIVE_D_LPF_1_H
00029
00030 #include "../utility/utility.h"
00031
00032 namespace control_system
00033 {
00038
00039     namespace derivative
00040     {
00045
00078         template <typename T>
00079         class D_LPF_1
00080         {
00081         public:
00089             D_LPF_1();
00090
00103             void init(T dt_, T Kd_, T fc_, T u_max_);

```

```

00104
00117     void set_param(T dt_, T Kd_, T fc_, T u_max_);
00118
00133     T update(T x_0, T x);
00134
00142     void reset();
00143
00154     void merge(T u_k_1_);
00155
00161     void set_dt(T dt_);
00162
00168     void set_Kd(T Kd_);
00169
00175     void set_fc(T fc_);
00176
00182     void set_u_max(T u_max_);
00183
00189     T get_dt();
00190
00196     T get_Kd();
00197
00203     T get_fc();
00204
00210     T get_e_k_1();
00211
00217     T get_u_k_1();
00218
00224     T get_u_max();
00225
00226 private:
00234     T dt = 0.0;
00235
00242     T e_k_1 = 0.0;
00243
00247     T Kd = 0.0;
00248
00255     bool start = true;
00256
00263     T u_k_1 = 0.0;
00264
00268     T tau = 0.0;
00269
00277     T u_max = 9999999999;
00278 };
00279
00280 #include "../src/derivative/d_lpf_1.tpp"
00281 }
00282 }
00283
00284 #endif

```

10.8 include/integral/i.h File Reference

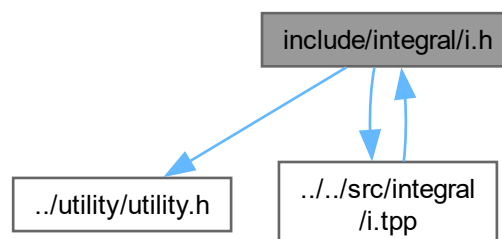
Integral (I) controller.

```

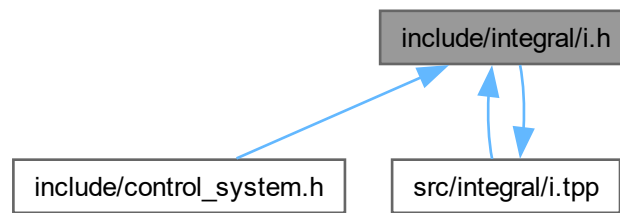
#include "../utility/utility.h"
#include "../src/integral/i.tpp"

```

Include dependency graph for i.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `control_system::integral::I< T >`
Discrete-time integral controller.

Namespaces

- namespace `control_system`
Contains control-system algorithms and controllers.
- namespace `control_system::integral`
- namespace `integral`
Contains integral controller implementations.

10.8.1 Detailed Description

Integral (I) controller.

This file defines the `control_system::integral::I` class, which implements a discrete-time integral controller.

The controller generates a control output by integrating the error between a reference value and the measured or current value over time.

The controller maintains the previous output as internal state and applies a configurable maximum output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller calculations.
----------------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [i.h](#).

10.9 i.h

[Go to the documentation of this file.](#)

```

00001
00025
00026 #ifndef CONTROL_SYSTEM_INTEGRAL_I_H
00027 #define CONTROL_SYSTEM_INTEGRAL_I_H
00028
00029 #include "../utility/utility.h"
00030
00031 namespace control_system
00032 {
00037
00038     namespace integral
00039     {
00044
00072         template <typename T>
00073         class I
00074         {
00075         public:
00083             I();
00084
00097             void init(T dt_, T Ki_, T u_k_1_, T u_max_);
00098
00111             void set_param(T dt_, T Ki_, T u_k_1_, T u_max_);
00112
00127             T update(T x_0, T x);
00128
00135             void reset();
00136
00147             void merge(T u_k_1_);
00148
00154             void set_dt(T dt_);
00155
00161             void set_Ki(T Ki_);
00162
00168             void set_u_0(T u_k_1_);
00169
00175             void set_u_max(T u_max_);
00176
00182             T get_dt();
00183
00189             T get_Ki();
00190
00196             T get_u_k_1();
00197
00203             T get_u_max();
00204
00205         private:
00213             T dt = 0.0;
00214
00221             T u_k_1 = 0.0;
00222
00226             T Ki = 0.0;
00227
00235             T u_max = 9999999999;
00236         };
00237
00238 #include "../src/integral/i.tpp"
00239     }
00240 }
00241
00242 #endif

```

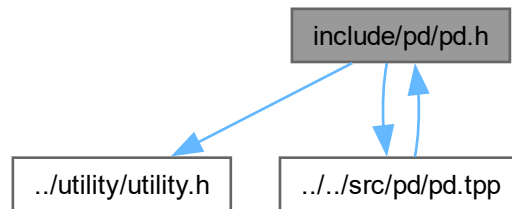
10.10 include/pd/pd.h File Reference

Proportional-Derivative (PD) controller.

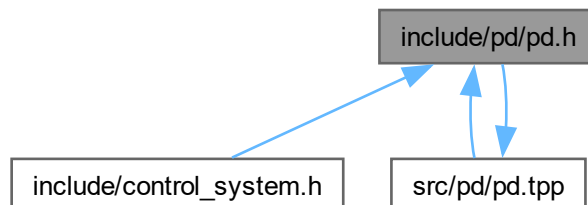
```
#include "../utility/utility.h"
```

```
#include "../../src/pd/pd.hpp"
```

Include dependency graph for pd.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `control_system::pd::PD< T >`
Discrete-time proportional-derivative controller.

Namespaces

- namespace `control_system`
Contains control-system algorithms and controllers.
- namespace `control_system::pd`
- namespace `pd`
Contains proportional-derivative controller implementations.

10.10.1 Detailed Description

Proportional-Derivative (PD) controller.

This file defines the `control_system::pd::PD` class, which implements a discrete-time proportional-derivative (PD) controller.

The controller combines proportional and derivative actions to generate a control output based on the error between a reference value and the measured or current value.

The controller maintains the previous control error as internal state and applies a configurable maximum output limit.

Template Parameters

<code>T</code>	Numeric data type used for the controller calculations.
----------------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [pd.h](#).

10.11 pd.h

[Go to the documentation of this file.](#)

```

00001
00025
00026 #ifndef CONTROL_SYSTEM_PD_PD_H
00027 #define CONTROL_SYSTEM_PD_PD_H
00028
00029 #include "../utility/utility.h"
00030
00031 namespace control_system
00032 {
00037
00038     namespace pd
00039     {
00044
00073         template <typename T>
00074         class PD
00075         {
00076         public:
00084             PD();
00085

```



```

00098         void init(T dt_, T Kp_, T Kd_, T u_max_);
00099
00112         void set_param(T dt_, T Kp_, T Kd_, T u_max_);
00113
00128         T update(T x_0, T x);
00129
00136         void reset();
00137
00148         void merge(T u_k_1_);
00149
00155         void set_dt(T dt_);
00156
00162         void set_Kp(T Kp_);
00163
00169         void set_Kd(T Kd_);
00170
00176         void set_u_max(T u_max_);
00177
00183         T get_dt();
00184
00190         T get_Kp();
00191
00197         T get_Kd();
00198
00204         T get_e_k_1();
00205
00211         T get_u_max();
00212
00213     private:
00221         T dt = 0.0;
00222
00229         T e_k_1 = 0.0;
00230
00234         T Kp = 0.0;
00235
00239         T Kd = 0.0;
00240
00247         bool start = true;
00248
00256         T u_max = 9999999999;
00257     };
00258
00259 #include "../src/pd/pd_tpp"
00260     }
00261 }
00262
00263 #endif

```

10.12 include/pd/pd_lpf_1.h File Reference

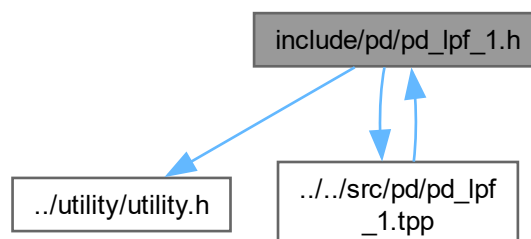
Proportional-Derivative (PD) controller with first-order low-pass filtering.

```

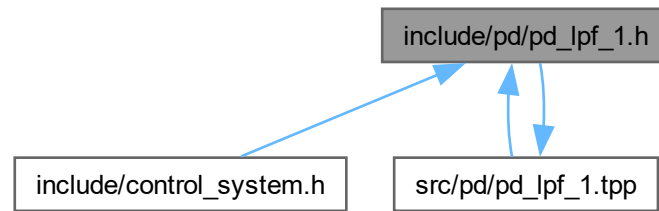
#include "../utility/utility.h"
#include "../src/pd/pd_lpf_1.tpp"

```

Include dependency graph for pd_lpf_1.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `control_system::pd::PD_LPF_1< T >`
PD controller with first-order low-pass filtered derivative.

Namespaces

- namespace `control_system`
Contains control-system algorithms and controllers.
- namespace `control_system::pd`
- namespace `pd`
Contains proportional-derivative controller implementations.

10.12.1 Detailed Description

Proportional-Derivative (PD) controller with first-order low-pass filtering.

This file defines the `control_system::pd::PD_LPF_1` class, which implements a discrete-time proportional-derivative (PD) controller with a first-order low-pass filter associated with the derivative action.

The controller uses configurable proportional and derivative gains, sampling time, derivative filter cutoff frequency, and maximum output limit.

The previous control error and controller output are maintained as internal state variables.

Template Parameters

<code>T</code>	Numeric data type used for the controller calculations.
----------------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [pd_lpf_1.h](#).

10.13 pd_lpf_1.h

[Go to the documentation of this file.](#)

```

00001
00027
00028 #ifndef CONTROL_SYSTEM_PD_PD_LPF_1_H
00029 #define CONTROL_SYSTEM_PD_PD_LPF_1_H
00030
00031 #include "../utility/utility.h"
00032
00033 namespace control_system
00034 {
00039     namespace pd
00040     {
00041     {
00046
00081         template <typename T>
00082         class PD_LPF_1
00083         {
00084         public:
00092             PD_LPF_1();
00093
00108             void init(T dt_, T Kp_, T Kd_, T fc_, T u_max_);
00109
00124             void set_param(T dt_, T Kp_, T Kd_, T fc_, T u_max_);
00125
00141             T update(T x_0, T x);
00142
00150             void reset();
00151
00162             void merge(T u_k_1);
00163
00169             void set_dt(T dt_);
00170
00176             void set_Kp(T Kp_);
00177
00183             void set_Kd(T Kd_);
00184
00190             void set_fc(T fc_);
00191
00197             void set_u_max(T u_max_);
00198
00204             T get_dt();
00205
00211             T get_Kp();
00212
00218             T get_Kd();
00219
00225             T get_fc();
00226
00232             T get_e_k_1();
00233
00239             T get_u_k_1();
00240
00246             T get_u_max();
00247
00248         private:
00256             T dt = 0.0;
00257

```

```

00264         T e_k_1 = 0.0;
00265
00269         T Kp = 0.0;
00270
00274         T Kd = 0.0;
00275
00282         bool start = true;
00283
00290         T u_k_1 = 0.0;
00291
00299         T tau = 0.0;
00300
00308         T u_max = 9999999999;
00309     };
00310
00311     #include "../../src/pd/pd_lpf_1.tpp"
00312     }
00313 }
00314
00315 #endif

```

10.14 include/pi/pi.h File Reference

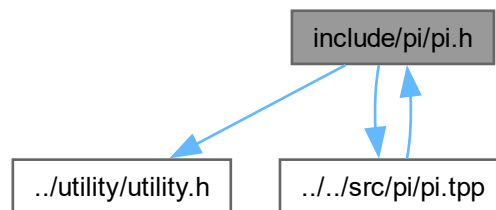
Proportional-Integral (PI) controller.

```

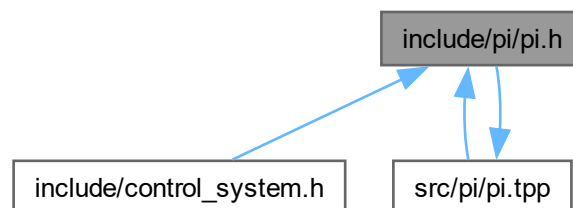
#include "../utility/utility.h"
#include "../../src/pi/pi.tpp"

```

Include dependency graph for pi.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [control_system::pi::PI< T >](#)
Discrete-time proportional-integral controller.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::pi](#)
- namespace [pi](#)
Contains proportional-integral controller implementations.

10.14.1 Detailed Description

Proportional-Integral (PI) controller.

This file defines the [control_system::pi::PI](#) class, which implements a discrete-time proportional-integral (PI) controller.

The controller combines proportional and integral actions to generate a control output based on the error between a reference value and the measured or current value.

The controller maintains the previous control error and previous controller output as internal state variables and applies a configurable maximum output limit.

Template Parameters

<i>T</i>	Numeric data type used for the controller calculations.
----------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [pi.h](#).

10.15 pi.h

[Go to the documentation of this file.](#)

```

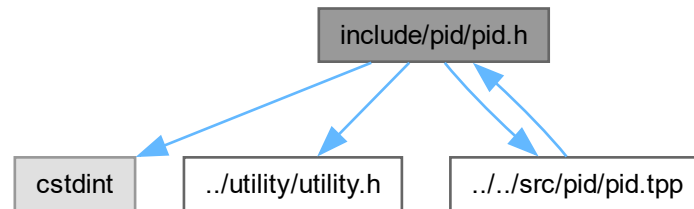
00001
00026
00027 #ifndef CONTROL_SYSTEM_PI_PI_H
00028 #define CONTROL_SYSTEM_PI_PI_H
00029
00030 #include "../utility/utility.h"
00031
00032 namespace control_system
00033 {
00038
00039     namespace pi
00040     {
00045
00075         template <typename T>
00076         class PI
00077         {
00078         public:
00086             PI();
00087
00101             void init(T dt_, T Kp_, T Ki_, T u_k_1_, T u_max_);
00102
00116             void set_param(T dt_, T Kp_, T Ki_, T u_k_1_, T u_max_);
00117
00132             T update(T x_0, T x);
00133
00141             void reset();
00142
00153             void merge(T u_k_1_);
00154
00160             void set_dt(T dt_);
00161
00167             void set_Kp(T Kp_);
00168
00174             void set_Ki(T Ki_);
00175
00181             void set_u_0(T u_k_1_);
00182
00188             void set_u_max(T u_max_);
00189
00195             T get_dt();
00196
00202             T get_Kp();
00203
00209             T get_Ki();
00210
00216             T get_u_k_1();
00217
00223             T get_u_max();
00224
00225         private:
00233             T dt = 0.0;
00234
00241             T u_k_1 = 0.0;
00242
00249             T e_k_1 = 0.0;
00250
00254             T Kp = 0.0;
00255
00259             T Ki = 0.0;
00260
00268             T u_max = 9999999999;
00269
00276             bool start = true;
00277         };
00278
00279 #include "../src/pi/pi.tpp"
00280     }
00281 }
00282
00283 #endif

```

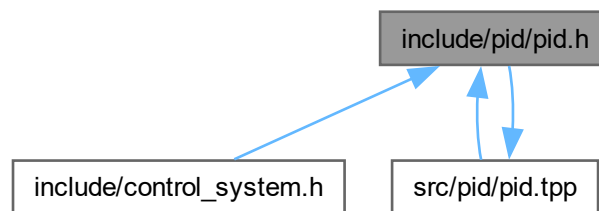
10.16 include/pid/pid.h File Reference

Proportional-Integral-Derivative (PID) controller.

```
#include <stdint>
#include "../utility/utility.h"
#include "../../src/pid/pid.hpp"
Include dependency graph for pid.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [control_system::pid::PID< T >](#)
Discrete-time proportional-integral-derivative controller.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::pid](#)
- namespace [pid](#)
Contains proportional-integral-derivative controller implementations.

10.16.1 Detailed Description

Proportional-Integral-Derivative (PID) controller.

This file defines the `control_system::pid::PID` class, which implements a discrete-time PID controller.

The controller combines proportional, integral, and derivative components to generate a control output based on the error between a reference value and the measured value.

The controller maintains previous error and output values as internal state variables and applies a configurable maximum output limit.

The class is templated to support different numeric types such as `float` and `double`.

Template Parameters

<code>T</code>	Numeric data type used for the controller calculations.
----------------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file `pid.h`.

10.17 pid.h

[Go to the documentation of this file.](#)

```

00001
00028
00029 #ifndef CONTROL_SYSTEM_PID_PID_H
00030 #define CONTROL_SYSTEM_PID_PID_H
00031
00032 #include <cstdint>
00033 #include "../utility/utility.h"
00034
00035 namespace control_system
00036 {
00041
00042     namespace pid
00043     {
00048
00081         template <typename T>
00082         class PID
00083         {

```



```

00084     public:
00092         PID();
00093
00108         void init(T dt_, T Kp_, T Ki_, T Kd_, T u_k_1_, T u_max_);
00109
00124         void set_param(T dt_, T Kp_, T Ki_, T Kd_, T u_k_1_, T u_max_);
00125
00140         T update(T x_0, T x);
00141
00149         void reset();
00150
00161         void merge(T u_k_1_);
00162
00168         void set_dt(T dt_);
00169
00175         void set_Kp(T Kp_);
00176
00182         void set_Ki(T Ki_);
00183
00189         void set_Kd(T Kd_);
00190
00196         void set_u_0(T u_k_1_);
00197
00203         void set_u_max(T u_max_);
00204
00210         T get_dt();
00211
00217         T get_Kp();
00218
00224         T get_Ki();
00225
00231         T get_Kd();
00232
00238         T get_u_k_1();
00239
00245         T get_e_k_1();
00246
00252         T get_e_k_2();
00253
00259         T get_u_max();
00260
00261     private:
00269         T dt = 0.0;
00270
00277         T u_k_1 = 0.0;
00278
00285         T e_k_1 = 0.0;
00286
00290         T e_k_2 = 0.0;
00291
00295         T Kp = 0.0;
00296
00300         T Ki = 0.0;
00301
00305         T Kd = 0.0;
00306
00314         T u_max = 9999999999;
00315
00323         uint8_t start = 0;
00324     };
00325
00326 #include "../src/pid/pid.tpp"
00327     }
00328 }
00329
00330 #endif

```

10.18 include/pid/pid_gs.h File Reference

PID controller with gain scheduling.

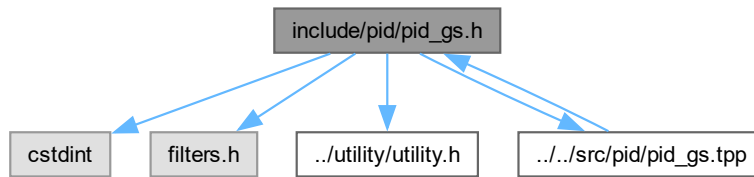
```

#include <cstdint>
#include "filters.h"
#include "../utility/utility.h"

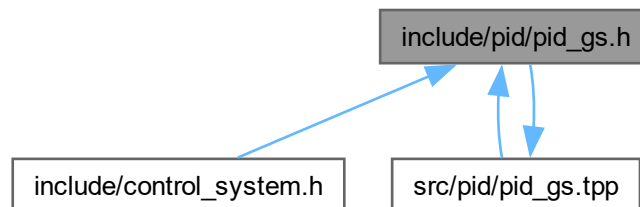
```

```
#include "../../src/pid/pid_gs.hpp"
```

Include dependency graph for pid_gs.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [control_system::pid::PID_GS< T >](#)
PID controller with gain scheduling.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::pid](#)
- namespace [pid](#)
Contains proportional-integral-derivative controller implementations.

10.18.1 Detailed Description

PID controller with gain scheduling.

This file defines the [control_system::pid::PID_GS](#) class, which implements a discrete-time proportional-integral-derivative (PID) controller with gain scheduling.

The controller supports configurable proportional, integral, and derivative gains, integral and output limits, optional derivative filtering, and feed-forward control.

The PID gains can be scheduled according to an operating condition using user-provided lookup tables. The controller interpolates the gain values between lookup-table points.

Template Parameters

<i>T</i>	Numeric data type used for the controller calculations.
----------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [pid_gs.h](#).

10.19 pid_gs.h

[Go to the documentation of this file.](#)

```

00001
00027
00028 #ifndef CONTROL_SYSTEM_PID_PID_GS_H
00029 #define CONTROL_SYSTEM_PID_PID_GS_H
00030
00031 #include <stdint>
00032 #include "filters.h"
00033 #include "../utility/utility.h"
00034
00035 namespace control_system
00036 {
00041     namespace pid
00042     {
00043     {
00048
00101         template <typename T>
00102         class PID_GS
00103         {
00104         public:
00112             PID_GS();
00113
00130             void init(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_ = false, T fc_ =
00131 10.0);
00148             void set_param(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_ = false, T
00149 fc_ = 10.0);
00170             T update(T x_0, T x, T u_ff_ = 0.0, T op_cond_ = 0.0);
00171
00179             void reset();
00180

```

```

00191         void merge(T u_k_l_);
00192
00208         void set_gain_lookup_table(T *op_cond_arr_, T *Kp_arr_, T *Ki_arr_, T *Kd_arr_, uint16_t
len_arr_);
00209
00215         void set_dt(T dt_);
00216
00222         void set_Kp(T Kp_);
00223
00229         void set_Ki(T Ki_);
00230
00236         void set_Kd(T Kd_);
00237
00243         void set_I_max(T I_max_);
00244
00250         void set_u_max(T u_max_);
00251
00257         void set_d_filter(bool d_filter_);
00258
00264         void set_fc(T fc_);
00265
00271         void set_ff(T u_ff_);
00272
00278         T get_dt();
00279
00285         T get_Kp();
00286
00292         T get_Ki();
00293
00299         T get_Kd();
00300
00306         T get_I_max();
00307
00313         T get_u_max();
00314
00321         bool get_d_filter();
00322
00328         T get_fc();
00329
00335         T get_ff();
00336
00342         T get_P();
00343
00349         T get_I();
00350
00356         T get_D();
00357
00363         T get_u();
00364
00370         T get_e_k_l();
00371
00372     private:
00376         LPF_1<T> lpf;
00377
00388         void update_gain(T op_cond_);
00389
00401         T interpolate(T x, T x1, T y1, T x2, T y2);
00402
00410         T dt = 0.0;
00411
00415         T Kp = 0.0;
00416
00420         T Ki = 0.0;
00421
00425         T Kd = 0.0;
00426
00430         T I_max = 0.0;
00431
00435         T u_max = 0.0;
00436
00440         bool d_filter = false;
00441
00448         T fc = 0.0;
00449
00453         T e_k_l = 0.0;
00454
00458         T P = 0.0;
00459
00463         T I = 0.0;
00464
00468         T D = 0.0;
00469
00473         T u = 0.0;
00474
00478         T u_ff = 0.0;
00479
00487         T *op_cond_arr;

```

```

00488
00492     T *Kp_arr;
00493
00497     T *Ki_arr;
00498
00502     T *Kd_arr;
00503
00507     uint16_t len_arr;
00508
00515     bool start = true;
00516 };
00517
00518 #include "../src/pid/pid_gs.tpp"
00519 }
00520 }
00521
00522 #endif

```

10.20 include/pid/pid_lpf_1.h File Reference

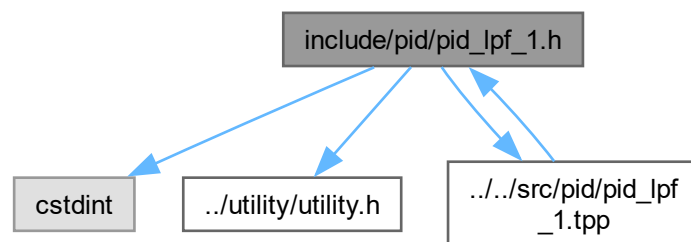
PID controller with first-order low-pass filtered derivative.

```

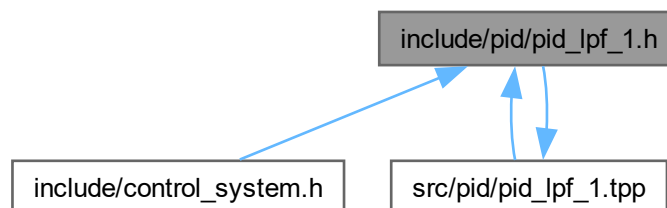
#include <stdint>
#include "../utility/utility.h"
#include "../src/pid/pid_lpf_1.tpp"

```

Include dependency graph for pid_lpf_1.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `control_system::pid::PID_LPF_1< T >`
PID controller with a first-order low-pass filtered derivative.

Namespaces

- namespace `control_system`
Contains control-system algorithms and controllers.
- namespace `control_system::pid`
- namespace `pid`
Contains proportional-integral-derivative controller implementations.

10.20.1 Detailed Description

PID controller with first-order low-pass filtered derivative.

This file defines the `control_system::pid::PID_LPF_1` class, which implements a discrete-time proportional-integral-derivative (PID) controller with a first-order low-pass filter applied to the derivative contribution.

The controller uses configurable proportional, integral, and derivative gains, together with a derivative filter cutoff frequency. Previous control errors and controller outputs are maintained as internal state variables.

The controller output is constrained by a configurable maximum output limit.

Template Parameters

<i>T</i>	Numeric data type used for the controller calculations.
----------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file `pid_lpf_1.h`.

10.21 pid_lpf_1.h

[Go to the documentation of this file.](#)

```

00001
00028
00029 #ifndef CONTROL_SYSTEM_PID_PID_LPF_1_H
00030 #define CONTROL_SYSTEM_PID_PID_LPF_1_H
00031
00032 #include <cstdint>
00033 #include "../utility/utility.h"
00034
00035 namespace control_system
00036 {
00041
00042     namespace pid
00043     {
00048
00085         template <typename T>
00086         class PID_LPF_1
00087         {
00088         public:
00096             PID_LPF_1();
00097
00115             void init(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_);
00116
00134             void set_param(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_);
00135
00151             T update(T x_0, T x);
00152
00160             void reset();
00161
00172             void merge(T u_k_1_);
00173
00179             void set_dt(T dt_);
00180
00186             void set_Kp(T Kp_);
00187
00193             void set_Ki(T Ki_);
00194
00200             void set_Kd(T Kd_);
00201
00207             void set_fc(T fc_);
00208
00214             void set_u_0(T u_k_1_);
00215
00221             void set_u_max(T u_max_);
00222
00228             T get_dt();
00229
00235             T get_Kp();
00236
00242             T get_Ki();
00243
00249             T get_Kd();
00250
00256             T get_fc();
00257
00263             T get_u_k_1();
00264
00270             T get_e_k_1();
00271
00277             T get_e_k_2();
00278
00284             T get_u_max();
00285
00286         private:
00294             T dt = 0.0;
00295
00302             T u_k_1 = 0.0;
00303
00307             T u_k_2 = 0.0;
00308
00315             T e_k_1 = 0.0;
00316
00320             T e_k_2 = 0.0;
00321
00325             T Kp = 0.0;
00326
00330             T Ki = 0.0;
00331
00335             T Kd = 0.0;
00336
00344             T tau = 0.0;
00345
00353             T u_max = 9999999999;

```

```

00354
00362     uint8_t start = 0;
00363 };
00364
00365 #include "../src/pid/pid_lpf_1.tpp"
00366 }
00367 }
00368
00369 #endif

```

10.22 include/pid/pid_lpf_1_ff.h File Reference

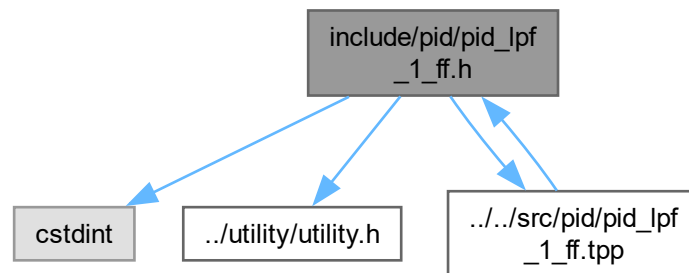
PID controller with first-order low-pass filtered derivative and feed-forward.

```

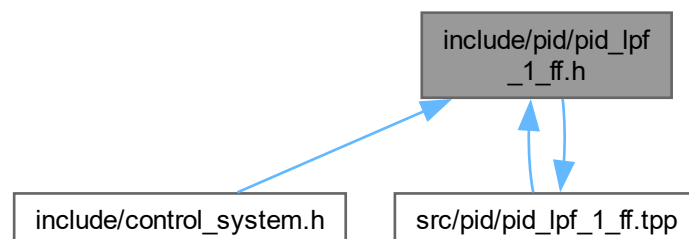
#include <stdint>
#include "../utility/utility.h"
#include "../src/pid/pid_lpf_1_ff.tpp"

```

Include dependency graph for pid_lpf_1_ff.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `control_system::pid::PID_LPF_1_FF< T >`
PID controller with first-order low-pass filtered derivative and feed-forward.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::pid](#)
- namespace [pid](#)
Contains proportional-integral-derivative controller implementations.

10.22.1 Detailed Description

PID controller with first-order low-pass filtered derivative and feed-forward.

This file defines the [control_system::pid::PID_LPF_1_FF](#) class, which implements a discrete-time proportional-integral-derivative (PID) controller with a first-order low-pass filtered derivative contribution and feed-forward control.

The controller uses configurable proportional, integral, and derivative gains together with a derivative filter cutoff frequency. A feed-forward contribution can be added to the PID controller output.

Previous controller outputs and control errors are maintained as internal state variables.

The controller output is constrained by a configurable maximum output limit.

Template Parameters

<i>T</i>	Numeric data type used for the controller calculations.
----------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [pid_lpf_1_ff.h](#).

10.23 pid_lpf_1_ff.h

[Go to the documentation of this file.](#)

```

00001
00030
00031 #ifndef CONTROL_SYSTEM_PID_PID_LPF_1_FF_H
00032 #define CONTROL_SYSTEM_PID_PID_LPF_1_FF_H
00033
00034 #include <cstdint>
00035 #include "../utility/utility.h"
00036
00037 namespace control_system
00038 {
00043
00044     namespace pid
00045     {
00050
00088         template <typename T>
00089         class PID_LPF_1_FF
00090         {
00091         public:
00100             PID_LPF_1_FF();
00101
00118             void init(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_);
00119
00137             void set_param(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T u_max_);
00138
00156             T update(T x_0, T x, T u_ff_ = 0.0);
00157
00165             void reset();
00166
00177             void merge(T u_k_1_);
00178
00184             void set_dt(T dt_);
00185
00191             void set_Kp(T Kp_);
00192
00198             void set_Ki(T Ki_);
00199
00205             void set_Kd(T Kd_);
00206
00212             void set_fc(T fc_);
00213
00219             void set_ff(T u_ff_);
00220
00226             void set_u_0(T u_k_1_);
00227
00233             void set_u_max(T u_max_);
00234
00240             T get_dt();
00241
00247             T get_Kp();
00248
00254             T get_Ki();
00255
00261             T get_Kd();
00262
00268             T get_fc();
00269
00275             T get_ff();
00276
00282             T get_u_k_1();
00283
00289             T get_e_k_1();
00290
00296             T get_e_k_2();
00297
00303             T get_u_max();
00304
00305         private:
00313             T dt = 0.0;
00314
00321             T u_k_1 = 0.0;
00322
00326             T u_k_2 = 0.0;
00327
00334             T e_k_1 = 0.0;
00335
00339             T e_k_2 = 0.0;
00340
00344             T Kp = 0.0;
00345
00349             T Ki = 0.0;
00350
00354             T Kd = 0.0;

```

```

00355
00363         T tau = 0.0;
00364
00368         T u_ff = 0.0;
00369
00377         T u_max = 9999999999;
00378
00386         uint8_t start = 0;
00387     };
00388
00389     #include "../../src/pid/pid_lpf_1_ff.hpp"
00390 }
00391 }
00392
00393 #endif

```

10.24 include/pid/pid_p.h File Reference

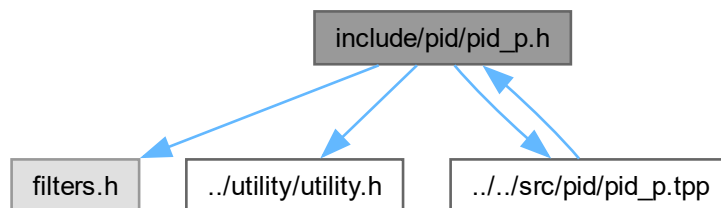
PID controller with configurable derivative filtering and feed-forward.

```

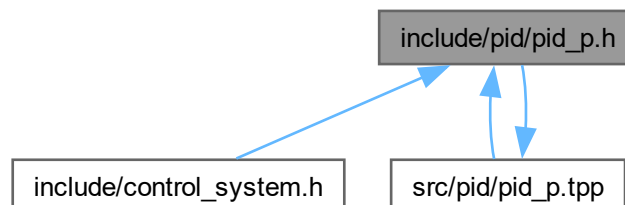
#include "filters.h"
#include "../utility/utility.h"
#include "../../src/pid/pid_p.hpp"

```

Include dependency graph for pid_p.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `control_system::pid::PID_P< T >`
PID controller with configurable derivative filtering and feed-forward.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::pid](#)
- namespace [pid](#)
Contains proportional-integral-derivative controller implementations.

10.24.1 Detailed Description

PID controller with configurable derivative filtering and feed-forward.

This file defines the [control_system::pid::PID_P](#) class, which implements a proportional-integral-derivative (PID) controller with configurable derivative filtering and feed-forward control.

The controller provides configurable proportional, integral, and derivative gains. The integral contribution and total controller output can be limited independently.

The derivative term can optionally be filtered using a first-order low-pass filter. A feed-forward term can also be added to the controller output.

The class provides access to the individual proportional, integral, and derivative contributions as well as the final controller output.

Template Parameters

<i>T</i>	Numeric data type used for the controller calculations.
----------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [pid_p.h](#).

10.25 pid_p.h

[Go to the documentation of this file.](#)

```

00001
00030
00031 #ifndef CONTROL_SYSTEM_PID_PID_P_H
00032 #define CONTROL_SYSTEM_PID_PID_P_H
00033
00034 #include "filters.h"
00035 #include "../utility/utility.h"
00036
00037 namespace control_system
00038 {
00043
00044     namespace pid
00045     {
00050
00085         template <typename T>
00086         class PID_P
00087         {
00088         public:
00096             PID_P();
00097
00114             void init(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_ = false, T fc_ =
00115 10.0);
00132             void set_param(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool d_filter_ = false, T
00133 fc_ = 10.0);
00151             T update(T x_0, T x, T u_ff_ = 0.0);
00152
00161             void reset();
00162
00173             void merge(T u_k_1_);
00174
00180             void set_dt(T dt_);
00181
00187             void set_Kp(T Kp_);
00188
00194             void set_Ki(T Ki_);
00195
00201             void set_Kd(T Kd_);
00202
00208             void set_I_max(T I_max_);
00209
00215             void set_u_max(T u_max_);
00216
00222             void set_d_filter(bool d_filter_);
00223
00229             void set_fc(T fc_);
00230
00236             void set_ff(T u_ff_);
00237
00243             T get_dt();
00244
00250             T get_Kp();
00251
00257             T get_Ki();
00258
00264             T get_Kd();
00265
00271             T get_I_max();
00272
00278             T get_u_max();
00279
00286             bool get_d_filter();
00287
00293             T get_fc();
00294
00300             T get_ff();
00301
00307             T get_P();
00308
00314             T get_I();
00315
00321             T get_D();
00322
00328             T get_u();
00329
00335             T get_e_k_1();
00336
00337         private:
00341             LPF_1<T> lpf;
00342
00349             T dt = 0.0;

```

```

00350
00354         T Kp = 0.0;
00355
00359         T Ki = 0.0;
00360
00364         T Kd = 0.0;
00365
00369         T I_max = 0.0;
00370
00374         T u_max = 0.0;
00375
00379         bool d_filter = false;
00380
00387         T fc = 0.0;
00388
00392         T e_k_1 = 0.0;
00393
00397         T P = 0.0;
00398
00402         T I = 0.0;
00403
00407         T D = 0.0;
00408
00412         T u = 0.0;
00413
00417         T u_ff = 0.0;
00418
00425         bool start = true;
00426     };
00427
00428     #include "../src/pid/pid_p.tpp"
00429     }
00430 }
00431
00432 #endif

```

10.26 include/pid/pid_s.h File Reference

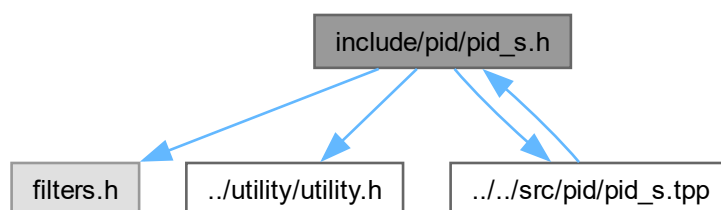
PID controller with setpoint weighting.

```

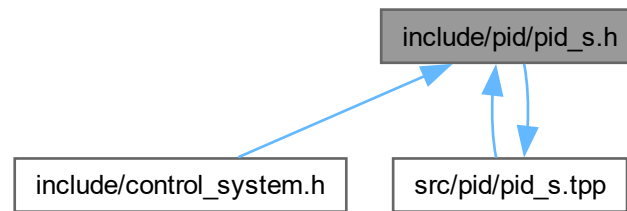
#include "filters.h"
#include "../utility/utility.h"
#include "../src/pid/pid_s.tpp"

```

Include dependency graph for pid_s.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `control_system::pid::PID_S< T >`
PID controller with configurable derivative filtering.

Namespaces

- namespace `control_system`
Contains control-system algorithms and controllers.
- namespace `control_system::pid`
- namespace `pid`
Contains proportional-integral-derivative controller implementations.

10.26.1 Detailed Description

PID controller with setpoint weighting.

This file defines the `control_system::pid::PID_S` class, which implements a proportional-integral-derivative (PID) controller with configurable derivative filtering and feed-forward control.

The controller provides access to the individual proportional, integral, and derivative contributions as well as the final control output.

Template Parameters

<i>T</i>	Numeric data type used for the controller calculations.
----------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file [pid_s.h](#).

10.27 pid_s.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 #ifndef CONTROL_SYSTEM_PID_PID_S_H
00025 #define CONTROL_SYSTEM_PID_PID_S_H
00026
00027 #include "filters.h"
00028 #include "../utility/utility.h"
00029
00030 namespace control_system
00031 {
00036
00037     namespace pid
00038     {
00043
00077         template <typename T>
00078         class PID_S
00079         {
00080         public:
00088             PID_S();
00089
00106             void init(T dt_, T Kp_, T Ti_, T Td_, T I_max_, T u_max_, bool d_filter_ = false, T fc_
= 10.0);
00107
00124             void set_param(T dt_, T Kp_, T Ti_, T Td_, T I_max_, T u_max_, bool d_filter_ = false, T
fc_ = 10.0);
00125
00143             T update(T x_0, T x, T u_ff_ = 0.0);
00144
00152             void reset();
00153
00164             void merge(T u_k_1);
00165
00171             void set_dt(T dt_);
00172
00178             void set_Kp(T Kp_);
00179
00185             void set_Ti(T Ti_);
00186
00192             void set_Td(T Td_);
00193
00199             void set_I_max(T I_max_);
00200
00206             void set_u_max(T u_max_);
00207
00213             void set_d_filter(bool d_filter_);
00214
00220             void set_fc(T fc_);
00221
00227             void set_ff(T u_ff_);
00228
00234             T get_dt();
00235
00241             T get_Kp();
00242
00248             T get_Ti();
00249
00255             T get_Td();
00256
00262             T get_I_max();
00263
00269             T get_u_max();
00270
00277             bool get_d_filter();

```



```

00278
00284         T get_fc();
00285
00291         T get_ff();
00292
00298         T get_P();
00299
00305         T get_I();
00306
00312         T get_D();
00313
00319         T get_u();
00320
00326         T get_e_k_1();
00327
00328     private:
00332         LPF_1<T> lpf;
00333
00340         T dt = 0.0;
00341
00345         T Kp = 0.0;
00346
00350         T T_i = 0.0;
00351
00355         T T_d = 0.0;
00356
00360         T I_max = 0.0;
00361
00365         T u_max = 0.0;
00366
00370         bool d_filter = false;
00371
00378         T fc = 0.0;
00379
00383         T e_k_1 = 0.0;
00384
00388         T P = 0.0;
00389
00393         T I = 0.0;
00394
00398         T D = 0.0;
00399
00403         T u = 0.0;
00404
00408         T u_ff = 0.0;
00409
00416         bool start = true;
00417     };
00418
00419 #include "../src/pid/pid_s.tpp"
00420 }
00421 }
00422
00423 #endif

```

10.28 include/proportional/p.h File Reference

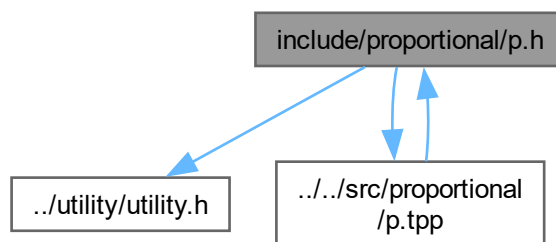
Proportional (P) controller.

```

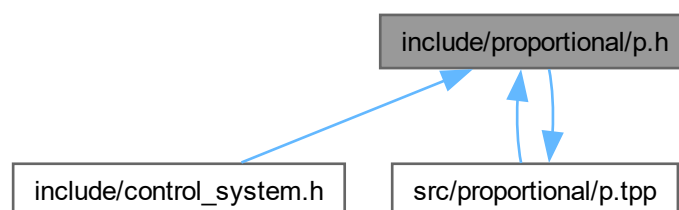
#include "../utility/utility.h"
#include "../src/proportional/p.tpp"

```

Include dependency graph for p.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [control_system::proportional::P< T >](#)
Proportional controller.

Namespaces

- namespace [control_system](#)
Contains control-system algorithms and controllers.
- namespace [control_system::proportional](#)
- namespace [proportional](#)
Contains proportional control implementations.

10.28.1 Detailed Description

Proportional (P) controller.

This file defines the `control_system::proportional::P` class, which implements a proportional controller.

The controller generates an output proportional to the difference between the reference input and the measured or current value.

The controller output is limited by the configured maximum output value.

The class is templated to support different numeric types such as `float` and `double`.

Template Parameters

<code>T</code>	Numeric data type used for the controller calculations.
----------------	---

Author

Abhinay Kumar

Version

1.0.0

Date

2026-08-11

Copyright

Copyright (c) 2026 Abhinay Kumar

Definition in file `p.h`.

10.29 p.h

[Go to the documentation of this file.](#)

```

00001
00027
00028 #ifndef CONTROL_SYSTEM_PROPORTIONAL_P_H
00029 #define CONTROL_SYSTEM_PROPORTIONAL_P_H
00030
00031 #include "../utility/utility.h"
00032
00033 namespace control_system
00034 {
00039     namespace proportional
00040     {
00041     {
00046
00069         template <typename T>
00070         class P
00071         {
00072         public:
00080             P();
00081
00091             void init(T Kp_, T u_max_);

```


10.30.2 Function Documentation

10.30.2.1 saturate()

```
template<typename T>
T saturate (
    T x,
    T x_min,
    T x_max) [inline], [constexpr]
```

Saturates a value within a specified range.

If `x` is less than the minimum limit, the minimum limit is returned. If `x` is greater than the maximum limit, the maximum limit is returned. Otherwise, the input value is returned unchanged.

The function is implemented as a generic function and can be used with different numeric types.

Template Parameters

<i>T</i>	Numeric data type.
----------	--------------------

Parameters

in	<i>x</i>	Input value to be saturated.
in	<i>x_min</i>	Minimum allowed value.
in	<i>x_max</i>	Maximum allowed value.

Returns

Saturated value within the range `[x_min, x_max]`.

Note

The caller should ensure that `x_min` is less than or equal to `x_max`.

Example

```
double output = saturate(input, -10.0, 10.0);
```

Definition at line 51 of file [utility.h](#).

10.31 utility.h

[Go to the documentation of this file.](#)

```

00001
00016
00046
00047 #ifndef CONTROL_SYSTEM_PID_UTILITY_H
00048 #define CONTROL_SYSTEM_PID_UTILITY_H
00049
00050 template <typename T>
00051 inline constexpr T saturate(T x, T x_min, T x_max)
00052 {
00053     if (x < x_min)
00054     {
00055         return x_min;
00056     }
00057     else if (x > x_max)
00058     {
00059         return x_max;
00060     }
00061     else
00062     {
00063         return x;
00064     }
00065 }
00066
00067 #endif // CONTROL_SYSTEM_PID_UTILITY_H

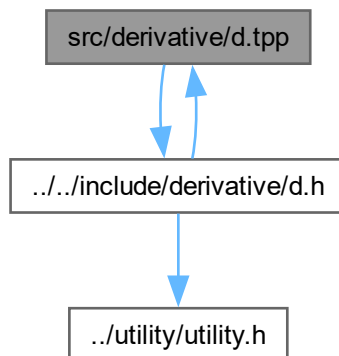
```

10.32 README.md File Reference

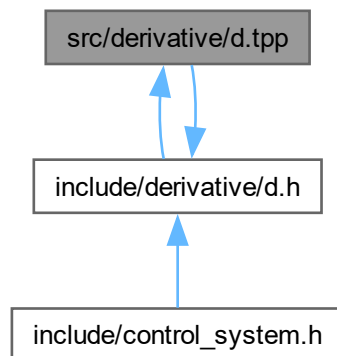
10.33 src/derivative/d.hpp File Reference

```
#include "../..//include/derivative/d.h"
```

Include dependency graph for d.hpp:



This graph shows which files directly or indirectly include this file:



10.34 d.hpp

[Go to the documentation of this file.](#)

```

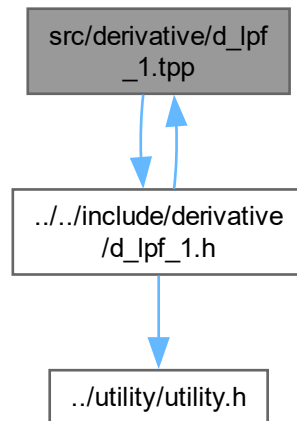
00001 #include "../include/derivative/d.h"
00002
00003 template <typename T>
00004 control_system::derivative::D<T>::D()
00005 {
00006     dt = 0.0;
00007     e_k_1 = 0.0;
00008     Kd = 0.0;
00009     start = true;
00010 }
00011
00012 template <typename T>
00013 void control_system::derivative::D<T>::init(T dt_, T Kd_, T u_max_)
00014 {
00015     set_param(dt_, Kd_, u_max_);
00016     start = true;
00017 }
00018
00019 template <typename T>
00020 void control_system::derivative::D<T>::set_param(T dt_, T Kd_, T u_max_)
00021 {
00022     dt = dt_;
00023     Kd = Kd_;
00024     u_max = u_max_;
00025 }
00026
00027 template <typename T>
00028 T control_system::derivative::D<T>::update(T x_0, T x)
00029 {
00030     T e_k = x_0 - x;
00031     T u_k = 0.0;
00032     if (start == true)
00033     {
00034         start = false;
00035         u_k = 0.0;
00036     }
00037     else
00038     {
00039         u_k = Kd * (e_k - e_k_1) / dt;
00040     }
00041     u_k = saturate(u_k, -u_max, u_max);
00042     e_k_1 = e_k;
00043     return u_k;
00044 }
00045
00046 template <typename T>
  
```

```
00047 void control_system::derivative::D<T>::reset ()
00048 {
00049     e_k_1 = 0.0;
00050     start = true;
00051 }
00052
00053 template <typename T>
00054 void control_system::derivative::D<T>::merge(T u_k_1_)
00055 {
00056 }
00057
00058 template <typename T>
00059 void control_system::derivative::D<T>::set_dt(T dt_)
00060 {
00061     dt = dt_;
00062 }
00063
00064 template <typename T>
00065 void control_system::derivative::D<T>::set_Kd(T Kd_)
00066 {
00067     Kd = Kd_;
00068 }
00069
00070 template <typename T>
00071 void control_system::derivative::D<T>::set_u_max(T u_max_)
00072 {
00073     u_max = u_max_;
00074 }
00075
00076 template <typename T>
00077 T control_system::derivative::D<T>::get_dt ()
00078 {
00079     return dt;
00080 }
00081
00082 template <typename T>
00083 T control_system::derivative::D<T>::get_Kd()
00084 {
00085     return Kd;
00086 }
00087
00088 template <typename T>
00089 T control_system::derivative::D<T>::get_e_k_1 ()
00090 {
00091     return e_k_1;
00092 }
```

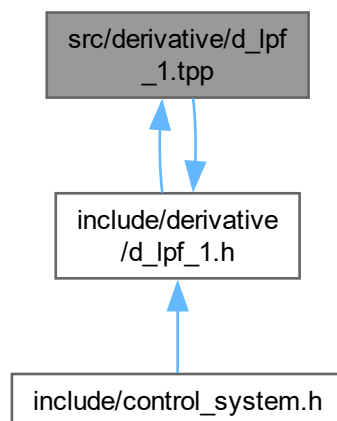

10.35 src/derivative/d_lpf_1.tpp File Reference

```
#include "../../include/derivative/d_lpf_1.h"
```

Include dependency graph for d_lpf_1.tpp:



This graph shows which files directly or indirectly include this file:



10.36 d_lpf_1.tpp

[Go to the documentation of this file.](#)

```

00001 #include "../include/derivative/d_lpf_1.h"
00002
00003 template <typename T>
00004 control_system::derivative::D_LPF_1<T>::D_LPF_1()
00005 {
00006     dt = 0.0;
00007     e_k_1 = 0.0;
00008     Kd = 0.0;
00009     start = true;
00010
00011     u_k_1 = 0.0;
00012     tau = 0.0;
00013 }
00014
00015 template <typename T>
00016 void control_system::derivative::D_LPF_1<T>::init(T dt_, T Kd_, T fc_, T u_max_)
00017 {
00018     set_param(dt_, Kd_, fc_, u_max_);
00019     start = true;
00020 }
00021
00022 template <typename T>
00023 void control_system::derivative::D_LPF_1<T>::set_param(T dt_, T Kd_, T fc_, T u_max_)
00024 {
00025     dt = dt_;
00026     Kd = Kd_;
00027     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00028     u_max = u_max_;
00029 }
00030
00031 template <typename T>
00032 T control_system::derivative::D_LPF_1<T>::update(T x_0, T x)
00033 {
00034     T e_k = x_0 - x;
00035     T u_k = 0.0;
00036     if (start == true)
00037     {
00038         start = false;
00039         u_k = 0.0;
00040     }
00041     else
00042     {
00043         u_k = (tau * u_k_1 + Kd * (e_k - e_k_1)) / (tau + dt);
00044     }
00045     u_k = saturate(u_k, -u_max, u_max);
00046     e_k_1 = e_k;
00047     u_k_1 = u_k;
00048     return u_k;
00049 }
00050
00051 template <typename T>
00052 void control_system::derivative::D_LPF_1<T>::reset()
00053 {
00054     e_k_1 = 0.0;
00055     start = true;
00056     u_k_1 = 0.0;
00057 }
00058
00059 template <typename T>
00060 void control_system::derivative::D_LPF_1<T>::merge(T u_k_1_)
00061 {
00062     u_k_1 = u_k_1_;
00063 }
00064
00065 template <typename T>
00066 void control_system::derivative::D_LPF_1<T>::set_dt(T dt_)
00067 {
00068     dt = dt_;
00069 }
00070
00071 template <typename T>
00072 void control_system::derivative::D_LPF_1<T>::set_Kd(T Kd_)
00073 {
00074     Kd = Kd_;
00075 }
00076
00077 template <typename T>
00078 void control_system::derivative::D_LPF_1<T>::set_fc(T fc_)
00079 {
00080     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00081 }
00082
00083 template <typename T>
00084 void control_system::derivative::D_LPF_1<T>::set_u_max(T u_max_)
00085 {
00086     u_max = u_max_;
00087 }

```

```

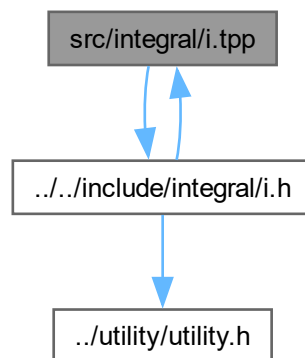
00088
00089 template <typename T>
00090 T control_system::derivative::D_LPF_1<T>::get_dt()
00091 {
00092     return dt;
00093 }
00094
00095 template <typename T>
00096 T control_system::derivative::D_LPF_1<T>::get_Kd()
00097 {
00098     return Kd;
00099 }
00100
00101 template <typename T>
00102 T control_system::derivative::D_LPF_1<T>::get_fc()
00103 {
00104     return 1.0 / (6.28318530717958647692528676655900576 * tau);
00105 }
00106
00107 template <typename T>
00108 T control_system::derivative::D_LPF_1<T>::get_e_k_1()
00109 {
00110     return e_k_1;
00111 }
00112
00113 template <typename T>
00114 T control_system::derivative::D_LPF_1<T>::get_u_k_1()
00115 {
00116     return u_k_1;
00117 }
00118
00119 template <typename T>
00120 T control_system::derivative::D_LPF_1<T>::get_u_max()
00121 {
00122     return u_max;
00123 }

```

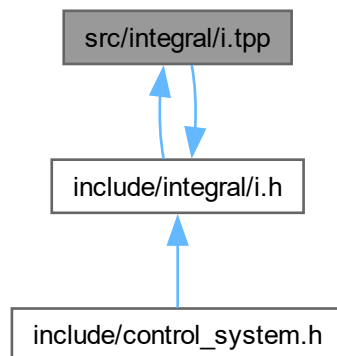
10.37 src/integral/i.tpp File Reference

```
#include "../..../include/integral/i.h"
```

Include dependency graph for i.tpp:



This graph shows which files directly or indirectly include this file:



10.38 i.tpp

[Go to the documentation of this file.](#)

```

00001 #include "../include/integral/i.h"
00002
00003 template <typename T>
00004 control_system::integral::I<T>::I()
00005 {
00006     dt = 0.0;
00007     u_k_1 = 0.0;
00008     Ki = 0.0;
00009 }
00010
00011 template <typename T>
00012 void control_system::integral::I<T>::init(T dt_, T Ki_, T u_k_1_, T u_max_)
00013 {
00014     set_param(dt_, Ki_, u_k_1_, u_max_);
00015 }
00016
00017 template <typename T>
00018 void control_system::integral::I<T>::set_param(T dt_, T Ki_, T u_k_1_, T u_max_)
00019 {
00020     dt = dt_;
00021     Ki = Ki_;
00022     u_k_1 = u_k_1_;
00023     u_max = u_max_;
00024 }
00025
00026 template <typename T>
00027 T control_system::integral::I<T>::update(T x_0, T x)
00028 {
00029     T e_k = x_0 - x;
00030     T u_k = u_k_1 + Ki * dt * e_k;
00031     u_k = saturate(u_k, -u_max, u_max);
00032     u_k_1 = u_k;
00033     return u_k;
00034 }
00035
00036 template <typename T>
00037 void control_system::integral::I<T>::reset()
00038 {
00039     u_k_1 = 0.0;
00040 }
00041
00042 template <typename T>
00043 void control_system::integral::I<T>::merge(T u_k_1_)
00044 {
00045     u_k_1 = u_k_1_;
00046 }
  
```

```

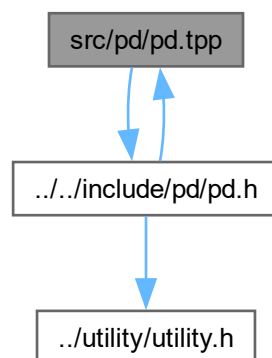
00047
00048 template <typename T>
00049 void control_system::integral::I<T>::set_dt(T dt_)
00050 {
00051     dt = dt_;
00052 }
00053
00054 template <typename T>
00055 void control_system::integral::I<T>::set_Ki(T Ki_)
00056 {
00057     Ki = Ki_;
00058 }
00059
00060 template <typename T>
00061 void control_system::integral::I<T>::set_u_0(T u_k_1_)
00062 {
00063     u_k_1 = u_k_1_;
00064 }
00065
00066 template <typename T>
00067 void control_system::integral::I<T>::set_u_max(T u_max_)
00068 {
00069     u_max = u_max_;
00070 }
00071
00072 template <typename T>
00073 T control_system::integral::I<T>::get_dt()
00074 {
00075     return dt;
00076 }
00077
00078 template <typename T>
00079 T control_system::integral::I<T>::get_Ki()
00080 {
00081     return Ki;
00082 }
00083
00084 template <typename T>
00085 T control_system::integral::I<T>::get_u_k_1()
00086 {
00087     return u_k_1;
00088 }
00089
00090 template <typename T>
00091 T control_system::integral::I<T>::get_u_max()
00092 {
00093     return u_max;
00094 }

```

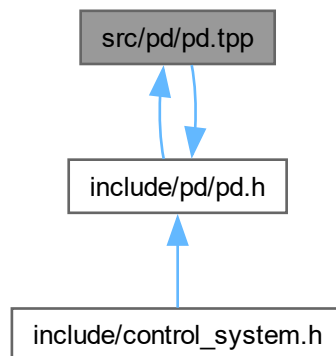
10.39 src/pd/pd.hpp File Reference

```
#include "../..//include/pd/pd.h"
```

Include dependency graph for pd.hpp:



This graph shows which files directly or indirectly include this file:



10.40 pd.hpp

[Go to the documentation of this file.](#)

```

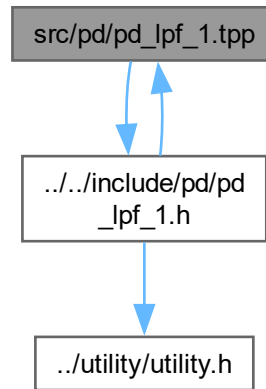
00001 #include "../include/pd/pd.h"
00002
00003 template <typename T>
00004 control_system::pd::PD<T>::PD()
00005 {
00006     dt = 0.0;
00007     e_k_1 = 0.0;
00008     Kp = 0.0;
00009     Kd = 0.0;
00010     start = true;
00011 }
00012
00013 template <typename T>
00014 void control_system::pd::PD<T>::init(T dt_, T Kp_, T Kd_, T u_max_)
00015 {
00016     set_param(dt_, Kp_, Kd_, u_max_);
00017     start = true;
00018 }
00019
00020 template <typename T>
00021 void control_system::pd::PD<T>::set_param(T dt_, T Kp_, T Kd_, T u_max_)
00022 {
00023     dt = dt_;
00024     Kp = Kp_;
00025     Kd = Kd_;
00026     u_max = u_max_;
00027 }
00028
00029 template <typename T>
00030 T control_system::pd::PD<T>::update(T x_0, T x)
00031 {
00032     T e_k = x_0 - x;
00033     T u_k = 0.0;
00034     if (start == true)
00035     {
00036         start = false;
00037         u_k = Kp * e_k;
00038     }
00039     else
00040     {
00041         u_k = (Kp + Kd / dt) * e_k - (Kd / dt) * e_k_1;
00042     }
00043     u_k = saturate(u_k, -u_max, u_max);
00044     e_k_1 = e_k;
00045     return u_k;
00046 }
  
```

```
00047
00048 template <typename T>
00049 void control_system::pd::PD<T>::reset ()
00050 {
00051     e_k_1 = 0.0;
00052     start = true;
00053 }
00054
00055 template <typename T>
00056 void control_system::pd::PD<T>::merge (T u_k_1_)
00057 {
00058 }
00059
00060 template <typename T>
00061 void control_system::pd::PD<T>::set_dt (T dt_)
00062 {
00063     dt = dt_;
00064 }
00065
00066 template <typename T>
00067 void control_system::pd::PD<T>::set_Kp (T Kp_)
00068 {
00069     Kp = Kp_;
00070 }
00071
00072 template <typename T>
00073 void control_system::pd::PD<T>::set_Kd (T Kd_)
00074 {
00075     Kd = Kd_;
00076 }
00077
00078 template <typename T>
00079 void control_system::pd::PD<T>::set_u_max (T u_max_)
00080 {
00081     u_max = u_max_;
00082 }
00083
00084 template <typename T>
00085 T control_system::pd::PD<T>::get_dt ()
00086 {
00087     return dt;
00088 }
00089
00090 template <typename T>
00091 T control_system::pd::PD<T>::get_Kp ()
00092 {
00093     return Kp;
00094 }
00095
00096 template <typename T>
00097 T control_system::pd::PD<T>::get_Kd ()
00098 {
00099     return Kd;
00100 }
00101
00102 template <typename T>
00103 T control_system::pd::PD<T>::get_e_k_1 ()
00104 {
00105     return e_k_1;
00106 }
00107
00108 template <typename T>
00109 T control_system::pd::PD<T>::get_u_max ()
00110 {
00111     return u_max;
00112 }
```

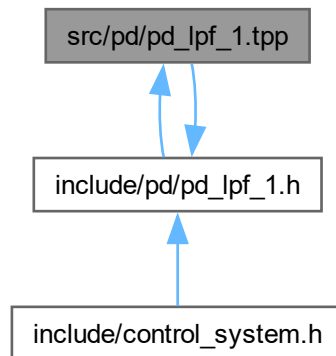
10.41 src/pd/pd_lpf_1.tpp File Reference

```
#include "../../include/pd/pd_lpf_1.h"
```

Include dependency graph for pd_lpf_1.tpp:



This graph shows which files directly or indirectly include this file:



10.42 pd_lpf_1.tpp

[Go to the documentation of this file.](#)

```

00001 #include "../../include/pd/pd_lpf_1.h"
00002
00003 template <typename T>
00004 control_system::pd::PD_LPF_1<T>::PD_LPF_1()
00005 {

```



```

00006     dt = 0.0;
00007     e_k_1 = 0.0;
00008     Kd = 0.0;
00009     start = true;
00010
00011     u_k_1 = 0.0;
00012     tau = 0.0;
00013 }
00014
00015 template <typename T>
00016 void control_system::pd::PD_LPF_1<T>::init(T dt_, T Kp_, T Kd_, T fc_, T u_max_)
00017 {
00018     set_param(dt_, Kp_, Kd_, fc_, u_max_);
00019     start = true;
00020 }
00021
00022 template <typename T>
00023 void control_system::pd::PD_LPF_1<T>::set_param(T dt_, T Kp_, T Kd_, T fc_, T u_max_)
00024 {
00025     dt = dt_;
00026     Kp = Kp_;
00027     Kd = Kd_;
00028     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00029     u_max = u_max_;
00030 }
00031
00032 template <typename T>
00033 T control_system::pd::PD_LPF_1<T>::update(T x_0, T x)
00034 {
00035     T e_k = x_0 - x;
00036     T u_k = 0.0;
00037     if (start == true)
00038     {
00039         start = false;
00040         u_k = Kp * e_k;
00041     }
00042     else
00043     {
00044         u_k = (tau * u_k_1 + (Kp * (dt + tau) + Kd) * e_k - (Kp * tau + Kd) * e_k_1) / (tau + dt);
00045     }
00046     u_k = saturate(u_k, -u_max, u_max);
00047     e_k_1 = e_k;
00048     u_k_1 = u_k;
00049     return u_k;
00050 }
00051
00052 template <typename T>
00053 void control_system::pd::PD_LPF_1<T>::reset()
00054 {
00055     e_k_1 = 0.0;
00056     start = true;
00057     u_k_1 = 0.0;
00058 }
00059
00060 template <typename T>
00061 void control_system::pd::PD_LPF_1<T>::merge(T u_k_1_)
00062 {
00063 }
00064
00065 template <typename T>
00066 void control_system::pd::PD_LPF_1<T>::set_dt(T dt_)
00067 {
00068     dt = dt_;
00069 }
00070
00071 template <typename T>
00072 void control_system::pd::PD_LPF_1<T>::set_Kp(T Kp_)
00073 {
00074     Kp = Kp_;
00075 }
00076
00077 template <typename T>
00078 void control_system::pd::PD_LPF_1<T>::set_Kd(T Kd_)
00079 {
00080     Kd = Kd_;
00081 }
00082
00083 template <typename T>
00084 void control_system::pd::PD_LPF_1<T>::set_fc(T fc_)
00085 {
00086     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00087 }
00088
00089 template <typename T>
00090 void control_system::pd::PD_LPF_1<T>::set_u_max(T u_max_)
00091 {
00092     u_max = u_max_;

```

```

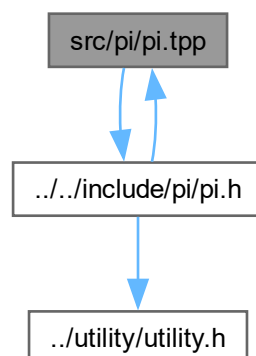
00093 }
00094
00095 template <typename T>
00096 T control_system::pd::PD_LPF_1<T>::get_dt()
00097 {
00098     return dt;
00099 }
00100
00101 template <typename T>
00102 T control_system::pd::PD_LPF_1<T>::get_Kp()
00103 {
00104     return Kp;
00105 }
00106
00107 template <typename T>
00108 T control_system::pd::PD_LPF_1<T>::get_Kd()
00109 {
00110     return Kd;
00111 }
00112
00113 template <typename T>
00114 T control_system::pd::PD_LPF_1<T>::get_fc()
00115 {
00116     return 1.0 / (6.28318530717958647692528676655900576 * tau);
00117 }
00118
00119 template <typename T>
00120 T control_system::pd::PD_LPF_1<T>::get_e_k_1()
00121 {
00122     return e_k_1;
00123 }
00124
00125 template <typename T>
00126 T control_system::pd::PD_LPF_1<T>::get_u_k_1()
00127 {
00128     return u_k_1;
00129 }
00130
00131 template <typename T>
00132 T control_system::pd::PD_LPF_1<T>::get_u_max()
00133 {
00134     return u_max;
00135 }

```

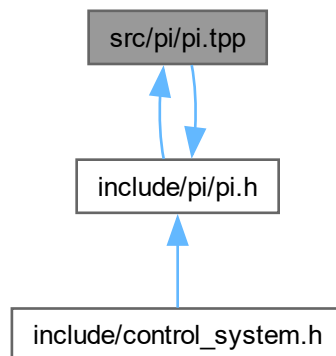
10.43 src/pi/pi.hpp File Reference

```
#include "../include/pi/pi.h"
```

Include dependency graph for pi.hpp:



This graph shows which files directly or indirectly include this file:



10.44 pi.hpp

[Go to the documentation of this file.](#)

```

00001 #include "../include/pi/pi.h"
00002
00003 template <typename T>
00004 control_system::pi::PI<T>::PI()
00005 {
00006     dt = 0.0;
00007     Kp = 0.0;
00008     Ki = 0.0;
00009     u_k_1 = 0.0;
00010     e_k_1 = 0.0;
00011     start = true;
00012 }
00013
00014 template <typename T>
00015 void control_system::pi::PI<T>::init(T dt_, T Kp_, T Ki_, T u_k_1_, T u_max_)
00016 {
00017     set_param(dt_, Kp_, Ki_, u_k_1_, u_max_);
00018     start = true;
00019 }
00020
00021 template <typename T>
00022 void control_system::pi::PI<T>::set_param(T dt_, T Kp_, T Ki_, T u_k_1_, T u_max_)
00023 {
00024     dt = dt_;
00025     Kp = Kp_;
00026     Ki = Ki_;
00027     u_k_1 = u_k_1_;
00028     u_max = u_max_;
00029 }
00030
00031 template <typename T>
00032 T control_system::pi::PI<T>::update(T x_0, T x)
00033 {
00034     T e_k = x_0 - x;
00035     T u_k = 0.0;
00036     if (start == true)
00037     {
00038         start = false;
00039         u_k = u_k_1 + Kp * e_k;
00040     }
00041     else
00042     {
00043         u_k = u_k_1 + (Kp + Ki * dt) * e_k - Kp * e_k_1;
00044     }
00045     u_k = saturate(u_k, -u_max, u_max);
00046     u_k_1 = u_k;
  
```

```

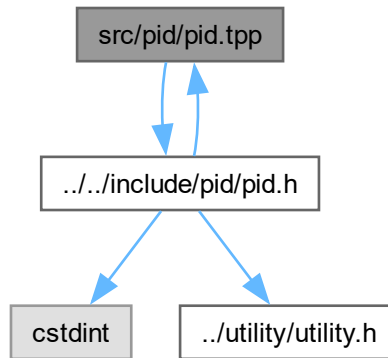
00047     e_k_1 = e_k;
00048     return u_k;
00049 }
00050
00051 template <typename T>
00052 void control_system::pi::PI<T>::reset ()
00053 {
00054     u_k_1 = 0.0;
00055     e_k_1 = 0.0;
00056     start = false;
00057 }
00058
00059 template <typename T>
00060 void control_system::pi::PI<T>::merge (T u_k_1_)
00061 {
00062     u_k_1 = u_k_1_;
00063 }
00064
00065 template <typename T>
00066 void control_system::pi::PI<T>::set_dt (T dt_)
00067 {
00068     dt = dt_;
00069 }
00070
00071 template <typename T>
00072 void control_system::pi::PI<T>::set_Kp (T Kp_)
00073 {
00074     Kp = Kp_;
00075 }
00076
00077 template <typename T>
00078 void control_system::pi::PI<T>::set_Ki (T Ki_)
00079 {
00080     Ki = Ki_;
00081 }
00082
00083 template <typename T>
00084 void control_system::pi::PI<T>::set_u_0 (T u_k_1_)
00085 {
00086     u_k_1 = u_k_1_;
00087 }
00088
00089 template <typename T>
00090 void control_system::pi::PI<T>::set_u_max (T u_max_)
00091 {
00092     u_max = u_max_;
00093 }
00094
00095 template <typename T>
00096 T control_system::pi::PI<T>::get_dt ()
00097 {
00098     return dt;
00099 }
00100
00101 template <typename T>
00102 T control_system::pi::PI<T>::get_Kp ()
00103 {
00104     return Kp;
00105 }
00106
00107 template <typename T>
00108 T control_system::pi::PI<T>::get_Ki ()
00109 {
00110     return Ki;
00111 }
00112
00113 template <typename T>
00114 T control_system::pi::PI<T>::get_u_k_1 ()
00115 {
00116     return u_k_1;
00117 }
00118
00119 template <typename T>
00120 T control_system::pi::PI<T>::get_u_max ()
00121 {
00122     return u_max;
00123 }

```

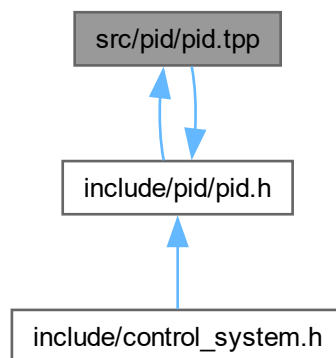
10.45 src/pid/pid.hpp File Reference

```
#include "../..//include/pid/pid.h"
```

Include dependency graph for pid.hpp:



This graph shows which files directly or indirectly include this file:



10.46 pid.hpp

[Go to the documentation of this file.](#)

```

00001 #include "../..//include/pid/pid.h"
00002
00003 template <typename T>
00004 control_system::pid::PID<T>::PID()
00005 {
00006     dt = 0.0;

```

```

00007     Kp = 0.0;
00008     Ki = 0.0;
00009     Kd = 0.0;
00010     u_k_1 = 0.0;
00011     e_k_1 = 0.0;
00012     e_k_2 = 0.0;
00013     start = 0;
00014 }
00015
00016 template <typename T>
00017 void control_system::pid::PID<T>::init(T dt_, T Kp_, T Ki_, T Kd_, T u_k_1_, T u_max_)
00018 {
00019     set_param(dt_, Kp_, Ki_, Kd_, u_k_1_, u_max_);
00020     start = 0;
00021 }
00022
00023 template <typename T>
00024 void control_system::pid::PID<T>::set_param(T dt_, T Kp_, T Ki_, T Kd_, T u_k_1_, T u_max_)
00025 {
00026     dt = dt_;
00027     Kp = Kp_;
00028     Ki = Ki_;
00029     Kd = Kd_;
00030     u_k_1 = u_k_1_;
00031     u_max = u_max_;
00032 }
00033
00034 template <typename T>
00035 T control_system::pid::PID<T>::update(T x_0, T x)
00036 {
00037     T e_k = x_0 - x;
00038     T u_k = 0.0;
00039     if ((start == 0) || (start == 1))
00040     {
00041         start++;
00042         u_k = u_k_1 + Kp * e_k;
00043     }
00044     else
00045     {
00046         u_k = u_k_1 + (Kp + Ki * dt + Kd / dt) * e_k - (Kp + 2.0 * Kd / dt) * e_k_1 + (Kd / dt) *
00047         e_k_2;
00048     }
00049     u_k = saturate(u_k, -u_max, u_max);
00050     u_k_1 = u_k;
00051     e_k_2 = e_k_1;
00052     e_k_1 = e_k;
00053     return u_k;
00054 }
00055
00056 template <typename T>
00057 void control_system::pid::PID<T>::reset()
00058 {
00059     u_k_1 = 0.0;
00060     e_k_1 = 0.0;
00061     e_k_2 = 0.0;
00062     start = 0;
00063 }
00064
00065 template <typename T>
00066 void control_system::pid::PID<T>::merge(T u_k_1_)
00067 {
00068     u_k_1 = u_k_1_;
00069 }
00070
00071 template <typename T>
00072 void control_system::pid::PID<T>::set_dt(T dt_)
00073 {
00074     dt = dt_;
00075 }
00076
00077 template <typename T>
00078 void control_system::pid::PID<T>::set_Kp(T Kp_)
00079 {
00080     Kp = Kp_;
00081 }
00082
00083 template <typename T>
00084 void control_system::pid::PID<T>::set_Ki(T Ki_)
00085 {
00086     Ki = Ki_;
00087 }
00088
00089 template <typename T>
00090 void control_system::pid::PID<T>::set_Kd(T Kd_)
00091 {
00092     Kd = Kd_;
00093 }

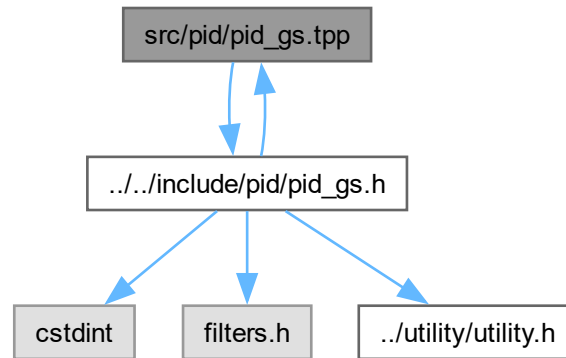
```

```
00093
00094 template <typename T>
00095 void control_system::pid::PID<T>::set_u_0(T u_k_1_)
00096 {
00097     u_k_1 = u_k_1_;
00098 }
00099
00100 template <typename T>
00101 void control_system::pid::PID<T>::set_u_max(T u_max_)
00102 {
00103     u_max = u_max_;
00104 }
00105
00106 template <typename T>
00107 T control_system::pid::PID<T>::get_dt()
00108 {
00109     return dt;
00110 }
00111
00112 template <typename T>
00113 T control_system::pid::PID<T>::get_Kp()
00114 {
00115     return Kp;
00116 }
00117
00118 template <typename T>
00119 T control_system::pid::PID<T>::get_Ki()
00120 {
00121     return Ki;
00122 }
00123
00124 template <typename T>
00125 T control_system::pid::PID<T>::get_Kd()
00126 {
00127     return Kd;
00128 }
00129
00130 template <typename T>
00131 T control_system::pid::PID<T>::get_u_k_1()
00132 {
00133     return u_k_1;
00134 }
00135
00136 template <typename T>
00137 T control_system::pid::PID<T>::get_e_k_1()
00138 {
00139     return e_k_1;
00140 }
00141
00142 template <typename T>
00143 T control_system::pid::PID<T>::get_e_k_2()
00144 {
00145     return e_k_2;
00146 }
00147
00148 template <typename T>
00149 T control_system::pid::PID<T>::get_u_max()
00150 {
00151     return u_max;
00152 }
```

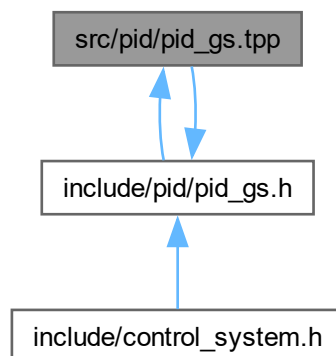
10.47 src/pid/pid_gs.tpp File Reference

```
#include "../../include/pid/pid_gs.h"
```

Include dependency graph for pid_gs.tpp:



This graph shows which files directly or indirectly include this file:



10.48 pid_gs.tpp

[Go to the documentation of this file.](#)

```

00001 #include "../../include/pid/pid_gs.h"
00002
00003 template <typename T>
00004 control_system::pid::PID_GS<T>::PID_GS()
00005 {
00006     dt = 0.0;

```



```

00007     Kp = 0.0;
00008     Ki = 0.0;
00009     Kd = 0.0;
00010     fc = 0.0;
00011     I_max = 0.0;
00012     u_max = 0.0;
00013     d_filter = false;
00014     e_k_1 = 0.0;
00015     P = 0.0;
00016     I = 0.0;
00017     D = 0.0;
00018     u = 0.0;
00019     u_ff = 0.0;
00020     start = true;
00021 }
00022
00023 template <typename T>
00024 void control_system::pid::PID_GS<T>::init(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool
    d_filter_, T fc_)
00025 {
00026     set_param(dt_, Kp_, Ki_, Kd_, I_max_, u_max_, d_filter_, fc_);
00027     start = true;
00028 }
00029
00030 template <typename T>
00031 void control_system::pid::PID_GS<T>::set_param(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool
    d_filter_, T fc_)
00032 {
00033     dt = dt_;
00034     Kp = Kp_;
00035     Ki = Ki_;
00036     Kd = Kd_;
00037     I_max = I_max_;
00038     u_max = u_max_;
00039     d_filter = d_filter_;
00040     fc = fc_;
00041     lpf.set_param(fc, dt);
00042 }
00043
00044 template <typename T>
00045 T control_system::pid::PID_GS<T>::update(T x_0, T x, T u_ff_, T op_cond_)
00046 {
00047     update_gain(op_cond_);
00048
00049     T e_k = x_0 - x;
00050     u_ff = u_ff_;
00051
00052     P = Kp * e_k;
00053
00054     I = I + Ki * e_k * dt;
00055     I = saturate(I, -I_max, I_max);
00056
00057     if (start == true)
00058     {
00059         D = 0.0;
00060         start = false;
00061     }
00062     else
00063     {
00064         D = Kd * (e_k - e_k_1) / dt;
00065     }
00066
00067     if (d_filter == true)
00068     {
00069         lpf.cal_y(D);
00070         D = lpf.get_y();
00071     }
00072     e_k_1 = e_k;
00073     u = u_ff + P + I + D;
00074     u = saturate(u, -u_max, u_max);
00075     return u;
00076 }
00077
00078 template <typename T>
00079 void control_system::pid::PID_GS<T>::reset()
00080 {
00081     e_k_1 = 0.0;
00082     P = 0.0;
00083     I = 0.0;
00084     D = 0.0;
00085     u = 0.0;
00086     u_ff = 0.0;
00087     start = true;
00088     lpf.reset();
00089 }
00090
00091 template <typename T>

```

```

00092 void control_system::pid::PID_GS<T>::merge(T u_k_l_)
00093 {
00094 }
00095
00096 template <typename T>
00097 void control_system::pid::PID_GS<T>::set_gain_lookup_table(T *op_cond_arr_, T *Kp_arr_, T *Ki_arr_, T
*Kd_arr_, uint16_t len_arr_)
00098 {
00099     op_cond_arr = op_cond_arr_;
00100     Kp_arr = Kp_arr_;
00101     Ki_arr = Ki_arr_;
00102     Kd_arr = Kd_arr_;
00103     len_arr = len_arr_;
00104 }
00105
00106 template <typename T>
00107 void control_system::pid::PID_GS<T>::update_gain(T op_cond_)
00108 {
00109     if (op_cond_ <= op_cond_arr[0])
00110     {
00111         Kp = Kp_arr[0];
00112         Ki = Ki_arr[0];
00113         Kd = Kd_arr[0];
00114     }
00115     else if (op_cond_ >= op_cond_arr[len_arr - 1])
00116     {
00117         Kp = Kp_arr[len_arr - 1];
00118         Ki = Ki_arr[len_arr - 1];
00119         Kd = Kd_arr[len_arr - 1];
00120     }
00121     else
00122     {
00123         uint16_t index_i = 0;
00124
00125         for (uint16_t i = 0; i < len_arr; i++)
00126         {
00127             if (op_cond_ > op_cond_arr[i])
00128             {
00129                 index_i++;
00130             }
00131             else
00132             {
00133                 break;
00134             }
00135         }
00136
00137         Kp = interpolate(op_cond_, op_cond_arr[index_i], Kp_arr[index_i], op_cond_arr[index_i + 1],
Kp_arr[index_i + 1]);
00138         Ki = interpolate(op_cond_, op_cond_arr[index_i], Ki_arr[index_i], op_cond_arr[index_i + 1],
Ki_arr[index_i + 1]);
00139         Kd = interpolate(op_cond_, op_cond_arr[index_i], Kd_arr[index_i], op_cond_arr[index_i + 1],
Kd_arr[index_i + 1]);
00140     }
00141 }
00142
00143 template <typename T>
00144 void control_system::pid::PID_GS<T>::set_dt(T dt_)
00145 {
00146     dt = dt_;
00147     lpf.set_param(fc, dt);
00148 }
00149
00150 template <typename T>
00151 void control_system::pid::PID_GS<T>::set_Kp(T Kp_)
00152 {
00153     Kp = Kp_;
00154 }
00155
00156 template <typename T>
00157 void control_system::pid::PID_GS<T>::set_Ki(T Ki_)
00158 {
00159     Ki = Ki_;
00160 }
00161
00162 template <typename T>
00163 void control_system::pid::PID_GS<T>::set_Kd(T Kd_)
00164 {
00165     Kd = Kd_;
00166 }
00167
00168 template <typename T>
00169 void control_system::pid::PID_GS<T>::set_I_max(T I_max_)
00170 {
00171     I_max = I_max_;
00172 }
00173
00174 template <typename T>

```

```

00175 void control_system::pid::PID_GS<T>::set_u_max(T u_max_)
00176 {
00177     u_max = u_max_;
00178 }
00179
00180 template <typename T>
00181 void control_system::pid::PID_GS<T>::set_d_filter(bool d_filter_)
00182 {
00183     d_filter = d_filter_;
00184 }
00185
00186 template <typename T>
00187 void control_system::pid::PID_GS<T>::set_fc(T fc_)
00188 {
00189     fc = fc_;
00190     lpf.set_param(fc, dt);
00191 }
00192
00193 template <typename T>
00194 void control_system::pid::PID_GS<T>::set_ff(T u_ff_)
00195 {
00196     u_ff = u_ff_;
00197 }
00198
00199 template <typename T>
00200 T control_system::pid::PID_GS<T>::get_dt()
00201 {
00202     return dt;
00203 }
00204
00205 template <typename T>
00206 T control_system::pid::PID_GS<T>::get_Kp()
00207 {
00208     return Kp;
00209 }
00210
00211 template <typename T>
00212 T control_system::pid::PID_GS<T>::get_Ki()
00213 {
00214     return Ki;
00215 }
00216
00217 template <typename T>
00218 T control_system::pid::PID_GS<T>::get_Kd()
00219 {
00220     return Kd;
00221 }
00222
00223 template <typename T>
00224 T control_system::pid::PID_GS<T>::get_I_max()
00225 {
00226     return I_max;
00227 }
00228
00229 template <typename T>
00230 T control_system::pid::PID_GS<T>::get_u_max()
00231 {
00232     return u_max;
00233 }
00234
00235 template <typename T>
00236 bool control_system::pid::PID_GS<T>::get_d_filter()
00237 {
00238     return d_filter;
00239 }
00240
00241 template <typename T>
00242 T control_system::pid::PID_GS<T>::get_fc()
00243 {
00244     return fc;
00245 }
00246
00247 template <typename T>
00248 T control_system::pid::PID_GS<T>::get_ff()
00249 {
00250     return u_ff;
00251 }
00252
00253 template <typename T>
00254 T control_system::pid::PID_GS<T>::get_P()
00255 {
00256     return P;
00257 }
00258
00259 template <typename T>
00260 T control_system::pid::PID_GS<T>::get_I()
00261 {

```

```

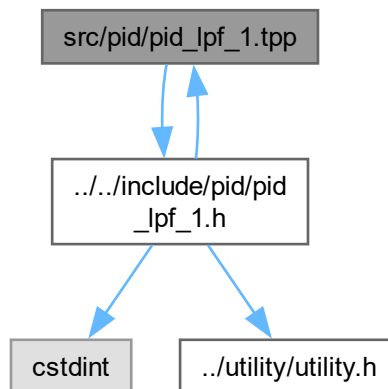
00262     return I;
00263 }
00264
00265 template <typename T>
00266 T control_system::pid::PID_GS<T>::get_D()
00267 {
00268     return D;
00269 }
00270
00271 template <typename T>
00272 T control_system::pid::PID_GS<T>::get_u()
00273 {
00274     return u;
00275 }
00276
00277 template <typename T>
00278 T control_system::pid::PID_GS<T>::get_e_k_1()
00279 {
00280     return e_k_1;
00281 }
00282
00283 template <typename T>
00284 T control_system::pid::PID_GS<T>::interpolate(T x, T x1, T y1, T x2, T y2)
00285 {
00286     return y1 + (x - x1) * (y2 - y1) / (x2 - x1);
00287 }

```

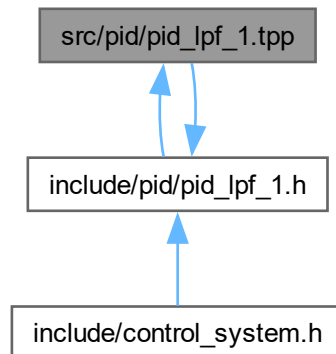
10.49 src/pid/pid_lpf_1.tpp File Reference

```
#include "../..//include/pid/pid_lpf_1.h"
```

Include dependency graph for pid_lpf_1.tpp:



This graph shows which files directly or indirectly include this file:



10.50 pid_lpf_1.tpp

[Go to the documentation of this file.](#)

```

00001 #include "../include/pid/pid_lpf_1.h"
00002
00003 template <typename T>
00004 control_system::pid::PID_LPF_1<T>::PID_LPF_1()
00005 {
00006     dt = 0.0;
00007     Kp = 0.0;
00008     Ki = 0.0;
00009     Kd = 0.0;
00010     u_k_1 = 0.0;
00011     u_k_2 = 0.0;
00012     e_k_1 = 0.0;
00013     e_k_2 = 0.0;
00014     start = 0;
00015 }
00016
00017 template <typename T>
00018 void control_system::pid::PID_LPF_1<T>::init(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_, T
u_max_)
00019 {
00020     set_param(dt_, Kp_, Ki_, Kd_, fc_, u_k_1_, u_k_2_, u_max_);
00021     start = 0;
00022 }
00023
00024 template <typename T>
00025 void control_system::pid::PID_LPF_1<T>::set_param(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T
u_k_2_, T u_max_)
00026 {
00027     dt = dt_;
00028     Kp = Kp_;
00029     Ki = Ki_;
00030     Kd = Kd_;
00031     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00032     u_k_1 = u_k_1_;
00033     u_k_2 = u_k_2_;
00034     u_max = u_max_;
00035 }
00036
00037 template <typename T>
00038 T control_system::pid::PID_LPF_1<T>::update(T x_0, T x)
00039 {
00040     T e_k = x_0 - x;
00041     T u_k = 0.0;
00042     if ((start == 0) || (start == 1))
00043     {
00044         start++;
    
```

```

00045         u_k = u_k_1 + Kp * e_k;
00046     }
00047     else
00048     {
00049         u_k = (-tau * u_k_2 + (2.0 * tau + dt) * u_k_1 + ((Kp + Ki * dt) * (dt + tau) + Kd) * e_k -
(Kp * (2.0 * tau + dt) + Ki * dt * tau + 2.0 * Kd) * e_k_1 + (tau + Kd) * e_k_2) / (dt + tau);
00050     }
00051     u_k = saturate(u_k, -u_max, u_max);
00052     u_k_2 = u_k_1;
00053     u_k_1 = u_k;
00054     e_k_2 = e_k_1;
00055     e_k_1 = e_k;
00056     return u_k;
00057 }
00058
00059 template <typename T>
00060 void control_system::pid::PID_LPF_1<T>::reset ()
00061 {
00062     u_k_1 = 0.0;
00063     u_k_2 = 0.0;
00064     e_k_1 = 0.0;
00065     e_k_2 = 0.0;
00066     start = 0;
00067 }
00068
00069 template <typename T>
00070 void control_system::pid::PID_LPF_1<T>::merge(T u_k_2_)
00071 {
00072     u_k_2 = u_k_2_;
00073 }
00074
00075 template <typename T>
00076 void control_system::pid::PID_LPF_1<T>::set_dt(T dt_)
00077 {
00078     dt = dt_;
00079 }
00080
00081 template <typename T>
00082 void control_system::pid::PID_LPF_1<T>::set_Kp(T Kp_)
00083 {
00084     Kp = Kp_;
00085 }
00086
00087 template <typename T>
00088 void control_system::pid::PID_LPF_1<T>::set_Ki(T Ki_)
00089 {
00090     Ki = Ki_;
00091 }
00092
00093 template <typename T>
00094 void control_system::pid::PID_LPF_1<T>::set_Kd(T Kd_)
00095 {
00096     Kd = Kd_;
00097 }
00098
00099 template <typename T>
00100 void control_system::pid::PID_LPF_1<T>::set_fc(T fc_)
00101 {
00102     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00103 }
00104
00105 template <typename T>
00106 void control_system::pid::PID_LPF_1<T>::set_u_0(T u_k_1_)
00107 {
00108     u_k_1 = u_k_1_;
00109 }
00110
00111 template <typename T>
00112 void control_system::pid::PID_LPF_1<T>::set_u_max(T u_max_)
00113 {
00114     u_max = u_max_;
00115 }
00116
00117 template <typename T>
00118 T control_system::pid::PID_LPF_1<T>::get_dt ()
00119 {
00120     return dt;
00121 }
00122
00123 template <typename T>
00124 T control_system::pid::PID_LPF_1<T>::get_Kp ()
00125 {
00126     return Kp;
00127 }
00128
00129 template <typename T>
00130 T control_system::pid::PID_LPF_1<T>::get_Ki ()

```

```

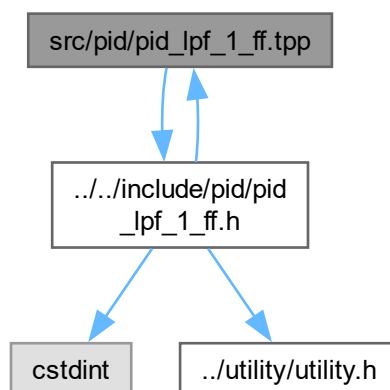
00131 {
00132     return Ki;
00133 }
00134
00135 template <typename T>
00136 T control_system::pid::PID_LPF_1<T>::get_Kd()
00137 {
00138     return Kd;
00139 }
00140
00141 template <typename T>
00142 T control_system::pid::PID_LPF_1<T>::get_fc()
00143 {
00144     return 1.0 / (6.28318530717958647692528676655900576 * tau);
00145 }
00146
00147 template <typename T>
00148 T control_system::pid::PID_LPF_1<T>::get_u_k_1()
00149 {
00150     return u_k_1;
00151 }
00152
00153 template <typename T>
00154 T control_system::pid::PID_LPF_1<T>::get_e_k_1()
00155 {
00156     return e_k_1;
00157 }
00158
00159 template <typename T>
00160 T control_system::pid::PID_LPF_1<T>::get_e_k_2()
00161 {
00162     return e_k_2;
00163 }
00164
00165 template <typename T>
00166 T control_system::pid::PID_LPF_1<T>::get_u_max()
00167 {
00168     return u_max;
00169 }

```

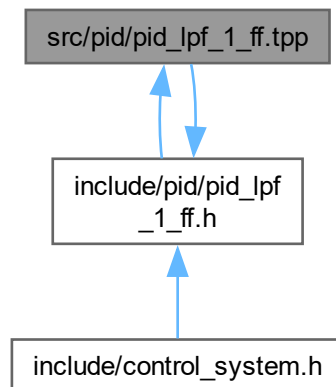
10.51 src/pid/pid_lpf_1_ff.tpp File Reference

```
#include "../..../include/pid/pid_lpf_1_ff.h"
```

Include dependency graph for pid_lpf_1_ff.tpp:



This graph shows which files directly or indirectly include this file:



10.52 pid_lpf_1_ff.tpp

[Go to the documentation of this file.](#)

```

00001 #include "../include/pid/pid_lpf_1_ff.h"
00002
00003 template <typename T>
00004 control_system::pid::PID_LPF_1_FF<T>::PID_LPF_1_FF()
00005 {
00006     dt = 0.0;
00007     Kp = 0.0;
00008     Ki = 0.0;
00009     Kd = 0.0;
00010     u_k_1 = 0.0;
00011     u_k_2 = 0.0;
00012     e_k_1 = 0.0;
00013     e_k_2 = 0.0;
00014     u_ff = 0.0;
00015     start = 0;
00016 }
00017
00018 template <typename T>
00019 void control_system::pid::PID_LPF_1_FF<T>::init(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T u_k_2_,
T u_max_)
00020 {
00021     set_param(dt_, Kp_, Ki_, Kd_, fc_, u_k_1_, u_k_2_, u_max_);
00022     start = 0;
00023 }
00024
00025 template <typename T>
00026 void control_system::pid::PID_LPF_1_FF<T>::set_param(T dt_, T Kp_, T Ki_, T Kd_, T fc_, T u_k_1_, T
u_k_2_, T u_max_)
00027 {
00028     dt = dt_;
00029     Kp = Kp_;
00030     Ki = Ki_;
00031     Kd = Kd_;
00032     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00033     u_k_1 = u_k_1_;
00034     u_k_2 = u_k_2_;
00035     u_max = u_max_;
00036 }
00037
00038 template <typename T>
00039 T control_system::pid::PID_LPF_1_FF<T>::update(T x_0, T x, T u_ff_)
00040 {
00041     T e_k = x_0 - x;
00042     u_ff = u_ff_;
00043     T u_k = 0.0;
  
```



```

00044     if ((start == 0) || (start == 1))
00045     {
00046         start++;
00047         u_k = u_ff + u_k_1 + Kp * e_k;
00048     }
00049     else
00050     {
00051         u_k = u_ff + (-tau * u_k_2 + (2.0 * tau + dt) * u_k_1 + ((Kp + Ki * dt) * (dt + tau) + Kd) *
e_k - (Kp * (2.0 * tau + dt) + Ki * dt * tau + 2.0 * Kd) * e_k_1 + (tau + Kd) * e_k_2) / (dt + tau);
00052     }
00053     u_k = saturate(u_k, -u_max, u_max);
00054     u_k_2 = u_k_1;
00055     u_k_1 = u_k;
00056     e_k_2 = e_k_1;
00057     e_k_1 = e_k;
00058     return u_k;
00059 }
00060
00061 template <typename T>
00062 void control_system::pid::PID_LPF_1_FF<T>::reset()
00063 {
00064     u_k_1 = 0.0;
00065     u_k_2 = 0.0;
00066     e_k_1 = 0.0;
00067     e_k_2 = 0.0;
00068     u_ff = 0.0;
00069     start = 0;
00070 }
00071
00072 template <typename T>
00073 void control_system::pid::PID_LPF_1_FF<T>::merge(T u_k_2_)
00074 {
00075     u_k_2 = u_k_2_;
00076 }
00077
00078 template <typename T>
00079 void control_system::pid::PID_LPF_1_FF<T>::set_dt(T dt_)
00080 {
00081     dt = dt_;
00082 }
00083
00084 template <typename T>
00085 void control_system::pid::PID_LPF_1_FF<T>::set_Kp(T Kp_)
00086 {
00087     Kp = Kp_;
00088 }
00089
00090 template <typename T>
00091 void control_system::pid::PID_LPF_1_FF<T>::set_Ki(T Ki_)
00092 {
00093     Ki = Ki_;
00094 }
00095
00096 template <typename T>
00097 void control_system::pid::PID_LPF_1_FF<T>::set_Kd(T Kd_)
00098 {
00099     Kd = Kd_;
00100 }
00101
00102 template <typename T>
00103 void control_system::pid::PID_LPF_1_FF<T>::set_fc(T fc_)
00104 {
00105     tau = 1.0 / (6.28318530717958647692528676655900576 * fc_);
00106 }
00107
00108 template <typename T>
00109 void control_system::pid::PID_LPF_1_FF<T>::set_ff(T u_ff_)
00110 {
00111     u_ff = u_ff_;
00112 }
00113
00114 template <typename T>
00115 void control_system::pid::PID_LPF_1_FF<T>::set_u_0(T u_k_1_)
00116 {
00117     u_k_1 = u_k_1_;
00118 }
00119
00120 template <typename T>
00121 void control_system::pid::PID_LPF_1_FF<T>::set_u_max(T u_max_)
00122 {
00123     u_max = u_max_;
00124 }
00125
00126 template <typename T>
00127 T control_system::pid::PID_LPF_1_FF<T>::get_dt()
00128 {
00129     return dt;

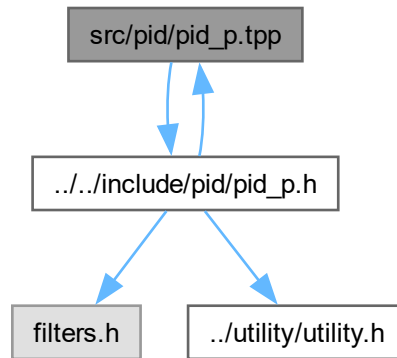
```

```
00130 }
00131
00132 template <typename T>
00133 T control_system::pid::PID_LPF_1_FF<T>::get_Kp()
00134 {
00135     return Kp;
00136 }
00137
00138 template <typename T>
00139 T control_system::pid::PID_LPF_1_FF<T>::get_Ki()
00140 {
00141     return Ki;
00142 }
00143
00144 template <typename T>
00145 T control_system::pid::PID_LPF_1_FF<T>::get_Kd()
00146 {
00147     return Kd;
00148 }
00149
00150 template <typename T>
00151 T control_system::pid::PID_LPF_1_FF<T>::get_fc()
00152 {
00153     return 1.0 / (6.28318530717958647692528676655900576 * tau);
00154 }
00155
00156 template <typename T>
00157 T control_system::pid::PID_LPF_1_FF<T>::get_ff()
00158 {
00159     return u_ff;
00160 }
00161
00162 template <typename T>
00163 T control_system::pid::PID_LPF_1_FF<T>::get_u_k_1()
00164 {
00165     return u_k_1;
00166 }
00167
00168 template <typename T>
00169 T control_system::pid::PID_LPF_1_FF<T>::get_e_k_1()
00170 {
00171     return e_k_1;
00172 }
00173
00174 template <typename T>
00175 T control_system::pid::PID_LPF_1_FF<T>::get_e_k_2()
00176 {
00177     return e_k_2;
00178 }
00179
00180 template <typename T>
00181 T control_system::pid::PID_LPF_1_FF<T>::get_u_max()
00182 {
00183     return u_max;
00184 }
```

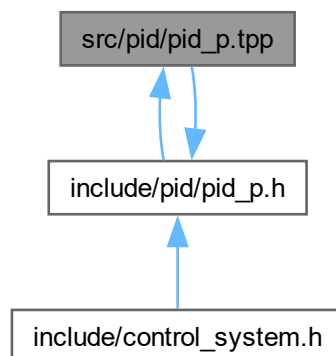
10.53 src/pid/pid_p.hpp File Reference

```
#include "../..//include/pid/pid_p.h"
```

Include dependency graph for pid_p.hpp:



This graph shows which files directly or indirectly include this file:



10.54 pid_p.hpp

[Go to the documentation of this file.](#)

```

00001 #include "../..//include/pid/pid_p.h"
00002
00003 template <typename T>
00004 control_system::pid::PID_P<T>::PID_P()
00005 {
00006     dt = 0.0;

```

```

00007     Kp = 0.0;
00008     Ki = 0.0;
00009     Kd = 0.0;
00010     fc = 0.0;
00011     I_max = 0.0;
00012     u_max = 0.0;
00013     d_filter = false;
00014     e_k_1 = 0.0;
00015     P = 0.0;
00016     I = 0.0;
00017     D = 0.0;
00018     u = 0.0;
00019     u_ff = 0.0;
00020     start = true;
00021 }
00022
00023 template <typename T>
00024 void control_system::pid::PID_P<T>::init(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool
    d_filter_, T fc_)
00025 {
00026     set_param(dt_, Kp_, Ki_, Kd_, I_max_, u_max_, d_filter_, fc_);
00027     start = true;
00028 }
00029
00030 template <typename T>
00031 void control_system::pid::PID_P<T>::set_param(T dt_, T Kp_, T Ki_, T Kd_, T I_max_, T u_max_, bool
    d_filter_, T fc_)
00032 {
00033     dt = dt_;
00034     Kp = Kp_;
00035     Ki = Ki_;
00036     Kd = Kd_;
00037     I_max = I_max_;
00038     u_max = u_max_;
00039     d_filter = d_filter_;
00040     fc = fc_;
00041     lpf.set_param(fc, dt);
00042 }
00043
00044 template <typename T>
00045 T control_system::pid::PID_P<T>::update(T x_0, T x, T u_ff_)
00046 {
00047     T e_k = x_0 - x;
00048     u_ff = u_ff_;
00049
00050     P = Kp * e_k;
00051
00052     I = I + Ki * e_k * dt;
00053     I = saturate(I, -I_max, I_max);
00054
00055     if (start == true)
00056     {
00057         D = 0.0;
00058         start = false;
00059     }
00060     else
00061     {
00062         D = Kd * (e_k - e_k_1) / dt;
00063     }
00064
00065     if (d_filter == true)
00066     {
00067         lpf.cal_y(D);
00068         D = lpf.get_y();
00069     }
00070     e_k_1 = e_k;
00071     u = u_ff + P + I + D;
00072     u = saturate(u, -u_max, u_max);
00073     return u;
00074 }
00075
00076 template <typename T>
00077 void control_system::pid::PID_P<T>::reset()
00078 {
00079     e_k_1 = 0.0;
00080     P = 0.0;
00081     I = 0.0;
00082     D = 0.0;
00083     u = 0.0;
00084     u_ff = 0.0;
00085     start = true;
00086     lpf.reset();
00087 }
00088
00089 template <typename T>
00090 void control_system::pid::PID_P<T>::merge(T u_k_1_)
00091 {

```

```

00092 }
00093
00094 template <typename T>
00095 void control_system::pid::PID_P<T>::set_dt(T dt_)
00096 {
00097     dt = dt_;
00098     lpf.set_param(fc, dt);
00099 }
00100
00101 template <typename T>
00102 void control_system::pid::PID_P<T>::set_Kp(T Kp_)
00103 {
00104     Kp = Kp_;
00105 }
00106
00107 template <typename T>
00108 void control_system::pid::PID_P<T>::set_Ki(T Ki_)
00109 {
00110     Ki = Ki_;
00111 }
00112
00113 template <typename T>
00114 void control_system::pid::PID_P<T>::set_Kd(T Kd_)
00115 {
00116     Kd = Kd_;
00117 }
00118
00119 template <typename T>
00120 void control_system::pid::PID_P<T>::set_I_max(T I_max_)
00121 {
00122     I_max = I_max_;
00123 }
00124
00125 template <typename T>
00126 void control_system::pid::PID_P<T>::set_u_max(T u_max_)
00127 {
00128     u_max = u_max_;
00129 }
00130
00131 template <typename T>
00132 void control_system::pid::PID_P<T>::set_d_filter(bool d_filter_)
00133 {
00134     d_filter = d_filter_;
00135 }
00136
00137 template <typename T>
00138 void control_system::pid::PID_P<T>::set_fc(T fc_)
00139 {
00140     fc = fc_;
00141     lpf.set_param(fc, dt);
00142 }
00143
00144 template <typename T>
00145 void control_system::pid::PID_P<T>::set_ff(T u_ff_)
00146 {
00147     u_ff = u_ff_;
00148 }
00149
00150 template <typename T>
00151 T control_system::pid::PID_P<T>::get_dt()
00152 {
00153     return dt;
00154 }
00155
00156 template <typename T>
00157 T control_system::pid::PID_P<T>::get_Kp()
00158 {
00159     return Kp;
00160 }
00161
00162 template <typename T>
00163 T control_system::pid::PID_P<T>::get_Ki()
00164 {
00165     return Ki;
00166 }
00167
00168 template <typename T>
00169 T control_system::pid::PID_P<T>::get_Kd()
00170 {
00171     return Kd;
00172 }
00173
00174 template <typename T>
00175 T control_system::pid::PID_P<T>::get_I_max()
00176 {
00177     return I_max;
00178 }

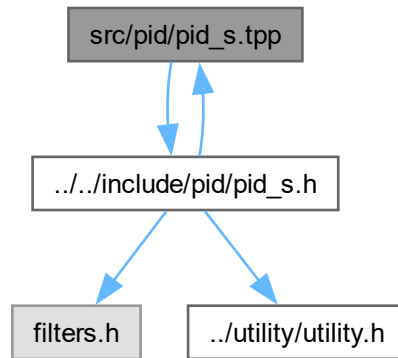
```

```
00179
00180 template <typename T>
00181 T control_system::pid::PID_P<T>::get_u_max()
00182 {
00183     return u_max;
00184 }
00185
00186 template <typename T>
00187 bool control_system::pid::PID_P<T>::get_d_filter()
00188 {
00189     return d_filter;
00190 }
00191
00192 template <typename T>
00193 T control_system::pid::PID_P<T>::get_fc()
00194 {
00195     return fc;
00196 }
00197
00198 template <typename T>
00199 T control_system::pid::PID_P<T>::get_ff()
00200 {
00201     return u_ff;
00202 }
00203
00204 template <typename T>
00205 T control_system::pid::PID_P<T>::get_P()
00206 {
00207     return P;
00208 }
00209
00210 template <typename T>
00211 T control_system::pid::PID_P<T>::get_I()
00212 {
00213     return I;
00214 }
00215
00216 template <typename T>
00217 T control_system::pid::PID_P<T>::get_D()
00218 {
00219     return D;
00220 }
00221
00222 template <typename T>
00223 T control_system::pid::PID_P<T>::get_u()
00224 {
00225     return u;
00226 }
00227
00228 template <typename T>
00229 T control_system::pid::PID_P<T>::get_e_k_1()
00230 {
00231     return e_k_1;
00232 }
```

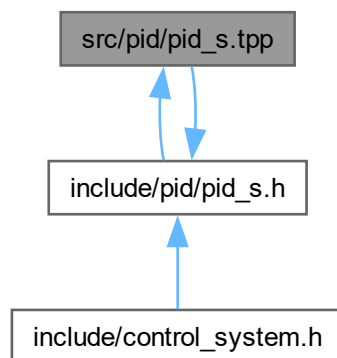
10.55 src/pid/pid_s.hpp File Reference

```
#include "../..//include/pid/pid_s.h"
```

Include dependency graph for pid_s.hpp:



This graph shows which files directly or indirectly include this file:



10.56 pid_s.hpp

[Go to the documentation of this file.](#)

```

00001 #include "../..//include/pid/pid_s.h"
00002
00003 template <typename T>
00004 control_system::pid::PID_S<T>::PID_S()
00005 {
00006     dt = 0.0;

```

```

00007     Kp = 0.0;
00008     T_i = 0.0;
00009     T_d = 0.0;
00010     fc = 0.0;
00011     I_max = 0.0;
00012     u_max = 0.0;
00013     d_filter = false;
00014     e_k_1 = 0.0;
00015     P = 0.0;
00016     I = 0.0;
00017     D = 0.0;
00018     u = 0.0;
00019     u_ff = 0.0;
00020     start = true;
00021 }
00022
00023 template <typename T>
00024 void control_system::pid::PID_S<T>::init(T dt_, T Kp_, T T_i_, T T_d_, T I_max_, T u_max_, bool
    d_filter_, T fc_)
00025 {
00026     set_param(dt_, Kp_, T_i_, T_d_, I_max_, u_max_, d_filter_, fc_);
00027     start = true;
00028 }
00029
00030 template <typename T>
00031 void control_system::pid::PID_S<T>::set_param(T dt_, T Kp_, T T_i_, T T_d_, T I_max_, T u_max_, bool
    d_filter_, T fc_)
00032 {
00033     dt = dt_;
00034     Kp = Kp_;
00035     T_i = T_i_;
00036     T_d = T_d_;
00037     I_max = I_max_;
00038     u_max = u_max_;
00039     d_filter = d_filter_;
00040     fc = fc_;
00041     lpf.set_param(fc, dt);
00042 }
00043
00044 template <typename T>
00045 T control_system::pid::PID_S<T>::update(T x_0, T x, T u_ff_)
00046 {
00047     T e_k = x_0 - x;
00048     u_ff = u_ff_;
00049
00050     P = Kp * e_k;
00051
00052     I = I + (Kp / T_i) * e_k * dt;
00053     I = saturate(I, -I_max, I_max);
00054
00055     if (start == true)
00056     {
00057         D = 0.0;
00058         start = false;
00059     }
00060     else
00061     {
00062         D = Kp * T_d * (e_k - e_k_1) / dt;
00063     }
00064
00065     if (d_filter == true)
00066     {
00067         lpf.cal_y(D);
00068         D = lpf.get_y();
00069     }
00070     e_k_1 = e_k;
00071     u = u_ff + P + I + D;
00072     u = saturate(u, -u_max, u_max);
00073     return u;
00074 }
00075
00076 template <typename T>
00077 void control_system::pid::PID_S<T>::reset()
00078 {
00079     e_k_1 = 0.0;
00080     P = 0.0;
00081     I = 0.0;
00082     D = 0.0;
00083     u = 0.0;
00084     u_ff = 0.0;
00085     start = true;
00086     lpf.reset();
00087 }
00088
00089 template <typename T>
00090 void control_system::pid::PID_S<T>::merge(T u_k_1_)
00091 {

```



```

00092 }
00093
00094 template <typename T>
00095 void control_system::pid::PID_S<T>::set_dt(T dt_)
00096 {
00097     dt = dt_;
00098     lpf.set_param(fc, dt);
00099 }
00100
00101 template <typename T>
00102 void control_system::pid::PID_S<T>::set_Kp(T Kp_)
00103 {
00104     Kp = Kp_;
00105 }
00106
00107 template <typename T>
00108 void control_system::pid::PID_S<T>::set_Ti(T Ti_)
00109 {
00110     Ti = Ti_;
00111 }
00112
00113 template <typename T>
00114 void control_system::pid::PID_S<T>::set_Td(T Td_)
00115 {
00116     Td = Td_;
00117 }
00118
00119 template <typename T>
00120 void control_system::pid::PID_S<T>::set_I_max(T I_max_)
00121 {
00122     I_max = I_max_;
00123 }
00124
00125 template <typename T>
00126 void control_system::pid::PID_S<T>::set_u_max(T u_max_)
00127 {
00128     u_max = u_max_;
00129 }
00130
00131 template <typename T>
00132 void control_system::pid::PID_S<T>::set_d_filter(bool d_filter_)
00133 {
00134     d_filter = d_filter_;
00135 }
00136
00137 template <typename T>
00138 void control_system::pid::PID_S<T>::set_fc(T fc_)
00139 {
00140     fc = fc_;
00141     lpf.set_param(fc, dt);
00142 }
00143
00144 template <typename T>
00145 void control_system::pid::PID_S<T>::set_ff(T u_ff_)
00146 {
00147     u_ff = u_ff_;
00148 }
00149
00150 template <typename T>
00151 T control_system::pid::PID_S<T>::get_dt()
00152 {
00153     return dt;
00154 }
00155
00156 template <typename T>
00157 T control_system::pid::PID_S<T>::get_Kp()
00158 {
00159     return Kp;
00160 }
00161
00162 template <typename T>
00163 T control_system::pid::PID_S<T>::get_Ti()
00164 {
00165     return Ti;
00166 }
00167
00168 template <typename T>
00169 T control_system::pid::PID_S<T>::get_Td()
00170 {
00171     return Td;
00172 }
00173
00174 template <typename T>
00175 T control_system::pid::PID_S<T>::get_I_max()
00176 {
00177     return I_max;
00178 }

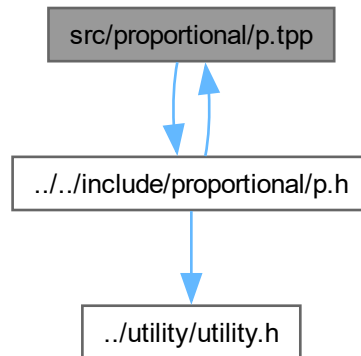
```

```
00179
00180 template <typename T>
00181 T control_system::pid::PID_S<T>::get_u_max()
00182 {
00183     return u_max;
00184 }
00185
00186 template <typename T>
00187 bool control_system::pid::PID_S<T>::get_d_filter()
00188 {
00189     return d_filter;
00190 }
00191
00192 template <typename T>
00193 T control_system::pid::PID_S<T>::get_fc()
00194 {
00195     return fc;
00196 }
00197
00198 template <typename T>
00199 T control_system::pid::PID_S<T>::get_ff()
00200 {
00201     return u_ff;
00202 }
00203
00204 template <typename T>
00205 T control_system::pid::PID_S<T>::get_P()
00206 {
00207     return P;
00208 }
00209
00210 template <typename T>
00211 T control_system::pid::PID_S<T>::get_I()
00212 {
00213     return I;
00214 }
00215
00216 template <typename T>
00217 T control_system::pid::PID_S<T>::get_D()
00218 {
00219     return D;
00220 }
00221
00222 template <typename T>
00223 T control_system::pid::PID_S<T>::get_u()
00224 {
00225     return u;
00226 }
00227
00228 template <typename T>
00229 T control_system::pid::PID_S<T>::get_e_k_1()
00230 {
00231     return e_k_1;
00232 }
```

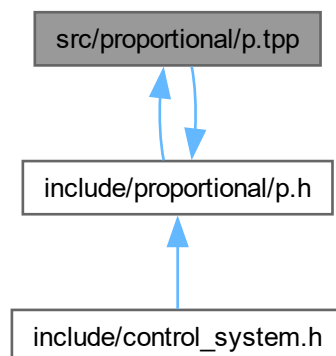
10.57 src/proportional/p.hpp File Reference

```
#include "../..//include/proportional/p.h"
```

Include dependency graph for p.hpp:



This graph shows which files directly or indirectly include this file:



10.58 p.hpp

[Go to the documentation of this file.](#)

```
00001 #include "../..//include/proportional/p.h"
00002
00003 template <typename T>
00004 control_system::proportional::P<T>::P ()
00005 {
00006     Kp = 0.0;
```

```
00007 }
00008
00009 template <typename T>
00010 void control_system::proportional::P<T>::init(T Kp_, T u_max_)
00011 {
00012     set_param(Kp_, u_max_);
00013 }
00014
00015 template <typename T>
00016 void control_system::proportional::P<T>::set_param(T Kp_, T u_max_)
00017 {
00018     Kp = Kp_;
00019     u_max = u_max_;
00020 }
00021
00022 template <typename T>
00023 T control_system::proportional::P<T>::update(T x_0, T x)
00024 {
00025     return saturate(Kp * (x_0 - x), -u_max, u_max);
00026 }
00027
00028 template <typename T>
00029 void control_system::proportional::P<T>::reset()
00030 {
00031 }
00032
00033 template <typename T>
00034 void control_system::proportional::P<T>::set_Kp(T Kp_)
00035 {
00036     Kp = Kp_;
00037 }
00038
00039 template <typename T>
00040 void control_system::proportional::P<T>::set_u_max(T u_max_)
00041 {
00042     u_max = u_max_;
00043 }
00044
00045 template <typename T>
00046 T control_system::proportional::P<T>::get_Kp()
00047 {
00048     return Kp;
00049 }
00050
00051 template <typename T>
00052 T control_system::proportional::P<T>::get_u_max()
00053 {
00054     return u_max;
00055 }
```

Index

control_system, [25](#)
control_system::derivative, [25](#)
control_system::derivative::D< T >, [29](#)
 D, [30](#)
 dt, [34](#)
 e_k_1, [34](#)
 get_dt, [31](#)
 get_e_k_1, [31](#)
 get_Kd, [31](#)
 get_u_max, [31](#)
 init, [31](#)
 Kd, [35](#)
 merge, [32](#)
 reset, [32](#)
 set_dt, [32](#)
 set_Kd, [33](#)
 set_param, [33](#)
 set_u_max, [33](#)
 start, [35](#)
 u_max, [35](#)
 update, [34](#)
control_system::derivative::D_LPF_1< T >, [36](#)
 D_LPF_1, [38](#)
 dt, [43](#)
 e_k_1, [43](#)
 get_dt, [38](#)
 get_e_k_1, [38](#)
 get_fc, [38](#)
 get_Kd, [39](#)
 get_u_k_1, [39](#)
 get_u_max, [39](#)
 init, [39](#)
 Kd, [43](#)
 merge, [40](#)
 reset, [40](#)
 set_dt, [40](#)
 set_fc, [41](#)
 set_Kd, [41](#)
 set_param, [42](#)
 set_u_max, [42](#)
 start, [44](#)
 tau, [44](#)
 u_k_1, [44](#)
 u_max, [44](#)
 update, [43](#)
control_system::integral, [25](#)
control_system::integral::I< T >, [45](#)
 dt, [51](#)
 get_dt, [46](#)
 get_Ki, [46](#)
 get_u_k_1, [47](#)
 get_u_max, [47](#)
 I, [46](#)
 init, [47](#)
 Ki, [51](#)
 merge, [48](#)
 reset, [48](#)
 set_dt, [48](#)
 set_Ki, [49](#)
 set_param, [49](#)
 set_u_0, [50](#)
 set_u_max, [50](#)
 u_k_1, [51](#)
 u_max, [52](#)
 update, [50](#)
control_system::pd, [26](#)
control_system::pd::PD< T >, [52](#)
 dt, [59](#)
 e_k_1, [59](#)
 get_dt, [54](#)
 get_e_k_1, [54](#)
 get_Kd, [55](#)
 get_Kp, [55](#)
 get_u_max, [55](#)
 init, [55](#)
 Kd, [59](#)
 Kp, [60](#)
 merge, [56](#)
 PD, [54](#)
 reset, [56](#)
 set_dt, [56](#)
 set_Kd, [57](#)
 set_Kp, [57](#)
 set_param, [58](#)
 set_u_max, [58](#)
 start, [60](#)
 u_max, [60](#)
 update, [59](#)
control_system::pd::PD_LPF_1< T >, [60](#)
 dt, [69](#)
 e_k_1, [69](#)
 get_dt, [63](#)
 get_e_k_1, [63](#)
 get_fc, [63](#)
 get_Kd, [63](#)
 get_Kp, [63](#)
 get_u_k_1, [64](#)
 get_u_max, [64](#)

- init, [64](#)
- Kd, [69](#)
- Kp, [69](#)
- merge, [65](#)
- PD_LPF_1, [62](#)
- reset, [65](#)
- set_dt, [66](#)
- set_fc, [66](#)
- set_Kd, [66](#)
- set_Kp, [67](#)
- set_param, [67](#)
- set_u_max, [68](#)
- start, [69](#)
- tau, [69](#)
- u_k_1, [70](#)
- u_max, [70](#)
- update, [68](#)
- control_system::pi, [26](#)
- control_system::pi::PI< T >, [70](#)
 - dt, [78](#)
 - e_k_1, [78](#)
 - get_dt, [72](#)
 - get_Ki, [72](#)
 - get_Kp, [73](#)
 - get_u_k_1, [73](#)
 - get_u_max, [73](#)
 - init, [73](#)
 - Ki, [78](#)
 - Kp, [78](#)
 - merge, [74](#)
 - PI, [72](#)
 - reset, [74](#)
 - set_dt, [75](#)
 - set_Ki, [75](#)
 - set_Kp, [75](#)
 - set_param, [76](#)
 - set_u_0, [76](#)
 - set_u_max, [77](#)
 - start, [78](#)
 - u_k_1, [78](#)
 - u_max, [79](#)
 - update, [77](#)
- control_system::pid, [26](#)
- control_system::pid::PID< T >, [79](#)
 - dt, [88](#)
 - e_k_1, [88](#)
 - e_k_2, [88](#)
 - get_dt, [81](#)
 - get_e_k_1, [81](#)
 - get_e_k_2, [82](#)
 - get_Kd, [82](#)
 - get_Ki, [82](#)
 - get_Kp, [82](#)
 - get_u_k_1, [83](#)
 - get_u_max, [83](#)
 - init, [83](#)
 - Kd, [89](#)
 - Ki, [89](#)
 - Kp, [89](#)
 - merge, [84](#)
 - PID, [81](#)
 - reset, [84](#)
 - set_dt, [85](#)
 - set_Kd, [85](#)
 - set_Ki, [85](#)
 - set_Kp, [86](#)
 - set_param, [86](#)
 - set_u_0, [87](#)
 - set_u_max, [87](#)
 - start, [89](#)
 - u_k_1, [89](#)
 - u_max, [90](#)
 - update, [87](#)
- control_system::pid::PID_GS< T >, [90](#)
 - D, [105](#)
 - d_filter, [105](#)
 - dt, [105](#)
 - e_k_1, [105](#)
 - fc, [105](#)
 - get_D, [94](#)
 - get_d_filter, [94](#)
 - get_dt, [94](#)
 - get_e_k_1, [94](#)
 - get_fc, [94](#)
 - get_ff, [95](#)
 - get_I, [95](#)
 - get_I_max, [95](#)
 - get_Kd, [95](#)
 - get_Ki, [96](#)
 - get_Kp, [96](#)
 - get_P, [96](#)
 - get_u, [96](#)
 - get_u_max, [97](#)
 - I, [106](#)
 - I_max, [106](#)
 - init, [97](#)
 - interpolate, [98](#)
 - Kd, [106](#)
 - Kd_arr, [106](#)
 - Ki, [106](#)
 - Ki_arr, [107](#)
 - Kp, [107](#)
 - Kp_arr, [107](#)
 - len_arr, [107](#)
 - lpf, [107](#)
 - merge, [98](#)
 - op_cond_arr, [107](#)
 - P, [108](#)
 - PID_GS, [93](#)
 - reset, [99](#)
 - set_d_filter, [99](#)
 - set_dt, [99](#)
 - set_fc, [100](#)
 - set_ff, [100](#)
 - set_gain_lookup_table, [100](#)
 - set_I_max, [101](#)

- set_Kd, [101](#)
- set_Ki, [102](#)
- set_Kp, [102](#)
- set_param, [102](#)
- set_u_max, [103](#)
- start, [108](#)
- u, [108](#)
- u_ff, [108](#)
- u_max, [108](#)
- update, [104](#)
- update_gain, [104](#)
- control_system::pid::PID_LPF_1< T >, [109](#)
 - dt, [119](#)
 - e_k_1, [119](#)
 - e_k_2, [119](#)
 - get_dt, [112](#)
 - get_e_k_1, [112](#)
 - get_e_k_2, [112](#)
 - get_fc, [112](#)
 - get_Kd, [112](#)
 - get_Ki, [113](#)
 - get_Kp, [113](#)
 - get_u_k_1, [113](#)
 - get_u_max, [113](#)
 - init, [114](#)
 - Kd, [119](#)
 - Ki, [119](#)
 - Kp, [119](#)
 - merge, [114](#)
 - PID_LPF_1, [111](#)
 - reset, [115](#)
 - set_dt, [115](#)
 - set_fc, [115](#)
 - set_Kd, [116](#)
 - set_Ki, [116](#)
 - set_Kp, [116](#)
 - set_param, [117](#)
 - set_u_0, [117](#)
 - set_u_max, [118](#)
 - start, [120](#)
 - tau, [120](#)
 - u_k_1, [120](#)
 - u_k_2, [120](#)
 - u_max, [120](#)
 - update, [118](#)
- control_system::pid::PID_LPF_1_FF< T >, [121](#)
 - dt, [132](#)
 - e_k_1, [132](#)
 - e_k_2, [132](#)
 - get_dt, [124](#)
 - get_e_k_1, [124](#)
 - get_e_k_2, [124](#)
 - get_fc, [124](#)
 - get_ff, [124](#)
 - get_Kd, [125](#)
 - get_Ki, [125](#)
 - get_Kp, [125](#)
 - get_u_k_1, [125](#)
 - get_u_max, [126](#)
 - init, [126](#)
 - Kd, [132](#)
 - Ki, [132](#)
 - Kp, [132](#)
 - merge, [127](#)
 - PID_LPF_1_FF, [123](#)
 - reset, [127](#)
 - set_dt, [127](#)
 - set_fc, [128](#)
 - set_ff, [128](#)
 - set_Kd, [128](#)
 - set_Ki, [129](#)
 - set_Kp, [129](#)
 - set_param, [129](#)
 - set_u_0, [130](#)
 - set_u_max, [131](#)
 - start, [133](#)
 - tau, [133](#)
 - u_ff, [133](#)
 - u_k_1, [133](#)
 - u_k_2, [133](#)
 - u_max, [134](#)
 - update, [131](#)
- control_system::pid::PID_P< T >, [134](#)
 - D, [146](#)
 - d_filter, [146](#)
 - dt, [147](#)
 - e_k_1, [147](#)
 - fc, [147](#)
 - get_D, [137](#)
 - get_d_filter, [137](#)
 - get_dt, [137](#)
 - get_e_k_1, [138](#)
 - get_fc, [138](#)
 - get_ff, [138](#)
 - get_I, [138](#)
 - get_I_max, [139](#)
 - get_Kd, [139](#)
 - get_Ki, [139](#)
 - get_Kp, [139](#)
 - get_P, [140](#)
 - get_u, [140](#)
 - get_u_max, [140](#)
 - I, [147](#)
 - I_max, [147](#)
 - init, [140](#)
 - Kd, [148](#)
 - Ki, [148](#)
 - Kp, [148](#)
 - lpf, [148](#)
 - merge, [141](#)
 - P, [148](#)
 - PID_P, [137](#)
 - reset, [142](#)
 - set_d_filter, [142](#)
 - set_dt, [142](#)
 - set_fc, [142](#)

- set_ff, 143
- set_I_max, 143
- set_Kd, 143
- set_Ki, 144
- set_Kp, 144
- set_param, 144
- set_u_max, 145
- start, 148
- u, 149
- u_ff, 149
- u_max, 149
- update, 146
- control_system::pid::PID_S< T >, 149
 - D, 162
 - d_filter, 162
 - dt, 162
 - e_k_1, 163
 - fc, 163
 - get_D, 153
 - get_d_filter, 153
 - get_dt, 153
 - get_e_k_1, 153
 - get_fc, 153
 - get_ff, 154
 - get_I, 154
 - get_I_max, 154
 - get_Kp, 154
 - get_P, 155
 - get_T_d, 155
 - get_T_i, 155
 - get_u, 155
 - get_u_max, 156
 - I, 163
 - I_max, 163
 - init, 156
 - Kp, 163
 - lpf, 164
 - merge, 157
 - P, 164
 - PID_S, 152
 - reset, 157
 - set_d_filter, 158
 - set_dt, 158
 - set_fc, 158
 - set_ff, 159
 - set_I_max, 159
 - set_Kp, 159
 - set_param, 160
 - set_Td, 160
 - set_Ti, 161
 - set_u_max, 161
 - start, 164
 - T_d, 164
 - T_i, 164
 - u, 165
 - u_ff, 165
 - u_max, 165
 - update, 161
- control_system::proportional, 26
- control_system::proportional::P< T >, 165
 - get_Kp, 167
 - get_u_max, 167
 - init, 167
 - Kp, 170
 - P, 167
 - reset, 168
 - set_Kp, 168
 - set_param, 168
 - set_u_max, 169
 - u_max, 170
 - update, 169
- D
 - control_system::derivative::D< T >, 30
 - control_system::pid::PID_GS< T >, 105
 - control_system::pid::PID_P< T >, 146
 - control_system::pid::PID_S< T >, 162
- d_filter
 - control_system::pid::PID_GS< T >, 105
 - control_system::pid::PID_P< T >, 146
 - control_system::pid::PID_S< T >, 162
- D_LPF_1
 - control_system::derivative::D_LPF_1< T >, 38
- derivative, 26
- doc Directory Reference, 16
- doc/mainpage.dox, 171
- dt
 - control_system::derivative::D< T >, 34
 - control_system::derivative::D_LPF_1< T >, 43
 - control_system::integral::I< T >, 51
 - control_system::pd::PD< T >, 59
 - control_system::pd::PD_LPF_1< T >, 69
 - control_system::pi::PI< T >, 78
 - control_system::pid::PID< T >, 88
 - control_system::pid::PID_GS< T >, 105
 - control_system::pid::PID_LPF_1< T >, 119
 - control_system::pid::PID_LPF_1_FF< T >, 132
 - control_system::pid::PID_P< T >, 147
 - control_system::pid::PID_S< T >, 162
- e_k_1
 - control_system::derivative::D< T >, 34
 - control_system::derivative::D_LPF_1< T >, 43
 - control_system::pd::PD< T >, 59
 - control_system::pd::PD_LPF_1< T >, 69
 - control_system::pi::PI< T >, 78
 - control_system::pid::PID< T >, 88
 - control_system::pid::PID_GS< T >, 105
 - control_system::pid::PID_LPF_1< T >, 119
 - control_system::pid::PID_LPF_1_FF< T >, 132
 - control_system::pid::PID_P< T >, 147
 - control_system::pid::PID_S< T >, 163
- e_k_2
 - control_system::pid::PID< T >, 88
 - control_system::pid::PID_LPF_1< T >, 119
 - control_system::pid::PID_LPF_1_FF< T >, 132

- fc
 - control_system::pid::PID_GS< T >, 105
 - control_system::pid::PID_P< T >, 147
 - control_system::pid::PID_S< T >, 163
- get_D
 - control_system::pid::PID_GS< T >, 94
 - control_system::pid::PID_P< T >, 137
 - control_system::pid::PID_S< T >, 153
- get_d_filter
 - control_system::pid::PID_GS< T >, 94
 - control_system::pid::PID_P< T >, 137
 - control_system::pid::PID_S< T >, 153
- get_dt
 - control_system::derivative::D< T >, 31
 - control_system::derivative::D_LPF_1< T >, 39
 - control_system::integral::I< T >, 46
 - control_system::pd::PD< T >, 54
 - control_system::pd::PD_LPF_1< T >, 63
 - control_system::pi::PI< T >, 72
 - control_system::pid::PID< T >, 81
 - control_system::pid::PID_GS< T >, 94
 - control_system::pid::PID_LPF_1< T >, 112
 - control_system::pid::PID_LPF_1_FF< T >, 124
 - control_system::pid::PID_P< T >, 137
 - control_system::pid::PID_S< T >, 153
- get_e_k_1
 - control_system::derivative::D< T >, 31
 - control_system::derivative::D_LPF_1< T >, 39
 - control_system::pd::PD< T >, 54
 - control_system::pd::PD_LPF_1< T >, 63
 - control_system::pid::PID< T >, 81
 - control_system::pid::PID_GS< T >, 94
 - control_system::pid::PID_LPF_1< T >, 112
 - control_system::pid::PID_LPF_1_FF< T >, 124
 - control_system::pid::PID_P< T >, 138
 - control_system::pid::PID_S< T >, 153
- get_e_k_2
 - control_system::pid::PID< T >, 82
 - control_system::pid::PID_LPF_1< T >, 112
 - control_system::pid::PID_LPF_1_FF< T >, 124
- get_fc
 - control_system::derivative::D_LPF_1< T >, 39
 - control_system::pd::PD_LPF_1< T >, 63
 - control_system::pid::PID_GS< T >, 94
 - control_system::pid::PID_LPF_1< T >, 112
 - control_system::pid::PID_LPF_1_FF< T >, 124
 - control_system::pid::PID_P< T >, 138
 - control_system::pid::PID_S< T >, 153
- get_ff
 - control_system::pid::PID_GS< T >, 95
 - control_system::pid::PID_LPF_1_FF< T >, 124
 - control_system::pid::PID_P< T >, 138
 - control_system::pid::PID_S< T >, 154
- get_I
 - control_system::pid::PID_GS< T >, 95
 - control_system::pid::PID_P< T >, 138
 - control_system::pid::PID_S< T >, 154
- get_I_max
 - control_system::pid::PID_GS< T >, 95
 - control_system::pid::PID_P< T >, 139
 - control_system::pid::PID_S< T >, 154
- get_Kd
 - control_system::derivative::D< T >, 31
 - control_system::derivative::D_LPF_1< T >, 39
 - control_system::pd::PD< T >, 55
 - control_system::pd::PD_LPF_1< T >, 63
 - control_system::pid::PID< T >, 82
 - control_system::pid::PID_GS< T >, 95
 - control_system::pid::PID_LPF_1< T >, 112
 - control_system::pid::PID_LPF_1_FF< T >, 125
 - control_system::pid::PID_P< T >, 139
- get_Ki
 - control_system::integral::I< T >, 46
 - control_system::pi::PI< T >, 72
 - control_system::pid::PID< T >, 82
 - control_system::pid::PID_GS< T >, 96
 - control_system::pid::PID_LPF_1< T >, 113
 - control_system::pid::PID_LPF_1_FF< T >, 125
 - control_system::pid::PID_P< T >, 139
- get_Kp
 - control_system::pd::PD< T >, 55
 - control_system::pd::PD_LPF_1< T >, 63
 - control_system::pi::PI< T >, 73
 - control_system::pid::PID< T >, 82
 - control_system::pid::PID_GS< T >, 96
 - control_system::pid::PID_LPF_1< T >, 113
 - control_system::pid::PID_LPF_1_FF< T >, 125
 - control_system::pid::PID_P< T >, 139
 - control_system::pid::PID_S< T >, 154
 - control_system::proportional::P< T >, 167
- get_P
 - control_system::pid::PID_GS< T >, 96
 - control_system::pid::PID_P< T >, 140
 - control_system::pid::PID_S< T >, 155
- get_T_d
 - control_system::pid::PID_S< T >, 155
- get_T_i
 - control_system::pid::PID_S< T >, 155
- get_u
 - control_system::pid::PID_GS< T >, 96
 - control_system::pid::PID_P< T >, 140
 - control_system::pid::PID_S< T >, 155
- get_u_k_1
 - control_system::derivative::D_LPF_1< T >, 39
 - control_system::integral::I< T >, 47
 - control_system::pd::PD_LPF_1< T >, 64
 - control_system::pi::PI< T >, 73
 - control_system::pid::PID< T >, 83
 - control_system::pid::PID_LPF_1< T >, 113
 - control_system::pid::PID_LPF_1_FF< T >, 125
- get_u_max
 - control_system::derivative::D< T >, 31
 - control_system::derivative::D_LPF_1< T >, 39
 - control_system::integral::I< T >, 47
 - control_system::pd::PD< T >, 55
 - control_system::pd::PD_LPF_1< T >, 64

- control_system::pi::PI< T >, 73
 - control_system::pid::PID< T >, 83
 - control_system::pid::PID_GS< T >, 97
 - control_system::pid::PID_LPF_1< T >, 113
 - control_system::pid::PID_LPF_1_FF< T >, 126
 - control_system::pid::PID_P< T >, 140
 - control_system::pid::PID_S< T >, 156
 - control_system::proportional::P< T >, 167
- I
- control_system::integral::I< T >, 46
 - control_system::pid::PID_GS< T >, 106
 - control_system::pid::PID_P< T >, 147
 - control_system::pid::PID_S< T >, 163
- I_max
- control_system::pid::PID_GS< T >, 106
 - control_system::pid::PID_P< T >, 147
 - control_system::pid::PID_S< T >, 163
- include Directory Reference, 16
- include/control_system.h, 171, 172
- include/derivative Directory Reference, 15
- include/derivative/d.h, 173, 175
- include/derivative/d_lpf_1.h, 175, 177
- include/integral Directory Reference, 17
- include/integral/i.h, 178, 180
- include/pd Directory Reference, 18
- include/pd/pd.h, 181, 182
- include/pd/pd_lpf_1.h, 183, 185
- include/pi Directory Reference, 19
- include/pi/pi.h, 186, 188
- include/pid Directory Reference, 20
- include/pid/pid.h, 188, 190
- include/pid/pid_gs.h, 191, 193
- include/pid/pid_lpf_1.h, 195, 197
- include/pid/pid_lpf_1_ff.h, 198, 200
- include/pid/pid_p.h, 201, 203
- include/pid/pid_s.h, 204, 206
- include/proportional Directory Reference, 22
- include/proportional/p.h, 207, 209
- include/utility Directory Reference, 23
- include/utility/utility.h, 210, 212
- init
- control_system::derivative::D< T >, 31
 - control_system::derivative::D_LPF_1< T >, 39
 - control_system::integral::I< T >, 47
 - control_system::pd::PD< T >, 55
 - control_system::pd::PD_LPF_1< T >, 64
 - control_system::pi::PI< T >, 73
 - control_system::pid::PID< T >, 83
 - control_system::pid::PID_GS< T >, 97
 - control_system::pid::PID_LPF_1< T >, 114
 - control_system::pid::PID_LPF_1_FF< T >, 126
 - control_system::pid::PID_P< T >, 140
 - control_system::pid::PID_S< T >, 156
 - control_system::proportional::P< T >, 167
- integral, 27
- interpolate
- control_system::pid::PID_GS< T >, 98
- Kd
- control_system::derivative::D< T >, 35
 - control_system::derivative::D_LPF_1< T >, 43
 - control_system::pd::PD< T >, 59
 - control_system::pd::PD_LPF_1< T >, 69
 - control_system::pid::PID< T >, 89
 - control_system::pid::PID_GS< T >, 106
 - control_system::pid::PID_LPF_1< T >, 119
 - control_system::pid::PID_LPF_1_FF< T >, 132
 - control_system::pid::PID_P< T >, 148
- Kd_arr
- control_system::pid::PID_GS< T >, 106
- Ki
- control_system::integral::I< T >, 51
 - control_system::pi::PI< T >, 78
 - control_system::pid::PID< T >, 89
 - control_system::pid::PID_GS< T >, 106
 - control_system::pid::PID_LPF_1< T >, 119
 - control_system::pid::PID_LPF_1_FF< T >, 132
 - control_system::pid::PID_P< T >, 148
- Ki_arr
- control_system::pid::PID_GS< T >, 107
- Kp
- control_system::pd::PD< T >, 60
 - control_system::pd::PD_LPF_1< T >, 69
 - control_system::pi::PI< T >, 78
 - control_system::pid::PID< T >, 89
 - control_system::pid::PID_GS< T >, 107
 - control_system::pid::PID_LPF_1< T >, 119
 - control_system::pid::PID_LPF_1_FF< T >, 132
 - control_system::pid::PID_P< T >, 148
 - control_system::pid::PID_S< T >, 163
 - control_system::proportional::P< T >, 170
- Kp_arr
- control_system::pid::PID_GS< T >, 107
- len_arr
- control_system::pid::PID_GS< T >, 107
- lpf
- control_system::pid::PID_GS< T >, 107
 - control_system::pid::PID_P< T >, 148
 - control_system::pid::PID_S< T >, 164
- merge
- control_system::derivative::D< T >, 32
 - control_system::derivative::D_LPF_1< T >, 40
 - control_system::integral::I< T >, 48
 - control_system::pd::PD< T >, 56
 - control_system::pd::PD_LPF_1< T >, 65
 - control_system::pi::PI< T >, 74
 - control_system::pid::PID< T >, 84
 - control_system::pid::PID_GS< T >, 98
 - control_system::pid::PID_LPF_1< T >, 114
 - control_system::pid::PID_LPF_1_FF< T >, 127
 - control_system::pid::PID_P< T >, 141
 - control_system::pid::PID_S< T >, 157
- op_cond_arr
- control_system::pid::PID_GS< T >, 107

P

control_system::pid::PID_GS< T >, 108
 control_system::pid::PID_P< T >, 148
 control_system::pid::PID_S< T >, 164
 control_system::proportional::P< T >, 167

PD

control_system::pd::PD< T >, 54

pd, 27

PD_LPF_1

control_system::pd::PD_LPF_1< T >, 62

PI

control_system::pi::PI< T >, 72

pi, 27

PID, 5

control_system::pid::PID< T >, 81

pid, 27

PID Control Library, 1

PID_GS

control_system::pid::PID_GS< T >, 93

PID_LPF_1

control_system::pid::PID_LPF_1< T >, 111

PID_LPF_1_FF

control_system::pid::PID_LPF_1_FF< T >, 123

PID_P

control_system::pid::PID_P< T >, 137

PID_S

control_system::pid::PID_S< T >, 152

proportional, 27

README.md, 212

reset

control_system::derivative::D< T >, 32
 control_system::derivative::D_LPF_1< T >, 40
 control_system::integral::I< T >, 48
 control_system::pd::PD< T >, 56
 control_system::pd::PD_LPF_1< T >, 65
 control_system::pi::PI< T >, 74
 control_system::pid::PID< T >, 84
 control_system::pid::PID_GS< T >, 99
 control_system::pid::PID_LPF_1< T >, 115
 control_system::pid::PID_LPF_1_FF< T >, 127
 control_system::pid::PID_P< T >, 142
 control_system::pid::PID_S< T >, 157
 control_system::proportional::P< T >, 168

saturate

utility.h, 211

set_d_filter

control_system::pid::PID_GS< T >, 99
 control_system::pid::PID_P< T >, 142
 control_system::pid::PID_S< T >, 158

set_dt

control_system::derivative::D< T >, 32
 control_system::derivative::D_LPF_1< T >, 40
 control_system::integral::I< T >, 48
 control_system::pd::PD< T >, 56
 control_system::pd::PD_LPF_1< T >, 66
 control_system::pi::PI< T >, 75
 control_system::pid::PID< T >, 85

control_system::pid::PID_GS< T >, 99
 control_system::pid::PID_LPF_1< T >, 115
 control_system::pid::PID_LPF_1_FF< T >, 127
 control_system::pid::PID_P< T >, 142
 control_system::pid::PID_S< T >, 158

set_fc

control_system::derivative::D_LPF_1< T >, 41
 control_system::pd::PD_LPF_1< T >, 66
 control_system::pid::PID_GS< T >, 100
 control_system::pid::PID_LPF_1< T >, 115
 control_system::pid::PID_LPF_1_FF< T >, 128
 control_system::pid::PID_P< T >, 142
 control_system::pid::PID_S< T >, 158

set_ff

control_system::pid::PID_GS< T >, 100
 control_system::pid::PID_LPF_1_FF< T >, 128
 control_system::pid::PID_P< T >, 143
 control_system::pid::PID_S< T >, 159

set_gain_lookup_table

control_system::pid::PID_GS< T >, 100

set_I_max

control_system::pid::PID_GS< T >, 101
 control_system::pid::PID_P< T >, 143
 control_system::pid::PID_S< T >, 159

set_Kd

control_system::derivative::D< T >, 33
 control_system::derivative::D_LPF_1< T >, 41
 control_system::pd::PD< T >, 57
 control_system::pd::PD_LPF_1< T >, 66
 control_system::pid::PID< T >, 85
 control_system::pid::PID_GS< T >, 101
 control_system::pid::PID_LPF_1< T >, 116
 control_system::pid::PID_LPF_1_FF< T >, 128
 control_system::pid::PID_P< T >, 143

set_Ki

control_system::integral::I< T >, 49
 control_system::pi::PI< T >, 75
 control_system::pid::PID< T >, 85
 control_system::pid::PID_GS< T >, 102
 control_system::pid::PID_LPF_1< T >, 116
 control_system::pid::PID_LPF_1_FF< T >, 129
 control_system::pid::PID_P< T >, 144

set_Kp

control_system::pd::PD< T >, 57
 control_system::pd::PD_LPF_1< T >, 67
 control_system::pi::PI< T >, 75
 control_system::pid::PID< T >, 86
 control_system::pid::PID_GS< T >, 102
 control_system::pid::PID_LPF_1< T >, 116
 control_system::pid::PID_LPF_1_FF< T >, 129
 control_system::pid::PID_P< T >, 144
 control_system::pid::PID_S< T >, 159
 control_system::proportional::P< T >, 168

set_param

control_system::derivative::D< T >, 33
 control_system::derivative::D_LPF_1< T >, 42
 control_system::integral::I< T >, 49
 control_system::pd::PD< T >, 58

- control_system::pd::PD_LPF_1< T >, 67
- control_system::pi::PI< T >, 76
- control_system::pid::PID< T >, 86
- control_system::pid::PID_GS< T >, 102
- control_system::pid::PID_LPF_1< T >, 117
- control_system::pid::PID_LPF_1_FF< T >, 129
- control_system::pid::PID_P< T >, 144
- control_system::pid::PID_S< T >, 160
- control_system::proportional::P< T >, 168
- set_Td
 - control_system::pid::PID_S< T >, 160
- set_Ti
 - control_system::pid::PID_S< T >, 161
- set_u_0
 - control_system::integral::I< T >, 50
 - control_system::pi::PI< T >, 76
 - control_system::pid::PID< T >, 87
 - control_system::pid::PID_LPF_1< T >, 117
 - control_system::pid::PID_LPF_1_FF< T >, 130
- set_u_max
 - control_system::derivative::D< T >, 33
 - control_system::derivative::D_LPF_1< T >, 42
 - control_system::integral::I< T >, 50
 - control_system::pd::PD< T >, 58
 - control_system::pd::PD_LPF_1< T >, 68
 - control_system::pi::PI< T >, 77
 - control_system::pid::PID< T >, 87
 - control_system::pid::PID_GS< T >, 103
 - control_system::pid::PID_LPF_1< T >, 118
 - control_system::pid::PID_LPF_1_FF< T >, 131
 - control_system::pid::PID_P< T >, 145
 - control_system::pid::PID_S< T >, 161
 - control_system::proportional::P< T >, 169
- src Directory Reference, 23
- src/derivative Directory Reference, 16
- src/derivative/d.tpp, 212, 213
- src/derivative/d_lpf_1.tpp, 215
- src/integral Directory Reference, 18
- src/integral/i.tpp, 217, 218
- src/pd Directory Reference, 19
- src/pd/pd.tpp, 219, 220
- src/pd/pd_lpf_1.tpp, 222
- src/pi Directory Reference, 20
- src/pi/pi.tpp, 224, 225
- src/pid Directory Reference, 21
- src/pid/pid.tpp, 227
- src/pid/pid_gs.tpp, 230
- src/pid/pid_lpf_1.tpp, 234, 235
- src/pid/pid_lpf_1_ff.tpp, 237, 238
- src/pid/pid_p.tpp, 241
- src/pid/pid_s.tpp, 245
- src/proportional Directory Reference, 22
- src/proportional/p.tpp, 249
- start
 - control_system::derivative::D< T >, 35
 - control_system::derivative::D_LPF_1< T >, 44
 - control_system::pd::PD< T >, 60
 - control_system::pd::PD_LPF_1< T >, 69
 - control_system::pi::PI< T >, 78
 - control_system::pid::PID< T >, 89
 - control_system::pid::PID_GS< T >, 108
 - control_system::pid::PID_LPF_1< T >, 120
 - control_system::pid::PID_LPF_1_FF< T >, 133
 - control_system::pid::PID_P< T >, 148
 - control_system::pid::PID_S< T >, 164
- T_d
 - control_system::pid::PID_S< T >, 164
- T_i
 - control_system::pid::PID_S< T >, 164
- tau
 - control_system::derivative::D_LPF_1< T >, 44
 - control_system::pd::PD_LPF_1< T >, 69
 - control_system::pid::PID_LPF_1< T >, 120
 - control_system::pid::PID_LPF_1_FF< T >, 133
- u
 - control_system::pid::PID_GS< T >, 108
 - control_system::pid::PID_P< T >, 149
 - control_system::pid::PID_S< T >, 165
- u_ff
 - control_system::pid::PID_GS< T >, 108
 - control_system::pid::PID_LPF_1_FF< T >, 133
 - control_system::pid::PID_P< T >, 149
 - control_system::pid::PID_S< T >, 165
- u_k_1
 - control_system::derivative::D_LPF_1< T >, 44
 - control_system::integral::I< T >, 51
 - control_system::pd::PD_LPF_1< T >, 70
 - control_system::pi::PI< T >, 78
 - control_system::pid::PID< T >, 89
 - control_system::pid::PID_LPF_1< T >, 120
 - control_system::pid::PID_LPF_1_FF< T >, 133
- u_k_2
 - control_system::pid::PID_LPF_1< T >, 120
 - control_system::pid::PID_LPF_1_FF< T >, 133
- u_max
 - control_system::derivative::D< T >, 35
 - control_system::derivative::D_LPF_1< T >, 44
 - control_system::integral::I< T >, 52
 - control_system::pd::PD< T >, 60
 - control_system::pd::PD_LPF_1< T >, 70
 - control_system::pi::PI< T >, 79
 - control_system::pid::PID< T >, 90
 - control_system::pid::PID_GS< T >, 108
 - control_system::pid::PID_LPF_1< T >, 120
 - control_system::pid::PID_LPF_1_FF< T >, 134
 - control_system::pid::PID_P< T >, 149
 - control_system::pid::PID_S< T >, 165
 - control_system::proportional::P< T >, 170
- update
 - control_system::derivative::D< T >, 34
 - control_system::derivative::D_LPF_1< T >, 43
 - control_system::integral::I< T >, 50
 - control_system::pd::PD< T >, 59
 - control_system::pd::PD_LPF_1< T >, 68
 - control_system::pi::PI< T >, 77

- control_system::pid::PID< T >, [87](#)
- control_system::pid::PID_GS< T >, [104](#)
- control_system::pid::PID_LPF_1< T >, [118](#)
- control_system::pid::PID_LPF_1_FF< T >, [131](#)
- control_system::pid::PID_P< T >, [146](#)
- control_system::pid::PID_S< T >, [161](#)
- control_system::proportional::P< T >, [169](#)
- update_gain
 - control_system::pid::PID_GS< T >, [104](#)
- utility.h
 - saturate, [211](#)